



MCS220 Web Technologies

Block 1: Web Application Development using J2EE

Unit -1 Introduction to J2EE, Architecture and Design pattern

Topics to be Covered

Web Server & Web Container

Introduction to J2ee

Design Patters

MVC

Repository Design pattern

Singleton

Factory

Building java Application JAR and WAR and deployment in tomcat

Web Server & Web Container

Web server is a computer where the web content is stored.
Basically web server is used to host the web sites but there exists other web servers also such as gaming, storage, FTP, email etc.



Web Server & Web Container

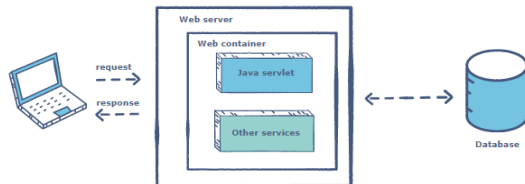
A Web Container is the component of an internet server that interacts with Java servlets.

An internet container manages the life cycle of servlets; it maps a URL to a specific servlet while ensuring that the requester has relevant access-rights.

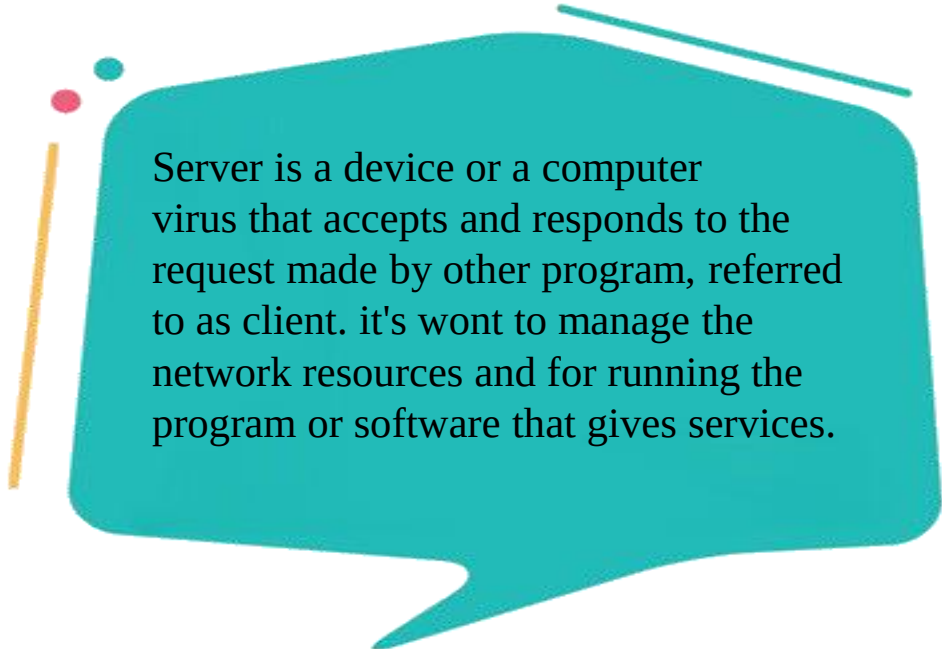
Web Server & Web Container

The web container implements the web component aspect of the Java engineering architecture.

It specifies a run time environment for various components such as security, concurrency, transaction, and deployment.

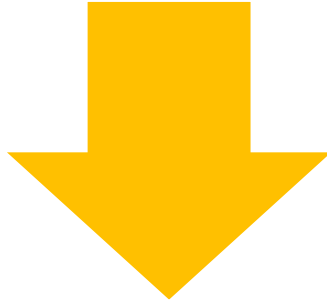


Web Server & Web Container



Server is a device or a computer virus that accepts and responds to the request made by other program, referred to as client. it's wont to manage the network resources and for running the program or software that gives services.

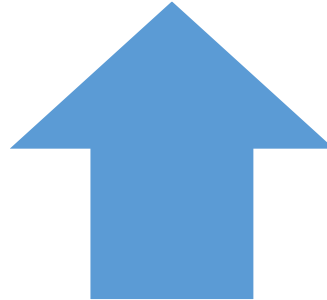
Web Server & Web Container



Web Server



Application
Server



Web Server & Web Container

Web server contains only web or servlet container. It are often used for servlet, jsp, struts, jsf etc.

It cannot be used for EJB.

It is a computer where the online content are often stored. generally web server are often wont to host the online sites but there also used another web servers also like FTP, email, storage, gaming etc.

Web Servers are: Apache Tomcat and Resin.

Web Server & Web Container

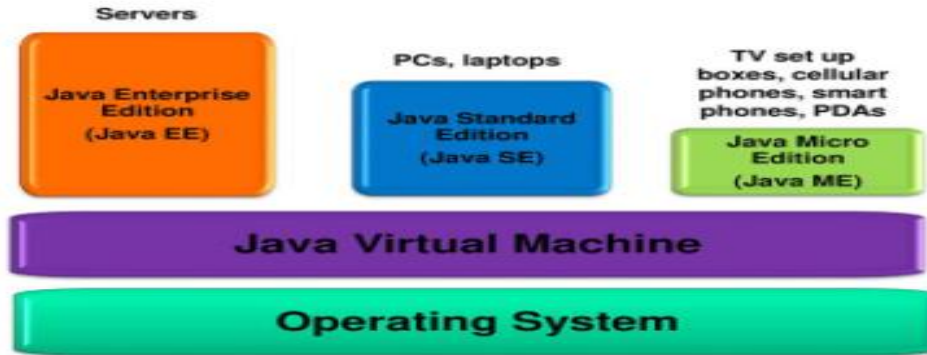
Both web and application servers answer user requests for content.

A web server may be a computing system that stores, processes, and delivers sites to clients.

The client is nearly always an internet browser or a mobile application. counting on the setup, an internet server can store one or more websites.

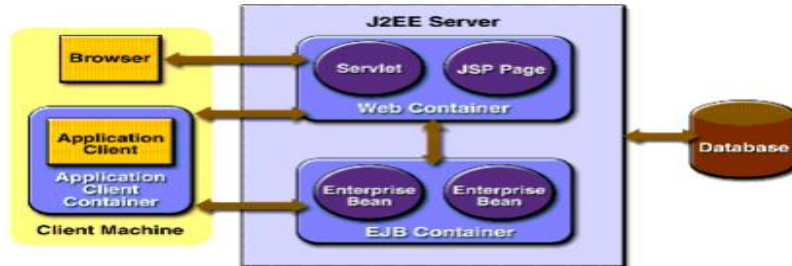
Introduction to J2ee

Open and standard based platform for developing, deploying and managing n-tier, Web-enabled, server-centric, and component-based enterprise applications

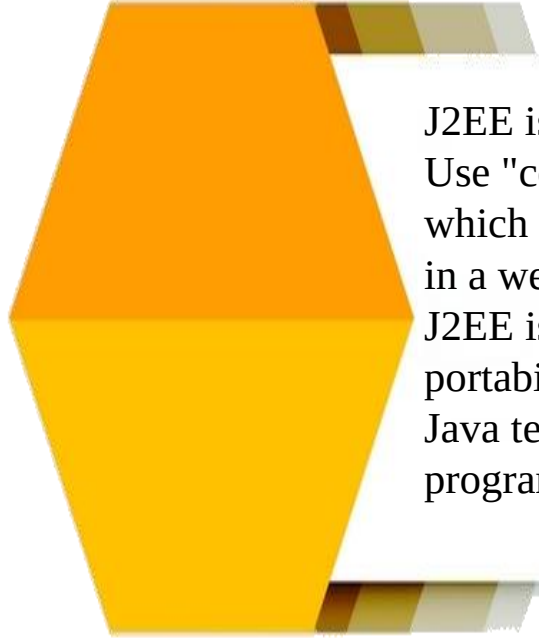


Introduction to J2ee

J2EE is that standard that also provides portability of code because it is based on Java technology and standard based Java programming APIs



Introduction to J2ee Open Standard



J2EE is a Open and Standard Solution
Use "component and container" model in
which container provides system services
in a well-defined and as industry standard.
J2EE is that standard that also provides
portability of code because it is based on
Java technology and standard based Java
programming APIs.

Introduction to J2ee

J2EE server



The runtime portion of a J2EE product.
A J2EE server provides EJB and web containers.

Introduction to J2ee

Enterprise JavaBeans (EJB) container

Manages the execution of enterprise beans for J2EE applications. Enterprise beans and their container run on the J2EE server.

Web container

Manages the execution of JSP page and servlet components for J2EE applications. web components and their container run on the J2EE server.

Introduction to J2ee

Application client container

Manages the execution of application client components.

Application clients and their container run on the client.

Applet container

Manages the execution of applets.
Consists of a web browser and Java Plug-in running on the client together.

Introduction to J2ee J2EE Development Life Cycle

Develop components for

- Persistence (Entities)
- Business Logic (EJBs)
- Presentation (JSF, XHTML)
- Configure/Prepare deployment descriptors

Build Components in packages (JARs, WARs, EARs)

Deploy packages on Container (J2EE Server)

Design Patterns

- They are reusable in multiple projects.
- They provide the solutions that help to define the system architecture.
- They capture the software engineering experiences.
- They provide transparency to the design of an application.
- They are well-proved and testified solutions since they have been built upon the knowledge and experience of expert software developers.
- Design patterns don't guarantee an absolute solution to a problem.
- They provide clarity to the system architecture and the possibility of building a better system.

Design Patterns SDLC

Design patterns are categorized into two parts:

Core Java (or JSE) Design Patterns.

JEE Design Patterns.

Design Patterns MVC

What is MVC Design Pattern?

MVC design pattern is additionally referred to as Model-View-Controller.

It's a standard architectural pattern that's wont to design and make interfaces and therefore the structure of an application.

Design Patterns MVC

This pattern divides the appliance into three parts that are dependent and connected to every other. These designs are wont to distinguish the presentation of knowledge from how the info is accepted from the user to the info shown. These design patterns became common within the use of web applications and for developing GUIs.

Design Patterns MVC

Controller actually contains both View and Model.

Each of the components holds a different responsibilities where:

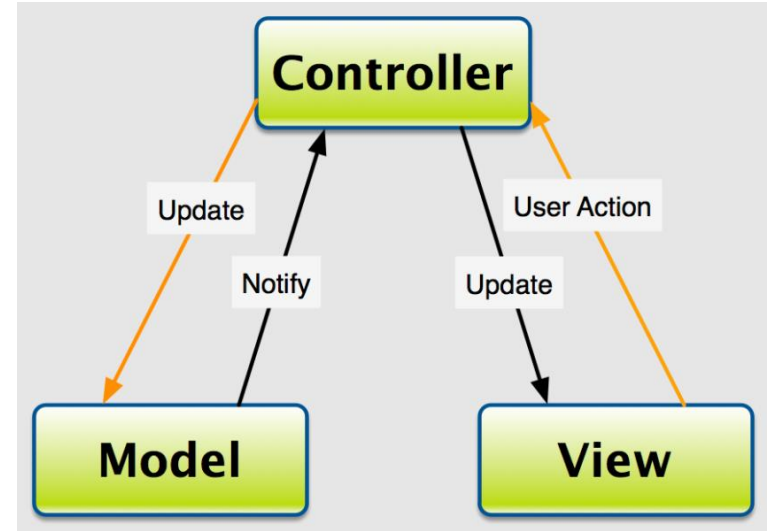
Models hold application data.

Views display visual elements and is usually related to UI.

Design Patterns MVC

1. Model

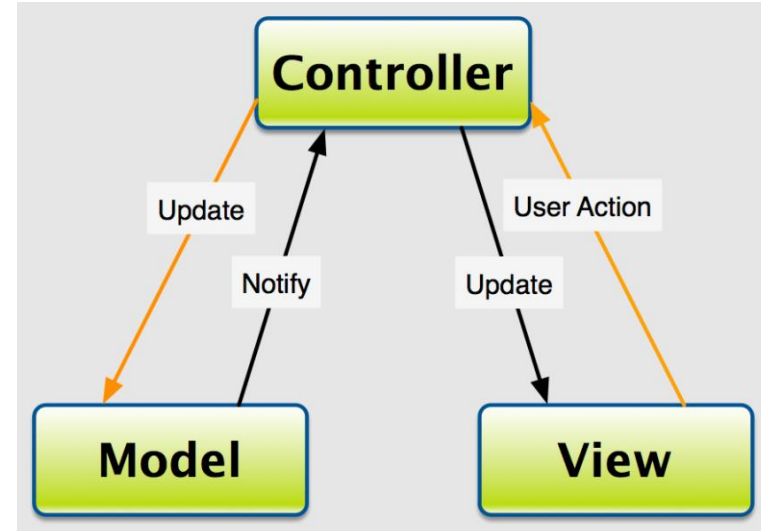
This a part of the planning pattern is that the primary part and contains application information purely. It doesn't contain any information on the way to show the info to the user. it's independent of the interface . It controls the logic and rules of application.



Design Patterns MVC

2. View

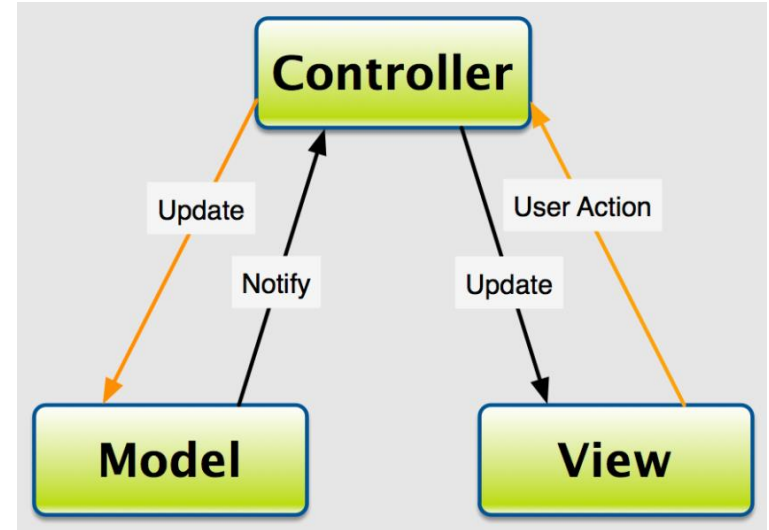
This part helps the user to ascertain the model's data. the most concern of this part is to access the model's data. The view section uses a chart, table or diagrams to represent the knowledge . It also can show similar data and use bar graphs and tables for various purposes. it's a visualization of data that the appliance contains.



Design Patterns MVC

3. Controller

Most of the work **is completed** by the controller. It provides the support for input and converts the input to commands for **the appliance** . **it's** used between the model **and consider** part. The model **and therefore the** view are interconnected, **therefore the** execution is reflected **within the** view part.



Repository Design pattern

The Repository pattern implements separation of concerns by abstracting the data persistence logic in your applications. Design patterns are used as a solution to recurring problems in your applications, and the Repository pattern is one of the most widely used design patterns.

Design Pattern Singleton

Define a class that has only one instance and provides a global point of access to it".

There are two forms of singleton design pattern

Early Instantiation: creation of instance at load time.

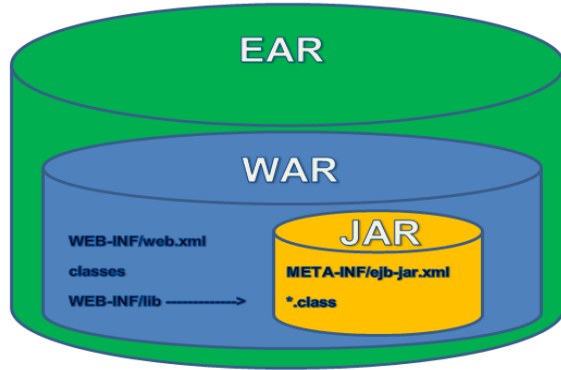
Lazy Instantiation: creation of instance when required.

Design Pattern Factory

A Factory Pattern or Factory Method Pattern says that just define an interface or abstract class for creating an object but let the subclasses decide which class to instantiate. In other words, subclasses are responsible to make the instance of the category . The Factory Method Pattern is additionally referred to as Virtual Constructor.

Building java Application JAR and WAR and deployment in tomcat

war file is a Web Application Archive which runs inside an application server while a .jar is Java Application Archive that runs a desktop application on a user's machine



Building java Application JAR and WAR and deployment in tomcat

Java Web application or J2EE web application is a terminology to call the application developed with J2EE web components and designed to run on the web container of any J2EE Application server.

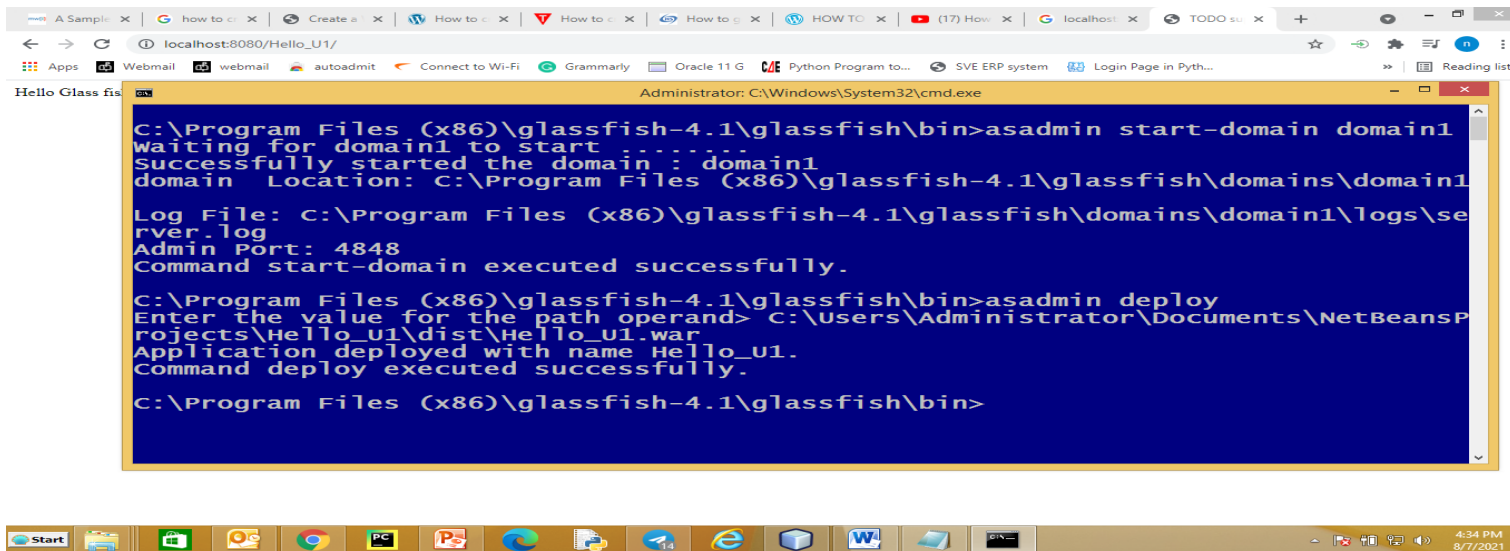
J2EE has various components and services and the J2EE components can be categorized into two major groups

Web Components (HTML, JSF, Servlet, JSP)

EJB Components (EJB, EDB, SDB, MDB)

The web application or a WAR is a Collective of these J2EE defined Web Components with some Standard Directory Structure and formats.

Building java Application JAR and WAR and deployment in tomcat



The screenshot shows a web browser window at the top with the address bar displaying 'localhost:8080/Hello_U1/'. Below the browser, a Windows command prompt window is open, showing the following commands and output:

```
C:\Program Files (x86)\glassfish-4.1\glassfish\bin>asadmin start-domain domain1
waiting for domain1 to start .....
Successfully started the domain : domain1
domain Location: C:\Program Files (x86)\glassfish-4.1\glassfish\domains\domain1

Log File: C:\Program Files (x86)\glassfish-4.1\glassfish\domains\domain1\logs\server.log
Admin Port: 4848
Command start-domain executed successfully.

C:\Program Files (x86)\glassfish-4.1\glassfish\bin>asadmin deploy
Enter the value for the path operand> C:\Users\Administrator\Documents\NetBeansProjects\Hello_U1\dist\Hello_U1.war
Application deployed with name Hello_U1.
Command deploy executed successfully.

C:\Program Files (x86)\glassfish-4.1\glassfish\bin>
```

Important Topics

- A Web Container is the component of an internet server that interacts with Java servlets.
- The web container implements the web component aspect of the Java engineering architecture.
- A design patterns are **well-proved solution** for solving the specific problem/task.
- The Repository pattern implements separation of concerns by abstracting the data persistence logic in your applications.
- .

Summary

- They are reusable in multiple projects.
 - They provide the solutions that help to define the system architecture.
 - They capture the software engineering experiences.
 - They provide transparency to the design of an application.
 - They are well-proved and testified solutions since they have been built upon the knowledge and experience of expert software developers.
 - Design patterns don't guarantee an absolute solution to a problem.
 - They provide clarity to the system architecture and the possibility of building a better system.
 - .
- 9319133134 query@cftedu.in

*Thank
You*



MCS 220 Web Technologies

Block 1: Web Application Development using J2EE

Unit -2 Basics of Servlet

Topics to be Covered

Introduction to Servlets

Http Protocol & Http Methods

Servlet Architecture

Servlet Life Cycle

Creating a Servlet

Servlet Communication(Servlet- Browser and Web-component)

Introduction to Servlets

A **servlet** is a Java programming language class that is used to extend the capabilities of servers that host applications accessed by means of a request-response programming model

Introduction to Servlets

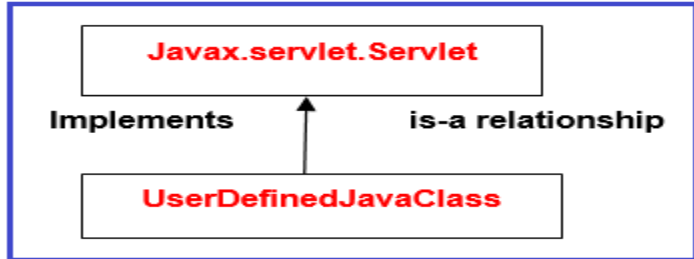
A Web Container is the component of an internet server that interacts with Java servlets.

An internet container manages the life cycle of servlets; it maps a URL to a specific servlet while ensuring that the requester has relevant access-rights.

Introduction to Servlets

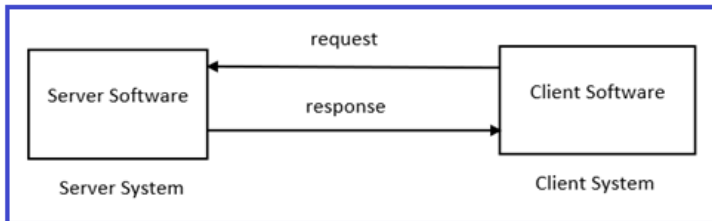
A servlet is a user-defined public java Class that implements **javax.servlet.Servlet** interface and it is managed by the servlet container.

- 1.A servlet is a **Dynamic Web Resource** that enhances the functionality of the webserver.
- 2.A servlet is a **Web Server-Side Piece of Code (server-side program)**.
- 3.A servlet is a **Web Component**.



Http Protocol & Http Methods

A protocol is a set of standard rules which must be followed by two systems in order to make communication possible.



Http Protocol & Http Methods

method	Request resource	Protocol/version
--------	------------------	------------------

When the client sends a request, then that request will be converted into an HTTP request format and send it to the server.

Http Protocol & Http Methods

HTTP Request Headers

User-Agent: It contains information about the version of browser/ client software.

Accept-Language: It contains information about what language is used by the client.



Servlet Architecture

Servlets are very robust and scalable.
Before introducing servlets, CGI (common gateway interface) was used.

Servlet Architecture

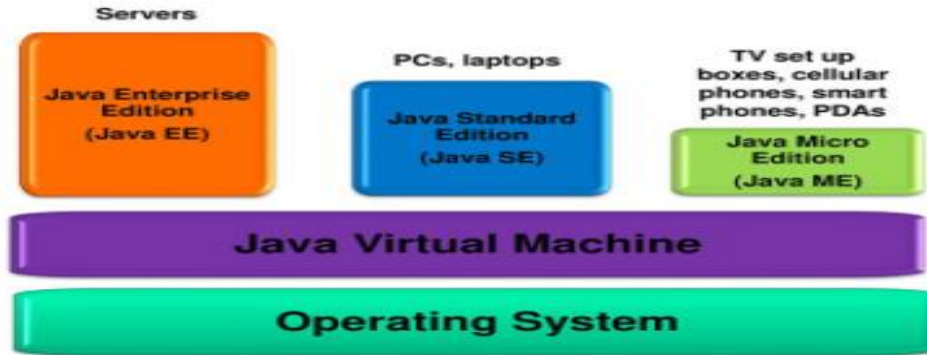
Both web and application servers answer user requests for content.

A web server may be a computing system that stores, processes, and delivers sites to clients.

The client is nearly always an internet browser or a mobile application. counting on the setup, an internet server can store one or more websites.

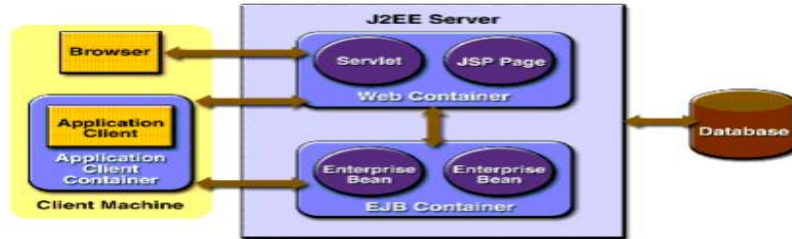
Servlet Architecture

Open and standard based platform for developing, deploying and managing n-tier, Web-enabled, server-centric, and component-based enterprise applications



Servlet Life Cycle

J2EE is that standard provides portability of code because it is based on Java technology and standard based Java programming APIs



Servlet Life Cycle – J2EE

J2EE is a Open and Standard Solution
Use "component and container" model in
which container provides system services
in a well-defined and as industry standard.
J2EE is a standard that also provides
portability of code because it is based on
Java technology and standard based Java
programming APIs.

Servlet Life Cycle

J2EE server



The runtime portion of a J2EE product.
A J2EE server provides EJB and web containers.

Creating a Servlet

Enterprise JavaBeans (EJB) container

Manages the execution of enterprise beans for J2EE applications. Enterprise beans and their container run on the J2EE server.

Web container

Manages the execution of JSP page and servlet components for J2EE applications. web components and their container run on the J2EE server.

Servlet Communication (Servlet- Browser and Web-component)

Application client container

Manages the execution of application client components.

Application clients and their container run on the client.

Applet container

Manages the execution of applets.

Consists of a web browser and Java Plug-in running on the client together.

Servlet Communication (Servlet- Browser and Web-component)



- Develop components for
 - Persistence (Entities)
 - Business Logic (EJBs)
 - Presentation (JSF, XHTML)
- Configure/Prepare deployment descriptors
- Build Components in packages (JARs, WARs, EARs)
- Deploy packages on Container (J2EE Server)

Servlet Communication (Servlet- Browser and Web-component)



They are reusable in multiple projects.

They provide the solutions that help to define the system architecture.

They capture the software engineering experiences.

They provide transparency to the design of an application.

They are well-proved and testified solutions since they have been built upon the knowledge and experience of expert software developers.

Design patterns don't guarantee an absolute solution to a problem.

They provide clarity to the system architecture and the possibility of building a better system.

Servlet Communication (Servlet- Browser and Web-component)

Design patterns are categorized into two parts:
Core Java (or JSE) Design Patterns.
JEE Design Patterns.

Servlet Communication (Servlet- Browser and Web-component)

The container loads the servlet (if it isn't already loaded), instantiates a request and response objects, and then hands these objects to the servlet by calling first its `init()` method, then its `service()` method, and lastly the `destroy()` method.

The `service()` method will typically call one of the `doXXX()` methods such as `doGet()`.

Important Topics

A servlet is a Java programming language class that is used to extend the capabilities of servers that host applications accessed by means of a request-response programming model.

A Web Container is the component of an internet server that interacts with Java servlets.

A servlet is a Dynamic Web Resource that enhances the functionality of the webserver

A protocol is a set of standard rules which must be followed by two systems in order to make communication possible.

Summary

- Servlet architecture comes under a java programming language used to create dynamic web applications.
- Mainly servlets are used to develop server-side applications. Servlets are very robust and scalable.
- Before introducing servlets, CGI (common gateway interface) was used.
- Servlets are used to perform client request and response tasks dynamically. Servlet architecture comes under a java programming language used to create dynamic web applications.
- Mainly servlets are used to develop server-side applications.
- Servlets are very robust and scalable.
- Before introducing servlets, CGI (common gateway interface) was used.

Thank
You



MCS 220 Web Technologies

Block 1: Web Application Development using J2EE

Unit -3 Session Management and Database Connectivity in Servlet

Topics to be Covered

Session Management

Database Connectivity in servlet,

Servlet Communication(Servlet- Browser, Web-component)

Servlet Collaboration

Servlet Session Management

Database Connectivity

Session Management

Session management refers to tracking the state of an end user across Web pages. Session management enables programmers to create applications where the state of an end user is required to be maintained across multiple Web pages.

Session Management

HTTP and Web Servers are both stateless. Hence, the only way to maintain the state of the user is by making use of technologies that implement session tracking.

Session Management

```
public HttpSession getSession(boolean create)
public String getId()
public long getCreationTime()
public long getLastAccessedTime()
public void invalidate()
```

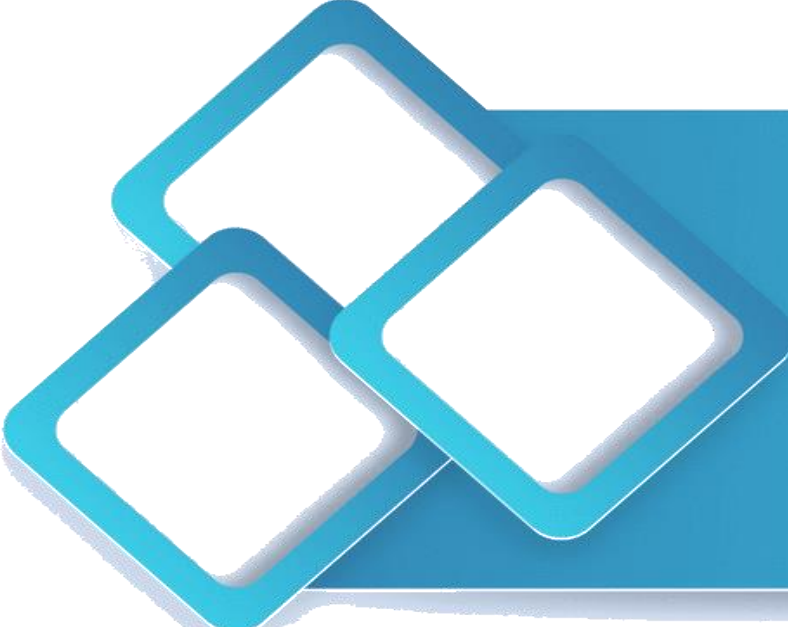
Database Connectivity in servlet

The term JDBC stands for **Java Database Connectivity**.

JDBC is a standard Java API for database and used for connecting the wide range database and the Java programming language.

In servlet connecting to the database is an important task why because while dealing with big projects more databases will be used.

Database Connectivity in servlet



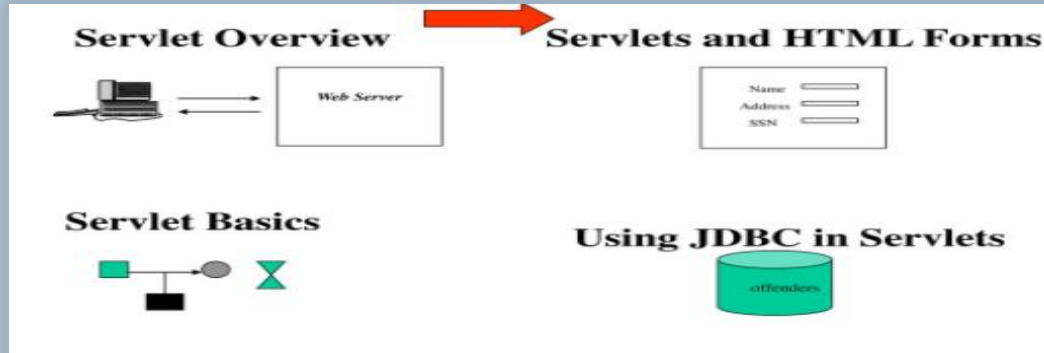
By using JDBC, one can connect to a database.

One can create SQL or MYSQL statements using JDBC.

The records can be viewed and modified using JDBC.

The SQL or MYSQL queries can be executed in the database using JDBC.

Database Connectivity in servlet



Database Connectivity in servlet

```
+ import ...12 lines
public class ConnectorServlet_JDBC extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        try{
            doProcess(request,response);
        }catch(SQLException e){}}
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        try{
            doProcess(request,response);
        }catch(SQLException e){}}
    private void doProcess(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException , SQLException{
        Connection conn = null;
        Statement stmt = null;
        @SuppressWarnings("UnusedAssignment")
        ResultSet rs = null;
        PrintWriter out = response.getWriter();
        try{
            Class.forName("com.mysql.jdbc.Driver").newInstance();
```

Database Connectivity in servlet

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <web-app version="3.1" xmlns="http://xmlns.jcp.org/xml/ns/javaee" xmlns:xsi="http://www.w3.
3      <servlet>
4          <display-name>dbconnection</display-name>
5          <servlet-name>ConnectorServlet_JDBC</servlet-name>
6          <servlet-class>ConnectorServlet_JDBC</servlet-class>
7      </servlet>
8      <servlet-mapping>
9          <servlet-name>ConnectorServlet_JDBC</servlet-name>
10         <url-pattern>/ConnectorServlet_JDBC</url-pattern>
11     </servlet-mapping>
12
13 </web-app>
```

Database Connectivity in servlet

```
mysql> create database mcs220nk;  
Query OK, 1 row affected (0.17 sec)
```

```
mysql> use mcs220nk;  
Database changed
```

```
mysql> create table userunit3(id int(20) primary key not null auto_increment, name  
e char(20), course varchar(25), address varchar(30));  
Query OK, 0 rows affected (0.27 sec)
```

Database Connectivity in servlet

```
mysql> desc userunit3;
```

Field	Type	Null	Key	Default	Extra
id	int(20)	NO	PRI	NULL	auto_increment
name	char(20)	YES		NULL	
course	varchar(25)	YES		NULL	
address	varchar(30)	YES		NULL	

4 rows in set (0.14 sec)

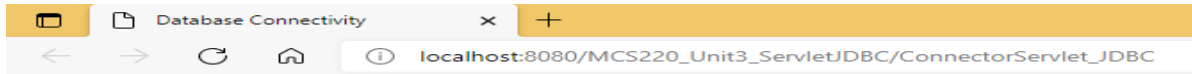
```
mysql> insert into userunit3(name,course,address)values('Aman','MCS220','Pitam p  
ura');  
Query OK, 1 row affected (0.14 sec)
```

```
mysql> insert into userunit3(name,course,address)values('Varun','MCS220','Rohini  
Query OK, 1 row affected (0.13 sec)
```

Database Connectivity in servlet

```
mysql> select * from userunit3;
+----+-----+-----+-----+
| id | name  | course | address |
+----+-----+-----+-----+
| 1  | Aman  | MCS220 | Pitam pura |
| 2  | Varun | MCS220 | Rohini |
+----+-----+-----+-----+
2 rows in set (0.03 sec)

mysql>
```



Database Connectivity in servlet!!

Name : Aman Course : MCS220Address : Pitam pura
Name : Varun Course : MCS220Address : Rohini

Servlet Collaboration

The exchange of information among servlets of a particular Java web application is known as **Servlet Collaboration**.

This enables passing/sharing information from one servlet to the other through method invocations.

Servlet Collaboration

The servlet api provides two interfaces namely:
`javax.servlet.RequestDispatcher`
`javax.servlet.http.HttpServletResponse`

Servlet Collaboration

Using RequestDispatcher Interface

```
public void forward(ServletRequest request,  
ServletResponse response)throws  
ServletException, java.io.IOException
```


Servlet Session Management

Session tracking is way to manage data of a user, this is known as session management in servlet.

Session in Java are managed through different ways, such as, HTTP Session API, Cookies, URL rewriting, etc.

Session management or tracking is an important feature of modern websites that allows server to remember clients.

session is a conversation between server and client, it contains series of continuous requests and responses.

Servlet Session Management

When there are continuous request and responses from same client to server, so the server cannot identify from which client are requests being sent. As HTTP is a Stateless protocol.

Servlet Session Management

User Authorization : user provides Username and password or any authentication credentials from login

URL rewriting: User can append session identifier parameter with every request and response to keep track of session

Hidden Fields: User has access to create unique field in HTML which is hidden, when user starts navigating, user will be able to set the value uniquely to customer and have track over the session

Session Tracking API: This type of session tracking is used for developers to minimize overhead of session tracking.

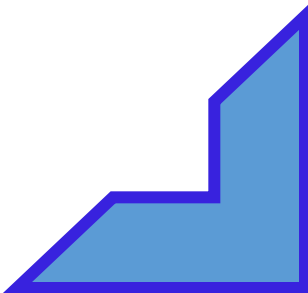
Cookies: Cookie is a key value pair of information sent by server to browsers. It is the most used technology for session tracking.

Servlet Session Management



`request.getSession()` is the method that creates new session object. Container generates new session ID for present session and sends back to client.

`HttpSession session = request.getSession();`
Store information in session object.



Servlet Session Management

isNew(): Returns true if user does not know about the session. If cookies are disabled, then session is new.

getId(): Returns string that contains unique identifier that is assigned to this session. Is used while using URL rewriting the session.

getAttribute(): Returns the object bound in present session.

setAttribute(): Binds object to present session, uses specified name.

invalidate(): Expires current session and unbinds the object binded

setMaxInactiveInterval(): Specifies time between client requests before servlet invalidates session. Negative time indicates session shouldn't timeout.

Servlet Session Management

Servlet Session Management means and the types of Session tracking or management methods such as User Authorization, Hidden fields, HTTPS or SSL, URL Rewriting, Cookies, and Session tracking API.

Database Connectivity

Approach-1:- Writing logics in multiple methods.

Approach-2:- Write all the logics in service(-,-) or doXxx(-,-) method.

Approach-3:- Use server-managed JDBC connection from the connection pool.

Approach-4:- Use DAO class for the persistence operation.

Database Connectivity

Approach-1:- Writing logics in multiple methods.

Create JDBC Connection in the init()

Use the JDBC Connection in the service(-,
)/doXxx(-,-)

Close JDBC Connection in the destroy method.

Here JDBC Connection (con) must be taken as an instance variable of the Servlet Component class.

Database Connectivity

Approach-2:- Write all the logics in service(-,-) or doXxx(-,-) method.

Create JDBC Connection in service(-,-)/doXxx(-,-) method.

Use JDBC Connection in service(-,-)/doXxx(-,-) method.

Close JDBC Connection in service(-,-)/doXxx(-,-) method.

Note:- Here JDBC Connection (con) will be a local variable to service(-,-)/doXxx(-,-) method.

Database Connectivity

Approach-3:- Use server-managed JDBC connection from the connection pool.

Get JDBC Connection object from JDBC

Connection pool being from service(-, -)/doXxx(-, -)

Use JDBC Connection object in service(-, -)/doXxx(-, -)

Return JDBC connection back to JDBC connection pool being from service(-, -)/doXxx(-, -)

Database Connectivity

Approach4:- Use DAO class for the persistence operation.

In this approach, we can use either approach-2 or approach-3 to get the JDBC connection.

The main task in this approach,

Write JDBC code (persistence logic) in DAO class either by using direct connection object or server-managed pooled connection object.

Create a DAO class object in Servlet Component and use its persistence logic in Servlet Component.

Summary

Session management refers to tracking the state of an end user across Web pages.

JDBC is an application programming interface (JDBC API) that characterizes an arrangement of standard operations for connecting with DBMS

Session is a conversation between server and client, it contains series of continuous requests and responses.

*Thank
You*



MCS-220

Unit-6 -Web technologies

Frameworks Available for J2EE Development

Topics to be covered

- Introduction
- Struts: Features
- Model 1 Vs Model 2 (MVC) Architecture
- Model 1 Architecture
- Model 2 (MVC) Architecture
- Struts2 Architecture
- Struts2 Features
- Spring Boot and MVC: features
- Spring MVC features
- Spring Boot features
- Hibernate with JPA: Features
- Compare among these frameworks
- Struts Vs Spring
- Spring Boot Vs Spring MVC
- Spring Vs Hibernate
- Maven: Introduction, Overview and Configuration

Topics to be covered

- Maven Installation
- Maven Core Concepts
- Basic Structure of the POM File
- Maven Build Life Cycles, Phases and Goals
- Maven Build Profiles
- Summary

Introduction

- In this session we discuss the Frameworks Available for J2EE Development.
- A conventional website mainly delivers only static pages, while a web application has the ability to create dynamic responses.
- A web application receives input from users, interacts with the database and executes the business logic to customize the view as a response to the users.

Introduction

- The ***Model-View-Controller (MVC)*** architecture provides the separation of concerns and makes the application easy to maintain and develop. The model represents the business logic, view represents the page design, and controller represents the navigational code.
- The ***Spring MVC and Struts*** are based on MVC architecture which helps developers to create a maintainable web application efficiently.

STRUTS: FEATURES

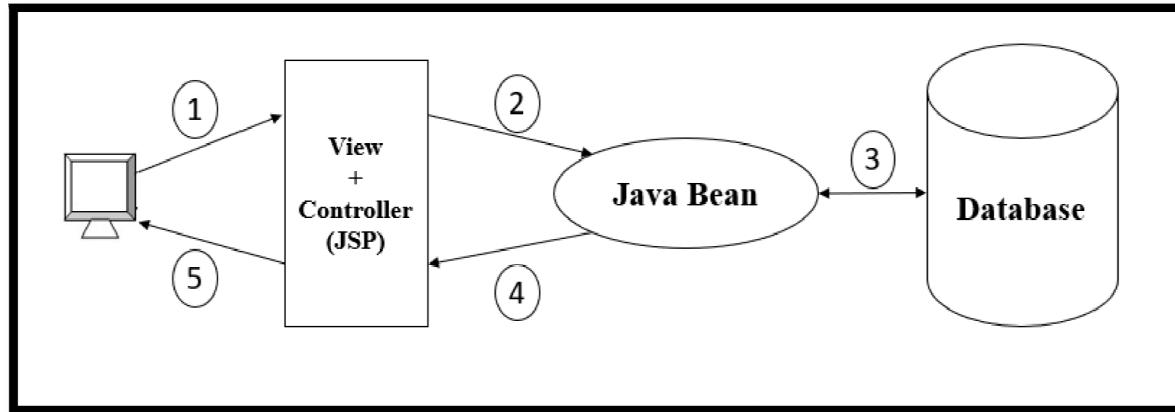
- The Struts framework was developed by Craig McClanahan and he donated it to Apache foundation in May, 2000.
- In June, 2001 Apache released the first version of Struts as Struts 1.0. Struts 2.5.22 is the most current version of the Struts framework in April, 2021.
- Apache Struts2 is an elegant, popular Java Model-View-Controller (MVC) framework to develop MVC based web applications. Struts2 is a complete rewrite of Struts architecture.

Model 1 Vs Model 2 (MVC) Architecture

- The knowledge about design models helps us to decide the architecture of a web application. There are two types of design models.
 - Model 1 Architecture
 - Model 2 (MVC) Architecture
- Model 1 Architecture
 - In earlier days, web application development using Servlet had multiple issues like mixing Business and Presentation logic and recompilation of Servlet if any design code changed.

Model 1 Architecture

- In model 1 architecture, JSP pages were the focal point for the entire web application. JSP provides the solutions to the problems of Servlet technology. JSP provides a better separation of concern not to mix business logic and presentation logic. Re- Deployment of web applications is not required if the JSP page is modified. JSP supports JavaBean, custom tags and JSTL to keep the Business logic separate from JSP.



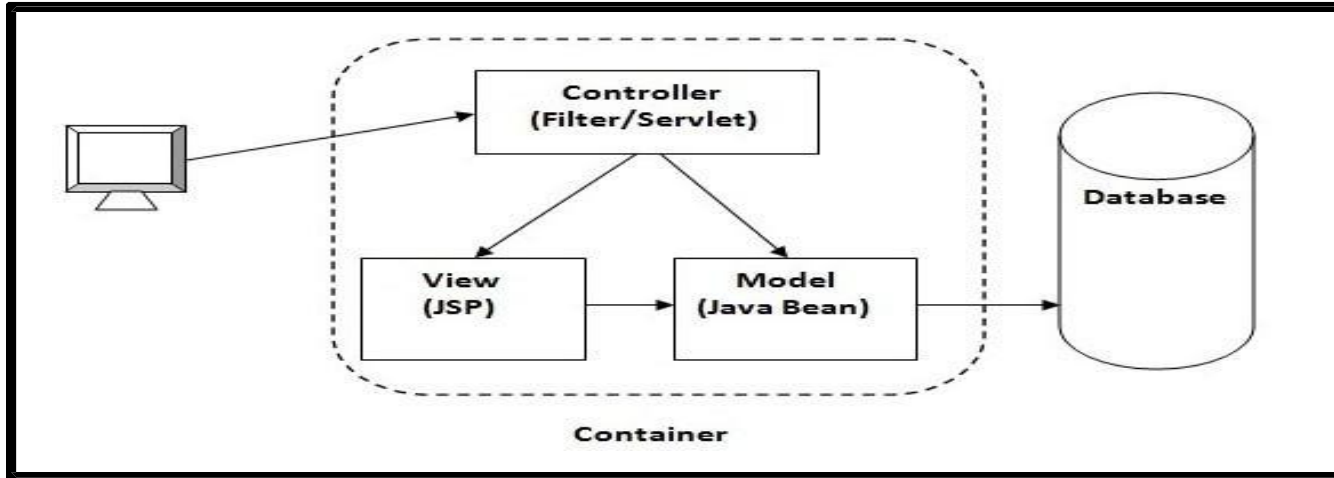
Model 1 Architecture

Model 1 Architecture

- **Advantages**
 - Quick and easy way to develop a web application
- **Disadvantages**
 - **Decentralized navigation Control:** There is no central control to determine the navigation since every JSP page contains the logic to determine the next page.
 - **Hard to maintain:** If a JSP is renamed, then every other JSP which refers to the renamed JSP, needs to be modified.
 - **Time Consuming:** Reusable components like custom tags development requires more time.
 - **Difficult to extend:** Model1 architecture is not suitable for large and complex applications.

Model 2 (MVC) Architecture

- In Model 2 architecture contrast to the Model 1 architecture, a Servlet known as **Controller Servlet** intercepts all client requests. The Controller Servlet performs all the initial request processing and determines which JSP to display next.



Model 2 (MVC) Architecture

Model 2 (MVC) Architecture

- In this architecture, the client never sends requests directly to the JSP.
- This architecture enables us to perform authentication and authorization, centralized logging etc.
- As it can be observed that the key difference between the two architectures is the ***Controller Servlet*** introduced into Model 2, which provides a single point of entry and encourages more reuse and extensibility than the Model 1 architecture.
- The Model 2 architecture clearly provides the separation of business logic, presentation logic, and request processing. This separation is commonly known as ***Model-View-Controller (MVC)*** pattern.

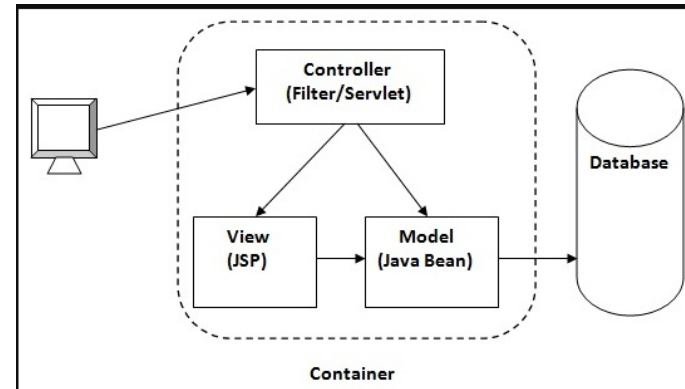
Model 2 (MVC) Architecture

- **Advantages**

- Centralized navigation Control
- Easy to maintain
- Separation of concern
- Easy to extend
- Easy to test

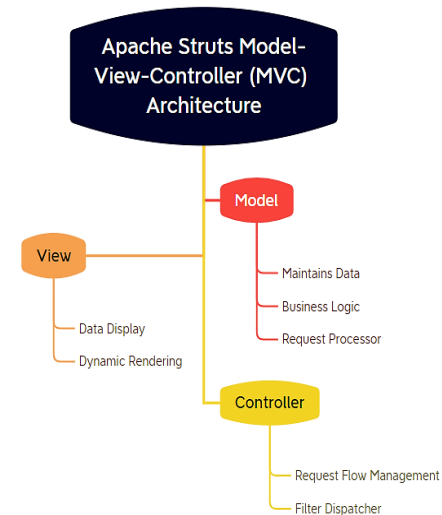
- **Disadvantages**

- Developer writes the Controller Servlet code and it needs to be compiled and redeployed if any logic changes for Controller Servlet

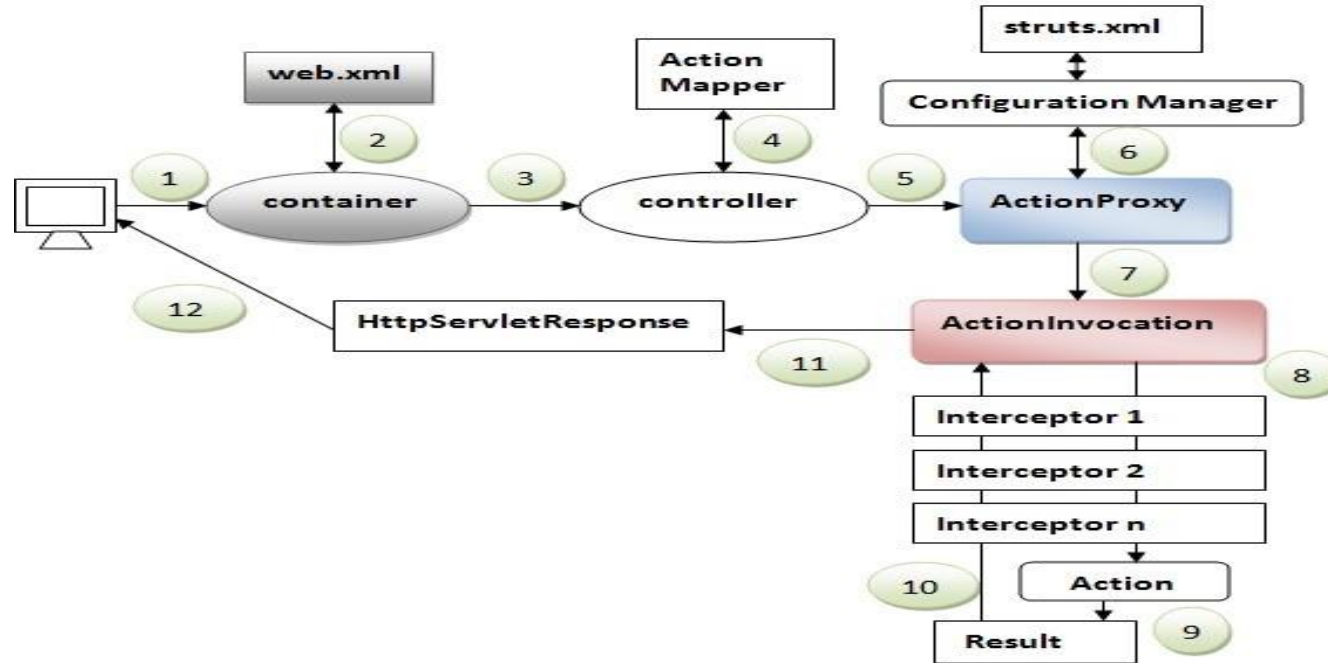


Struts2 Architecture

- Struts2 is a Pull-MVC (MVC2) based architecture, in which the view layer pulls data stored into Value Stack to render. Struts2 implements the Model-View-Controller architecture with the following five components.
 - Interceptors
 - Actions
 - Results/ Result Types
 - Value Stack / OGNL
 - View Technologies
- Struts2 architectural diagram is shown below.



Struts2 Architecture



Struts2 Architecture

Struts2 Features

- The Struts2 is enriched with many important features, and some of them are listed as follows.
 1. **Decoupling of view from action classes:** Struts2 allows us to associate a view with multiple action classes.
 2. **Ready to use view components:** Struts2 provides many ready to use view components such as style sheet driven tags which reduces the lines of code into the application for form design and form validation.
 3. **Intelligent defaults in all configuration files:** One of the important features of the Struts2 is the default value for the configurable properties. Hence, developers do not need to specify the obvious default values into configuration files.

Struts2 Features

4. **Use of OGNL:** Struts2 uses the expression language OGNL, which is more powerful and flexible than JSTL.
5. **Type conversion mechanism:** Struts2 supports any type of properties with the help of OGNL, while in Struts1 all the properties are mostly of string type.
6. **Strong validation support:** Struts2 provides a strong validation mechanism with the support of Xwork validation framework, which provides both client-side and server-side validation. It also supports custom validation with the support of Validate() method.

SPRING BOOT AND MVC: FEATURES

- Spring, Spring MVC and Spring Boot do not compete for the same space; instead, they solve the different problems very well. This section briefs about Spring, Spring MVC and Spring Boot. Later it explains the features of Spring MVC and Spring Boot.

1. Spring MVC features

- The **dependency injection (DI)** is the core concept of the **Spring framework**, which enables the development of loosely coupled and easily unit testable applications. Based on the DI, Spring framework created various modules such as ***Spring MVC***, ***Spring JDBC***, ***Spring Test***, ***Spring ORM***, ***Spring AOP*** etc. These modules do not bring any new functionality since we can do all these with J2EE.

SPRING BOOT AND MVC: FEATURES

- **These modules simply bring in abstraction** and the abstraction aims-
 - Reduce Duplication/ Reduce Boilerplate Code
 - Promote Decoupling/ Increase Unit testability
- Spring Web MVC framework enables us to develop **decoupled web applications**. It introduces the concepts of Dispatcher Servlet, Model And View and View Resolvers to develop loosely coupled web applications. Spring MVC holds many unique features listed below.
 - **Clear separation of responsibility:** The Dispatcher Servlet, controller, handler mapping, view resolver, form object, model object, command object, validation and so on are specialized to perform a single responsibility.

SPRING BOOT AND MVC: FEATURES

- **Adaptability and flexibility:** Spring MVC is very flexible and adaptable. One of the features to define the controller method signature as per need, possibly using one of the parameter annotations such as `@PathVariable`, `@RequestParam`, `@RequestHeader` etc.
- **Reusable business code:** It allows us to reuse the existing business object as a form object or command object instead of creating a duplicate business object.
- **Flexible model transfer:** It supports model transfer as name/ value map, which can be easily integrated with any view technology.

SPRING BOOT AND MVC: FEATURES

- **Customizable handler mapping and view resolution:** Spring Web MVC supports various Handler mapping and View resolution strategies ranging from simple URL-based configuration to sophisticated, purpose-built resolution strategies.
- **Robust JSP form tag library:** Spring 2.0 introduced a powerful JSP form tag library that simplifies the form design in JSP.

Spring Boot features

- Spring MVC requires lots of configurations such as dispatcher servlet, component scan, handler mapping, view resolver and many more.
- When an application uses Hibernate/JPA, it requires configurations such as data source, entity manager, transaction manager etc.
- **Spring Boot** project aims at simplifying and make it easier to develop a Spring based web application. Spring Boot brings intelligence and provides auto configuration features.
- Spring Boot looks at the framework / jars available in the classpath and existing configuration of the application in order to provide the basic configuration needed to configure the application. Features of Spring Boot are as follows.

Spring Boot Features

- The guiding principle of Spring Boot is **convention over configuration**.
- Spring Boot starter simplifies dependency management.
- It also supports the development of stand-alone Spring applications.
- Supports auto-configuration wherever possible.
- It supports production-ready features such as metrics, health checks, and externalized configuration.
- XML configuration is not required any more.
- It supports embedded servers such as Jetty, Tomcat or Undertow.

HIBERNATE WITH JPA: FEATURES

- Object-relational mapping (ORM, O/RM, and O/R mapping tool) in computer science is a programming technique for converting data between incompatible type systems using object-oriented programming languages. ORM makes it possible to perform **CRUD** operations without considering how those objects relate to their data source. It manages the mapping details between a set of objects and the underlying database. It hides and encapsulates the changes in the data source itself. Thus, when data sources or their APIs change, only ORM change is required rather than the application that uses ORM.

HIBERNATE WITH JPA: FEATURES

- Hibernate is a pure Java object-relational mapping (ORM) and persistence framework which maps the POJOs to relational database tables. The usage of Hibernate as a persistence framework enables the developers to concentrate more on the business logic instead of making efforts on SQLs and writing boilerplate code. Hibernate is having many features as listed below –
 - Open Source
 - High Performance
 - Light Weight
 - Database independent
 - Caching
 - Scalability
 - Lazy loading
 - Hibernate Query Language (HQL)

COMPARISON AMONG THESE FRAMEWORKS

- Spring, Struts and Hibernate are parts of MVC architecture. **These frameworks serve different purposes and can exist independently or together.**
- It depends on the project requirement to choose the combination of frameworks. This section gives an overview of comparison among the various frameworks such as Spring Vs Struts, Spring Boot Vs Spring MVC and Spring Vs Hibernate.

Struts Vs Spring

<u>Struts</u>	<u>Spring</u>
An open source framework which enables to extend Java Servlet API and MVC framework.	An open source framework to implement inversion of Control (IOC) and dependency Injection (DI).
It is a Heavyweight framework.	It is a Lightweight framework.
It is less flexible than Spring.	It is more flexible than Struts.
It has non layered architecture.	It has layered architecture.
It is tightly coupled.	It is loosely coupled.
It also provides integration with ORM and JDBC but manual coding is required.	It provides an easy integration with ORM and JDBC technologies.

Spring Boot Vs Spring MVC

<u>Spring Boot</u>	<u>Spring MVC</u>
Spring Boot removes all boilerplate code and supports the auto configuration based on jars available into classpath.	Spring MVC requires a lot of configuration and it contains a lot of boilerplate code.
Spring Boot provides many spring boot starters. Spring boot starter is a unit of related dependencies wrapped together. It solves the problem of dependency management.	Dependency management is a tough process since every dependency with the correct version needs to be declared.
Spring Boot makes the application development easier and faster.	Spring MVC requires a lot of configurations and dependency management is time consuming. Hence, development is slow.
Spring Boot supports embedded servers to be packaged along with applications to run the jar stand alone.	A lot of manual configuration is required to attain the same level of packaging.
Spring Boot supports many production ready features such as Actuators, Process monitoring out-of-box.	Spring MVC does not support any production ready feature.
Spring Boot is very flexible and provides many modules which can be used to build many different other applications.	Spring MVC is designed only for the development of dynamic web pages and RESTful web services.

Spring Vs Hibernate

<u>Spring</u>	<u>Hibernate</u>
Spring is an open source, complete and modular application framework to develop enterprise applications in java.	Hibernate is an ORM framework specialized in data persisting and data retrieval from DB.
Spring provides many useful features such as transaction management, aspect-oriented programming and dependency injection (DI).	Hibernate provides Object-Relational Persistence and Query service for applications.
Spring framework has many modules such as Spring-Core, Spring-MVC, Spring-Rest, Spring-Security and many more.	Hibernate does not have modules like Spring framework.
Spring framework has support for connection pooling by changing the configuration in the spring configuration file.	Hibernate supports robust connection pooling features. Hibernate supports two levels of cache which improves the application performance.

MAVEN: INTRODUCTION, OVERVIEW AND CONFIGURATION

- Building the process of a software project may involve a series of tasks among downloading required dependencies, putting jars on class path, generating source code (for auto generated code), generating documentation from source code, source code compilation, tests execution, packaging compiled code along with the dependencies into deployable artifacts such as WAR, EAR or JAR and deploying the generated artifacts to the application server or repository.
- Maven is a simple and powerful build automation and management tool used primarily for Java Projects. It can also be used to build and manage other language projects such as C#, Scala, Ruby, etc.
- A manual build process is error prone and time consuming. Maven minimizes the risk of humans making errors and speeds up the build process of a software project.

Maven Installation

- JDK installation is the prerequisite to execute maven into the system. Visit <http://maven.apache.org/download.cgi> to download the maven. Extract the downloaded file and perform the following configurations in order to execute maven.
 - Set JAVA_HOME environment variable pointing to valid JDK installation path.
 - Set M2_HOME environment variable pointing to the directory of extracted maven.
 - Add the executable maven path into PATH variable as **%M2_HOME%\bin**
 - on Windows and **%M2_HOME%/bin** on Unix.
- Open a command prompt and execute the command **mvn -version** to check the maven installation and configuration. You will get the output similar to below.

Maven Installation

- JDK installation is the prerequisite to execute maven into the system. Visit <http://maven.apache.org/download.cgi> to download the maven. Extract the downloaded file and perform the following configurations in order to execute maven.
 - Set JAVA_HOME environment variable pointing to valid JDK installation path.
 - Set M2_HOME environment variable pointing to the directory of extracted maven.
 - Add the executable maven path into PATH variable as **%M2_HOME%\bin** on Windows and **%M2_HOME%/bin** on Unix.
- Open a command prompt and execute the command ***mvn -version*** to check the maven installation and configuration. You will get the output similar to below.

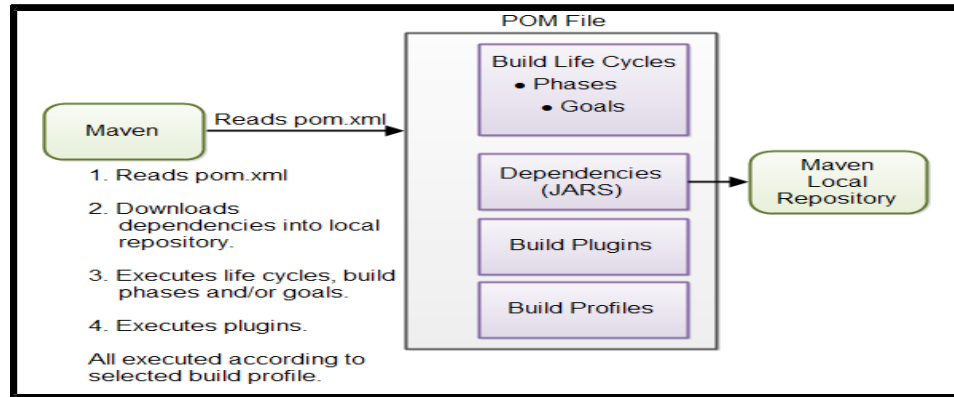
Maven Installation

```
$ mvn -version  
Apache Maven 3.6.3 (cecedd343002696d0abb50b32b541b8a6ba2883f)  
Maven home: D:\Rahul\software\apache-maven-3.6.3-bin\apache-maven-3.6.3  
Java version: 11.0.10, vendor: Oracle Corporation, runtime: C:\Program Files\Java\jdk-11.0.10  
Default locale: en_US, platform encoding: Cp1252  
OS name: "windows 10", version: "10.0", arch: "amd64", family: "windows"
```

Maven Version Verification

Maven Core Concepts

- POM file (Project Object Model) is the focal point of a maven project. A maven project is configured using a POM file known as pom.xml. A POM file is an XML file which describes the project and contains all the references of project resources like source code, test code, dependencies etc. Maven execution and the main content of the POM file has been illustrated below.



Maven Execution Flow

Basic Structure of the POM File

- Basic structure of a typical POM file contains project identifiers, dependencies, properties, build etc. POM file also supports the concept of profile. Profiles are used to customize the build configuration for different environments such as dev, test and prod.
 - **Project Identifiers:** Maven combines groupId:artifactId:version (GAV) to form the unique identifier to identify a project uniquely.
 - **Dependencies:** A mid-scale or large scale project uses external Java APIs or frameworks which are packaged in their own JAR files.
Maven has a built-in powerful dependency management feature.
 - **Maven Repositories:** Maven repositories are directories of packaged JAR files with extra Meta data. There are three types of repository in Maven.

Maven Build Life Cycles, Phases and Goals

Build Phase	Description
validate	Verifies the correctness of the project and make sure that all required dependencies are downloaded.
Compile	Compiles the source code to produce the binary artifacts
Test	Executes the unit test cases using a suitable unit testing framework without packaging the binary artifacts
Package	Packs the binary artifacts into distributable format such as JAR,WAR,EAR
intergration- test	Executes integration test cases which require packaging.
Verify	Verifies the correctness of package
Install	Installs the package into local maven repository
Deploy	Copies the final package to the remote repository for sharing with other developers and projects.

Summary

- The session we know that the Model 1 and Model 2 architecture of a web application with its advantages and disadvantages. Model 2 fixes the concerns into Model 1 architecture and Struts fixes the issues into Model 2 architecture and provides Pull- MVC (MVC2) based framework to develop enterprise web applications. The features of different frameworks such as Struts, Spring Boot, Spring MVC and Hibernate have been explained and these frameworks have been compared. The last section of this unit has explained Maven as a simple and powerful build and deployment management tool.

*Thank
You*



MCS-220

Unit-7 -Web technologies

Spring MVC Concepts

Topics to be covered

- Introduction
- Setting up Development Environment for Spring MVC
- First Hello World Project using Spring MVC
- Inversion of Control (IoC) and Dependency Injection
- Creating Controllers and Views
- Request Params and Request mapping
- Form Tags and Data binding
- Form Validation
- Summary

Introduction

- The Spring MVC framework is an open source Java platform. This framework was developed by Rod Johnson and it was first released in June 2003 under the Apache 2.0 license. In web based applications, the size of code plays a vital role. This framework is lightweight when it comes to size. This leads this framework to reach its heights. The basic version of this framework is of around 2MB. It follows the MVC (Model-View-Controller) design pattern. Spring Framework gears all the basic features of a core spring framework like Inversion of Control, and Dependency Injection.
- MVC is a software structural design - the building of the system - that separates the business logic (domain/application/business) (whatever the way business prefers) from the rest of the user interface.

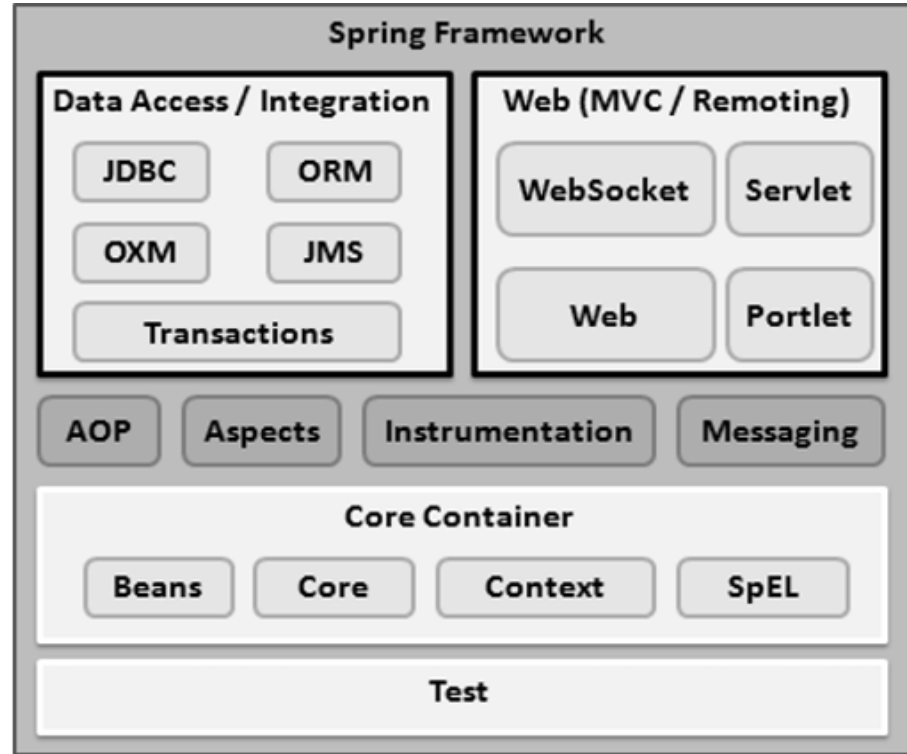
SETTING UP DEVELOPMENT ENVIRONMENT FOR SPRING MVC

- Spring Framework is used widely in the market and there are many Solution that have been developed by keeping Spring Framework as core architecture. It is widely used in the market as well and SAP Hybris is one such example.
- Spring Framework is used to build almost anything, it's make coding in Java much easy as it takes care of most of boilerplate (Boilerplate term refers to standardized text, methods, or procedures that can be used again without making any major changes to the original. It is mainly used for efficiency and to increase standardization in the structure and language of written or digital documents) code and let developer work on business logic.

SETTING UP DEVELOPMENT ENVIRONMENT FOR SPRING MVC

- Spring Framework is modular in nature which suggests that Spring Framework it's not a single entity. It has been divided into several modules which serve their own functionality:
 - Spring Framework allows application to integrate with various other backend technologies.
 - Spring Framework allows the development of standalone applications.
 - Spring Framework becomes very useful while using distributed architecture.
 - Due to lightweight in nature it reduces the complexity while using EJBs.
 - Spring Security make the applications more secure.
 - Spring Batch competently manage long running tasks.
 - IoC and DI adds up to the flexibility and modularity of the application.

SETTING UP DEVELOPMENT ENVIRONMENT FOR SPRING MVC



Spring Framework

+91 - 9319133134 , query@cftedu.in

SETTING UP DEVELOPMENT ENVIRONMENT FOR SPRING MVC

- **Core Container:** The Core Container contains the Core, Beans, Context, and Expression Language modules, the details of which are as follows –
 - IoC and DI features are the main features of the Framework which comes under the core container as it provides the fundamental parts of the framework.
 - BeanFactory comes under Bean module, it helps in the implementation of the factory pattern.
 - Core and Beans module provides the strong base to build the Context module and it becomes the middleman to access any defined and configured object. The Context module works as a focal point of Application Context.
 - A powerful expression language for querying and manipulation an object graph at runtime is provided by SpEL module.

SETTING UP DEVELOPMENT ENVIRONMENT FOR SPRING MVC

- **The Data Access/Integration:** This layer involves the JMS, OXM, ORM, & JDBC and Transaction modules and below are the details –
 - It eliminates the need of religious JDBC related coding efforts of the developer as this JDBC module provides a JDBC-abstraction layer.
 - The ORM Module takes care integration layer for some popular object-relational mapping APIs like iBatis, Hibernate, JDO and JPA.
 - Object/XML mapping implementations for some APIs like XStream, JiBX, XMLBeans, Castor and JAXB are supported by the OXM Module.
 - The Java Messaging Service JMS module holds features for producing and consuming messages.

SETTING UP DEVELOPMENT ENVIRONMENT FOR SPRING MVC

- **Web:** The Web layer holds of the following modules (Web, Web-MVC, Web-Socket, and Web-Portlet) and the details of these are as follows:
 - Basic features for multipart file-upload functionality and the initialization of the IoC container using servlet listeners and a web-oriented application context provided by the Web Module.
 - Spring's Model-View-Controller (MVC) implementation for web applications is covered under Web-MVC.
 - The Web-Socket module takes responsibility of communication (support for WebSocket-based, two-way communication between the client and the server in web applications).
 - The Web-Portlet module covers the MVC implementation to be used in a portlet setup and emulates the functionality of Web-Servlet module.

SETTING UP DEVELOPMENT ENVIRONMENT FOR SPRING MVC

- **Miscellaneous:** AOP, Aspects, Instrumentation, Web and Test modules are also some other important modules and the details of these are as follows –
 - To decouple code, there is a need to define method-interceptors and pointcuts and this is achieved by AOP Module as it provides an aspect-oriented programming implementation.
 - AspectJ integration is achieved by the Aspects Module and AspectJ is mature and powerful AOP Framework.
 - This module is basically based on application servers as class loader and class instrumentation is used and supported by The Instrumentation Module.
 - Support of Streaming Text Oriented Messaging Protocol (STOMP) as the WebSocket sub-protocol to use in applications is provided by the Messaging Module.
 - JUnit or TestNG frameworks are used for Spring Components Testing and all this is done by Test Module

FIRST HELLO WORLD PROJECT USING SPRING MVC

- We will see in the example given below how to write a simple Hello World application in Spring MVC Framework. First thing to notice is to have Eclipse or some other IDE in place and follow the steps to develop a Dynamic Web Application. Here Eclipse IDE is used for explanations.
- **Step 1:** Create a Dynamic Web Project with a name Hello IGNOU and create a package com.ignou the src folder in the created project.
- **Step 2:** Drag and drop below mentioned Spring and other libraries into the folder WebContent/WEB-INF/lib.

FIRST HELLO WORLD PROJECT USING SPRING MVC

- **Step 3:** Create a Java class HelloController under the com.ignou.springmvc package.
- **Step 4:** Create Spring configuration files web.xml and HelloWeb -servlet.xml under the WebContent/WEB-INF folder.
- **Step 5:** Create a sub-folder with a name jsp under the WebContent/WEB-INF folder. Create a view file hello.jsp under this sub-folder.
- **Step 6:** The final step is to create the content of all the source and configuration files. Once it is done then you have to export the application.

FIRST HELLO WORLD PROJECT USING SPRING MVC - I

- Web.xml

```
1 <web-app id="WebApp_ID" version="2.4"  
2   xmlns="http://java.sun.com/xml/ns/j2ee"  
3   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
4   xsi:schemaLocation="http://java.sun.com/xml/ns/j2ee  
5     http://java.sun.com/xml/ns/j2ee/web-app_2_4.xsd">  
6   <display-name>Spring MVC Application</display-name>  
7   <servlet>  
8     <servlet-name>HelloWeb</servlet-name>  
9     <servlet-class>  
10      org.springframework.web.servlet.DispatcherServlet  
11      </servlet-class>  
12      <load-on-startup>1</load-on-startup>  
13   </servlet>  
14   <servlet-mapping>  
15     <servlet-name>HelloWeb</servlet-name>  
16     <url-pattern>/</url-pattern>  
17   </servlet-mapping>  
18 </web-app>
```


FIRST HELLO WORLD PROJECT USING SPRING MVC - II

- HelloWorld-servlet.xml

```
1=<beans xmlns="http://www.springframework.org/schema/beans"
2  xmlns:context="http://www.springframework.org/schema/context"
3  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4  xsi:schemaLocation="
5  http://www.springframework.org/schema/beans
6  http://www.springframework.org/schema/beans/spring-beans-3.0.
7  http://www.springframework.org/schema/context
8  http://www.springframework.org/schema/context/spring-context-
9  <context:component-scan base-package="com.ignou" />
10=<bean class="org.springframework.web.servlet.view.InternalRe:
11  <property name="prefix" value="/WEB-INF/JSP/" />
```


FIRST HELLO WORLD PROJECT USING SPRING MVC - III

- HelloController.java

```
1 package com.ignou;
2
3 import org.springframework.stereotype.Controller;
4 import org.springframework.web.bind.annotation.RequestMapping;
5 import org.springframework.web.bind.annotation.RequestMethod;
6 import org.springframework.ui.ModelMap;
7 @Controller
8 @RequestMapping("/hello")
9 public class HelloController {
10     @RequestMapping(method = RequestMethod.GET)
11     public String printHello(ModelMap model) {
12         model.addAttribute("message", "Hello Spring MVC Framework!");
13         return "hello";
14     }
15 }
16
```

FIRST HELLO WORLD PROJECT USING SPRING MVC - IV

- Hello.jsp

```
1 <%@ page contentType="text/html; charset=UTF-8" %>
2 <html>
3 <head>
4 <title>Hello World</title>
5 </head>
6 <body>
7 <h2>${message}</h2>
8 </body>
9 </html>
```

- Need to include the below list of Spring and other libraries in our web application. We can do this by drag and drop them in in WebContent/WEB-INF/lib folder.

FIRST HELLO WORLD PROJECT USING SPRING MVC - III

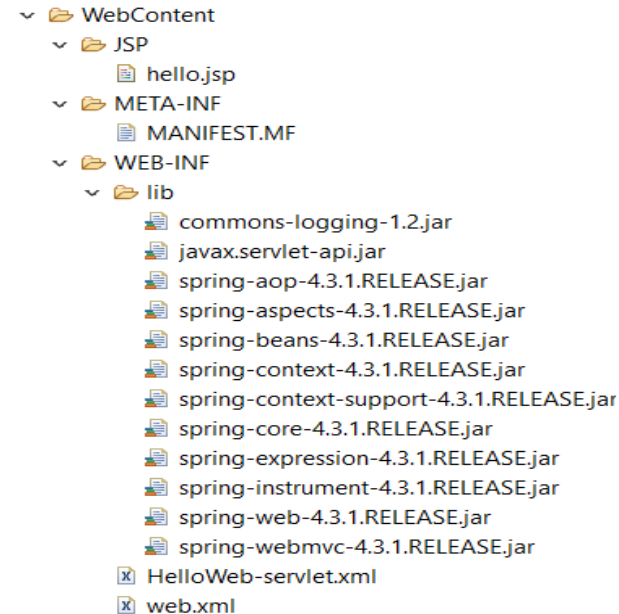
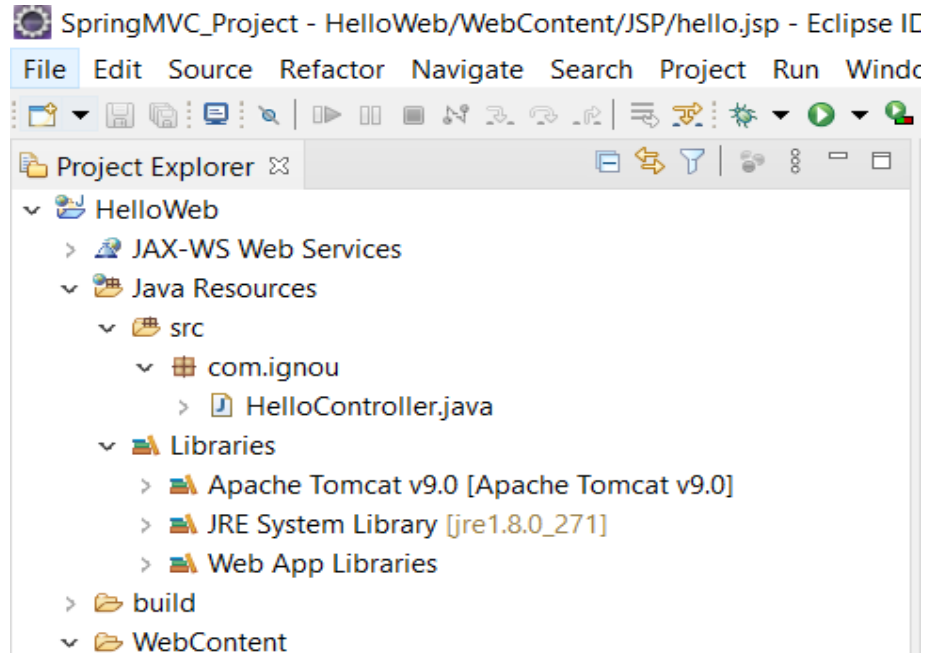
▼ WEB-INF

▼ lib

- commons-logging-1.2.jar
- javax.servlet-api.jar
- spring-aop-4.3.1.RELEASE.jar
- spring-aspects-4.3.1.RELEASE.jar
- spring-beans-4.3.1.RELEASE.jar
- spring-context-4.3.1.RELEASE.jar
- spring-context-support-4.3.1.RELEASE.jar
- spring-core-4.3.1.RELEASE.jar
- spring-expression-4.3.1.RELEASE.jar
- spring-instrument-4.3.1.RELEASE.jar
- spring-web-4.3.1.RELEASE.jar
- spring-webmvc-4.3.1.RELEASE.jar

FIRST HELLO WORLD PROJECT USING SPRING MVC - III

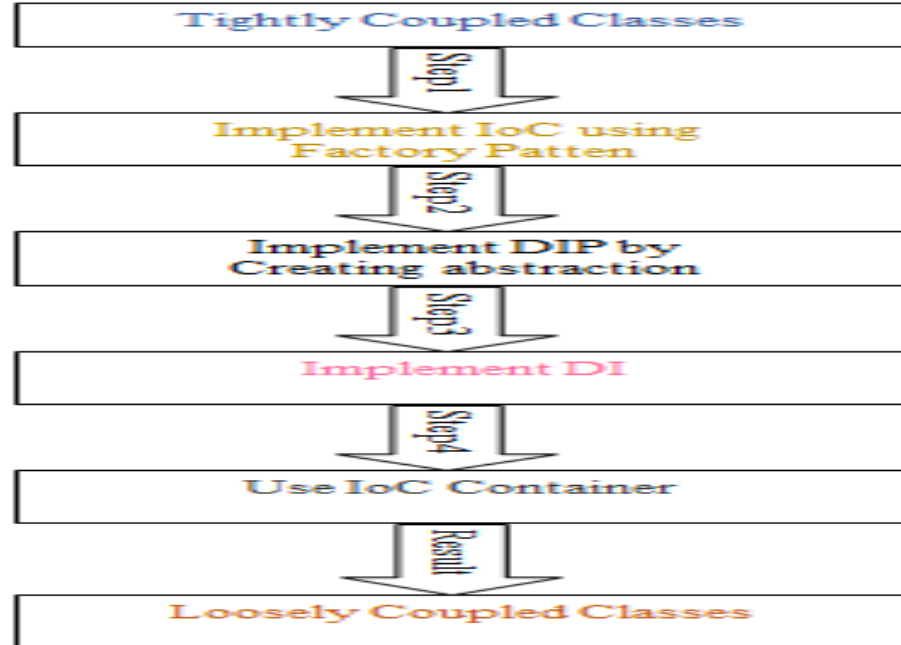
- Finally, Project Explorer looks as below:



INVERSION OF CONTROL (IOC) DEPENDENCY INJECTION

- **Inversion of Control (IoC):** Inversion of Control (IoC) is considered as a design principle but some of the developer society consider it as a pattern. This is designed to achieve loose coupling as its name suggests that is used to invert controls on Object- Oriented design. It also controls the flow of an application and also controls over the flow of dependent object or an object creation and binding.

INVERSION OF CONTROL (IOC) DEPENDENCY INJECTION



Inversion of Control Flow

+91 - 9319133134 , query@cftedu.in

INVERSION OF CONTROL (IOC) DEPENDENCY INJECTION

- **Dependency Injection:**
- The Dependency Injection is a design pattern that removes the dependency of the programs. To removes the dependency of the programs, Dependency Injection design pattern do this job. In that kind of scenario, information is provided by the external sources such as XML file. Dependency Injection plays a vital role to make the code loosely coupled and easier for testing. In such a case we write the code as:

INVERSION OF CONTROL (IOC) DEPENDENCY INJECTION

```
class Ignou
{
    Student student;
    Ignou(Student student)
    {
        this.student=student;
    }
    public void setStudent(Student student)
    {
        this.student=student;
    }
}
```

We can perform Dependency Injection via following two ways in Spring framework

By Constructor – In the above code, the below snapshot of the code uses the constructor

```
Ignou(Student student)
{
    this.student=student;
}
```

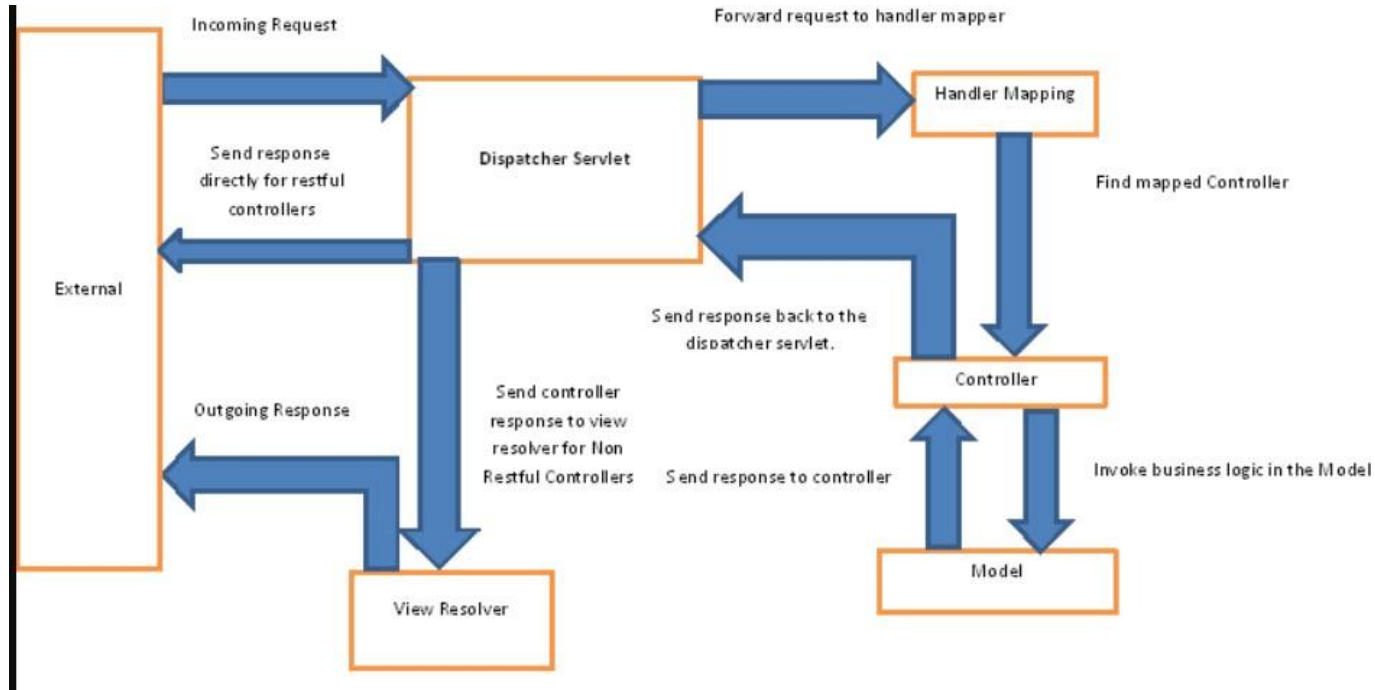
By Setter method – In the above code, the below snapshot of the code uses the Setter method

```
public void setStudent(Student student)
{
    this.student=student;
}
```


CREATING CONTROLLERS AND VIEWS

- In Spring MVC, controller classes are used to handle the request which are coming from the client. After getting the request from client then controller starts its task by invoking a business class to process business-related tasks, and once it gets fulfilled then it redirects the client request to a logical view name, and now it is resolved by Spring's dispatcher servlet in order to render results or output.
- Below are the main responsibilities:
 - Controllers main task is to intercept the incoming requests.
 - Controller Converts the payload of the request to the internal structure of the data (mapping basically).
 - After getting response from above two steps then it sends the data to Model for further processing,
 - Once it gets the processed data from the Model then that data is referred to the View for rendering/display.

CREATING CONTROLLERS AND VIEWS



High Level Flow in Spring MVC

REQUEST PARAMS AND REQUEST MAPPING

- **Request parameters**
- Request Parameter can be read in controller class with the help of Request-Param annotation. Or we can say that views are sending some parameters with keys.

Student_ID
Student_FirstName
Student_LastName
Student_DOB

Just imagine that we have filled the below details.

Student_ID=1001 (depends upon requirement as it may be system generated)
Student_FirstName=Dharmendra
Student_LastName=Singh
Student_DOB =21.01.1979

So, we have the below code for this
@Controller
@RequestMapping("/ignoustudent/*")

REQUEST PARAMS AND REQUEST MAPPING - II

```
public class IgnouStudentController
{
    @GetMapping("/fill") public String fill()
    {
        return "studentForm";
    }
    @PostMapping("/fill/process")
    public String processForm(ModelMap model, @RequestParam("Student_ID") int stud_id, @RequestParam("Student_FirstName") String stud_firstName, @RequestParam("Student_LastName") String stud_lastName, @RequestParam("Student_DOB") Date stud_dob)
    {
        model.addAttribute("Student_ID ", stud_id); model.addAttribute("Student_FirstName", stud_firstName);
        model.addAttribute("Student_LastName", stud_lastName); model.addAttribute("Student_DOB", stud_dob);
        return "index";
    }
}
```

REQUEST PARAMS AND REQUEST MAPPING - III

```
@Controller @RequestMapping("/ignoustudent/*") public class
IgnouStudentController
{
    @GetMapping("/fill") public String fill()
    {
        return "studentForm";
    }
    @PostMapping("/fill/process")
    public String processForm(ModelMap model, @RequestParam id, @RequestParam("FirstName") String firstName,
    @RequestParam("LastName")
    String lastName)
    {
        model.addAttribute("Student_id", id);
        model.addAttribute("Student_firstName", firstName); model.addAttribute("Student_lastName",
        lastName); model.addAttribute("Student_DOB", dob);
        return "index";
    }
}
```

REQUEST PARAMS AND REQUEST MAPPING

- **Request Mapping:**
- Spring Controller supports the three levels of request mapping and it depends on the different `@RequestMapping` annotations declared in its handler methods and controller class.

By path `@RequestMapping("path")`

By HTTP method

`@RequestMapping("path",method=RequestMethod.GET).`

Other Http methods such as POST,

PUT, DELETE, OPTIONS, and TRACE are also supported.

By query parameter

`@RequestMapping("path", method=RequestMethod.GET,
params="param1")`

By the presence of request header

`@RequestMapping("path", header="content-type=text/*")`

FORM TAGS AND DATA BINDING

- For the Web Pages, Spring MVC has provided the form tags which are configurable and reusable building blocks. These Tags are very helpful for easy development, easy to read and maintain. These Tags can be used as data binding tags where these tags can set data to java objects (Bean or POJO).
- These Tags provide support for corresponding HTML tags.
- The form tag library comes under the spring-webmvc.jar. To get the support from the form tag library, requires to reference the following directive at the beginning of the JSP page: %@ taglib prefix="form" uri="<http://www.springframework.org/tags/form>" %>

FORM TAGS AND DATA BINDING

- Some of the below frequently used Spring MVC form tags.

Form Tag	Description
form:form	It is a container tag that contains all other form tags. This tag is used to generate the text field.
form:input	This tag is used to generate the radio buttons.
form:radiobutton	This tag is used to generate the checkboxes.
form:checkbox	This tag is used to generate the password input field. This tag is used to generate the drop-down list.
form:password	This tag is used to generate the multi-line text field.
form:select	This tag is used to generate the hidden input field.
form:textarea	
form:hidden	

FORM TAGS AND DATA BINDING

- **Spring data binding** mechanism provides the functionality to bound the user inputs dynamically to the beans. It can be explained as it allows the setting of property values into a target object and this functionality is provided by the DataBinder class though BeanWrapper also provides the same functionality but DataBinder additionally supports field formatting, binding result analysis and validation.
- Where DataBinder works on a high level and BeanWrapper works on a low level. Both Binders are used in PropertyEditors to parse and format the property values. We should use the DataBinder as it works on a high level and internally it uses the BeanWrapper only.
- Data binding from web request parameters to JavaBean/POJO objects is done by WebDataBinder. We use it for customizing request parameter bindings.

FORM TAGS AND DATA BINDING

- We can understand better these topics with the help of the below example:
First, we are going to create a request page (Welcome.jsp)

```
<html>
<body>
<h2> Spring MVC Form Tag with Data Binding Example for IGNOU Students </h2>
<a href = "home_page">Home page | </a>
<a href = "about_us"> About Us | </a>
<a href = "student_form"> Student Detail Form</a>
</body>
</html>
```

FORM TAGS AND DATA BINDING

- Create model (getter/setter) class (Student.java)

```
package com.ignou.studentdetails; public class Student
{
    private String fname; private String lname; public Student()
    {
    }
    public String getFname()
    {
        return fname;
    }
    public void setFname(String fname)
    {
        this.fname = fname;
    }
    public String getLname()
    {
        return lname;
    }
    public void setLname(String lname)
    {
        this.lname = lname;
    }
}
```

FORM TAGS AND DATA BINDING

- Create the controller (StudentDetailController.java) now

```
package com.ignou.studentdetails;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
@Controller
public class StudentDetailController
{
    @RequestMapping("/student_form") public String showStudentForm( Model m)
    {
        Student student = new Student(); m.addAttribute("student", student);
        return "studentform" ;
    }
    @RequestMapping("/studentregis")
    public String showStudentData(@ModelAttribute("student") Student student)
    {
        System.out.println("student:" + student.getFname() +" "+ student.getLname());
        return "student-data" ;
    }
}
```

FORM TAGS AND DATA BINDING

- Add the entry of controller in Web.xml

```
<?xml version = "1.0" encoding = "UTF-8"?>
<web-app xmlns:xsi = "http://www.w3.org/2001/XMLSchema-instance" xmlns = http://xmlns.jcp.org/xml/ns/javaee
xsi:schemaLocation = http://xmlns.jcp.org/xml/ns/javaee http://xmlns.jcp.org/xml/ns/javaee/web-app\_3\_1.xsd
id = "WebApp_ID" version = "3.1">
<display-name>spring-mvc-demo</display-name>
<absolute-ordering />
<servlet>
<servlet-name>dispatcher</servlet-name>
<servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
<init-param>
<param-name>contextConfigLocation</param-name>
<param-value>/WEB-INF/spring-servlet.xml</param-value>
</init-param>
<load-on-startup>1</load-on-startup>
```

FORM VALIDATION

- Spring Validation is very important as it is used for validating the @Controller inputs. Using Spring validation, the user input on the server-side can be validated. It is used for checking and subsequently accepting the user input in any web application. From Spring framework version 4 and above, the Spring framework supports Bean Validation 1.0 (JSR-303) and Bean Validation 1.1 (JSR-349).
- BindingResult is an interface that represents the binding results. It extends an interface for error registration for allowing a validator to be applied, and adds binding-specific analysis.

FORM VALIDATION

Annotations	Description
@Valid	This annotation is used on a model object that we want to validate
@Size	It specifies that the size of the annotated element must be between the specified boundaries.
@Pattern	It specifies that the annotated CharSequence must match the regular expression.
@Past	It specifies that the annotated element must be a date in the past.
@Null	It specifies that the annotated element must be null.
@NotNull	It specifies that the annotated element should not be null.
@Min	It specifies that the annotated element must be a number whose value must be equal or greater than the specified minimum number.
@Max	It specifies that the annotated element must be a number whose value must be equal or smaller than the specified maximum.
@Future	It specifies that the annotated element must be a date in the future.
@Digits	It specifies that the annotated element must be a digit or number within the specified range. The supported types are short, int, long, byte, etc.

FORM VALIDATION

- @DecimalMin** It specifies that the annotated element must be a number whose value must be equal or greater than the specified minimum. It is very similar to @Min annotation, but the only difference is it supports CharSequence type.
- @DecimalMax** It specifies that the annotated element must be a number whose value should be equal or smaller than the specified maximum. It is very similar to @Max annotation, but the only difference is it supports CharSequence type.
- @AssertTrue** It determines that the annotated element must be true.
- @AssertFalse** It determines that the annotated element must be false

SUMMARY

- Spring's Web MVC Framework is very popular and used framework due to its request-driven, designed around a central Servlet that dispatches requests to controllers and offers other functionality that facilitates the development of web applications. It also facilitates fast and parallel development. It Reuses business code - instead of creating new objects. It allows us to use the existing business objects. It provides easy to test functionality as we create JavaBeans classes that enable us to inject test data using the setter methods. This framework is used in many products development like Hybris and banking domain products.

*Thank
You*



MCS-220

Unit-8 -Web technologies

SPRING MVC WITH BOOTSTRAP CSS

Topics to be covered

- Introduction
- Configuration of Bootstrap in Spring Application
- Different ways to configure Bootstrap
- Apply custom CSS in pages
- Setting UP Database using Hibernate
- Java based configuration for Spring and Hibernate Integration
- Create, Read, Update, and Delete (CRUD)
- Mapping CRUD to SQL Statements
- Mapping CRUD to REST
- CRUD examples in Spring MVC and Hibernate
- Summary

Introduction

- In this session we discuss about the spring MVC with bootstrap CSS.
- Nowadays, huge traffic comes from mobile devices, and most of the internet searches like google are done on mobile devices, whether smartphones or tablets.
- A responsive website prevents this loss and leads to substantial revenue gains.
- Responsive web design improves user experience and creates a positive perception for a company or business.

CONFIGURATION OF BOOTSTRAP IN SPRING APPLICATION

- Bootstrap was originally created by a designer and developer at Twitter in mid 2010. Prior to being an open-sourced framework, Bootstrap was known as *Twitter Blueprint*.
- **Bootstrap** is a free, popular and open-source CSS framework directed at responsive, mobile-first front-end web development.
- Bootstrap contains HTML and CSS based design templates for various interface components such as typography, forms, buttons, modals, image carousels, navigation etc.,
- Bootstrap provides the ability to create responsive designs faster and easily.

CONFIGURATION OF BOOTSTRAP IN SPRING APPLICATION

- The various factors which support learning Bootstrap are as follows.
 - **Browser Support:** Bootstrap supports all popular web browsers such as Chrome, Safari, Firefox, Opera, and Internet edge.
 - **Responsive Design:** Bootstrap enables us to write responsive websites very easily. Its responsive CSS adjusts automatically to Desktops, Tablets and Mobiles.



CONFIGURATION OF BOOTSTRAP IN SPRING APPLICATION

- **Uniform solution to build interface:** Bootstrap provides a clean and uniform solution to build an interface for the developers.
- **Customization:** Bootstrap provides a simple and easy way to customize the CSS.
- **Functional built-in components:** Bootstrap contains beautiful and functional built-in components which can be customized very easily.

Different ways to configure Bootstrap

- The Bootstrap 5 is the most recent version of the Bootstrap framework. There are two ways to include Bootstrap into JSP or HTML. The required configurations for Bootstrap 4 and Bootstrap 5 has been described as follows.
 1. **Bootstrap through Content Delivery Network (CDN):** Bootstrap CDN is an efficient and faster way to deliver the content from your website to the users. Bootstrap CDN speeds up the websites as CDN servers are intelligent enough to serve the resources based on proximity.

Different ways to configure Bootstrap

- **Including Bootstrap 4 into JSP/HTML.**
- For some of the functionalities Bootstrap 4 JavaScript file requires jQuery and Popper JavaScript, and these two must be loaded before loading of bootstrap.min.js file.
- **CSS:** Include the below link to the <head> tag of your desired HTML/JSP file.
- **JS:** Place the following <script>s near the end of your pages, right before the closing </body> tag, to enable them. jQuery must come first, then Popper.js, and then our JavaScript plugins.

Different ways to configure Bootstrap

- **Including Bootstrap 5 into JSP/HTML**
- Bootstrap 5 no longer needs jQuery as a dependency since JavaScript can provide the same functionality.
- Add the following CSS and JS through Bootstrap 5 CDN to JSP/HTML:
 - **CSS:** Copy-paste the below Stylesheet link to the `<head>` tag of your desired HTML/JSP file.
 - **JS:** Place the following `<script>`s near the end of your pages, right before the closing `</body>` tag.

Different ways to configure Bootstrap

2. Custom configuration of Bootstrap with Spring MVC (Offline by downloading files locally)

- Another approach to include Bootstrap is to directly download the Bootstrap CSS and JS files locally to the project folder and then include the local copy into JSP/HTML. The following links can be used to download the Bootstrap.
 - Bootstrap 4: <https://getbootstrap.com/docs/4.3/getting-started/download/>
 - Bootstrap 5: <https://v5.getbootstrap.com/docs/5.0/getting-started/download/>

Different ways to configure Bootstrap

- Download the Bootstrap and perform the following steps to configure Bootstrap in the Spring application.
 - Put static resources CSS and JS files into webapp/resources/static directory. To Segregate CSS and JS, keep them into respective folders webapp/resources/static/js and webapp/resources/static/css into the eclipse project.
 - Create the resource mapping into Spring using either XML configuration or Java configuration.
- XML configuration
- `<mvc:resources mapping="/js/**" location="/resources/static/js/" />`
- `<mvc:resources mapping="/css/**" location="/resources/static/j=css/" />`

APPLY CUSTOM CSS IN PAGES

- CSS or Cascading Style Sheets is a method of adding stylistic instructions for your website to its back-end code.
- The HTML (hypertext markup language) is the most common coding language for a website. CSS is an extension of HTML that outlines specific stylistic instructions to style websites.
- CSS is used to specify the color of a heading or what font your content should be written in. CSS can be embedded into HTML elements, as shown below.

```
<p style="color: green;">  
    This is Green Color Text.  
</p>
```

SETTING UP DATABASE USING HIBERNATE

- Spring supports bootstrapping the Hibernate Session Factory in order to set up the database using Hibernate. The database setup can be done with a few lines of Java code or XML configuration.
- Spring provides two key beans to support the Hibernate integration, available in the `org.springframework.orm.hibernate5` package:
 - **LocalSessionFactoryBean**: creates a Hibernate's **SessionFactory** which is injected into Hibernate-based DAO classes.
 - **PlatformTransactionManager**: provides transaction support code for a **SessionFactory**. Programmers can use **@Transactional** annotation in DAO methods to avoid writing boiler-plate transaction code explicitly.

CREATE, READ, UPDATE, AND DELETE (CRUD)

- In the computer programming, the acronym CRUD stands for CREATE, READ, UPDATE and DELETE. The CRUD paradigm is common in constructing web applications.
- It can describe user-interface conventions that allow viewing, searching and modifying information through computer-based forms and reports.
- Most applications have some form of CRUD functionality. Let's consider that we have a student database that stores information for students and grades.
- When programmers provide interactions with this database (often through stored procedures), the steps of CRUD are carried out as follows:

CREATE, READ, UPDATE, AND DELETE (CRUD)

- Create: A new student is entered into the database.
- Read: The student's information is displayed to the users.
- Update: Already, existing student's attributes are being updated with new values.
- Delete: If the student is not part of the institute, the student can be deleted from the records.

Mapping CRUD to SQL Statements

- The previous slide explained the concept of **CRUD** and defined it based on different contexts. The following table maps **CRUD to database sql** operation with an example.

Function	SQL Statement	Example
C(reate)	Insert	Insert into student (name,grade) values('Prasoon',10);
R(ead)	Select	Select * from student
U(pdate)	Update	Update student set grade=10 where id=1;
D(elete)	Delete	Delete from student where id=1;

Mapping CRUD to REST

- In **REST context**, CRUD corresponds to Rest web service **POST, GET, PUT and DELETE** respectively. The following table maps CRUD to REST with example.

Function	Rest web service	Example
C(reate)	POST	POST http://teststudent.com/students/
R(ead)	GET	GET http://teststudent.com/students/
U(pdate)	PUT	PUT http://teststudent.com/students/1
D(elete)	DELETE	DELETE http://teststudent.com/students/1

Mapping CRUD to REST

- In **REST context**, CRUD corresponds to Rest web service **POST, GET, PUT and DELETE** respectively. The following table maps CRUD to REST with example.

Function	Rest web service	Example
C(reate)	POST	POST http://teststudent.com/students/
R(ead)	GET	GET http://teststudent.com/students/
U(pdate)	PUT	PUT http://teststudent.com/students/1
D(elete)	DELETE	DELETE http://teststudent.com/students/1

Mapping CRUD to REST

- **Create:** To create a resource in a REST environment, HTTP Post method is used. Post method creates a new resource of the specified resource type. To create a new student record for the Student Management System, HTTP POST request and response.

Request: POST <http://teststudent.com/students/>

Request Body:

```
{
  "firstName": "Anurag",
  "lastName": "Singh",
  "grade": 10
}
```

Response: Status Code – 201(Created)

```
{
  "id": 1,
  "firstName": "Anurag",
  "lastName": "Singh",
  "grade": 10
}
```

Mapping CRUD to REST

- **Read:** To read resources in a REST environment, HTTP GET method is used. Read operation should not change the resource. The Get method returns the same response irrespective of number of times of execution. HTTP GET method is **idempotent**. A HTTP method is called idempotent if an identical request can be made once or several times in a row with the same effect while leaving the server in the same state. To retrieve all student records from the Student Management System, HTTP GET method request and response is.

Request: GET <http://teststudent.com/students/>

Response: Status Code – 200 (OK)

```
[  
  {"id": 1, "firstName": "Anurag", "lastName": "Singh", "grade": 10},  
  {"id": 1, "firstName": "Dinesh", "lastName": "Chaurasia", "grade": 10}  
]
```

Mapping CRUD to REST

- **Update:** To update a resource in a REST environment, HTTP **PUT** or **PATCH** method is used. This operation is idempotent. Request and response to update the grade of student id 1.

Request: PUT <http://teststudent.com/students/1>

Request Body:

```
{  
  "id": 1,  
  "firstName": "Anurag",  
  "lastName": "Singh",  
  "grade": 8  
}
```

- **Response:** Status Code – **200 (OK)**. The response includes a Status Code of **200 (OK)** to signify that the operation was successful, but it need not return a response body.

Mapping CRUD to REST

- **Delete:** To delete a resource in a REST environment, **HTTP DELETE** method is used. It is used to remove a resource from the system. This operation is also **idempotent**. Request and response to delete a student with id 1, is shown below.
 - Request: DELETE <http://teststudent.com/students/1>
 - **Response:** Status Code – **204 (No Content)**. Successful delete operation returns a response code of **204 (NO CONTENT)**, with no response body.

CRUD EXAMPLES IN SPRING MVC AND HIBERNATE

- The previous section has explained the CRUD operation for the REST environment. This section is full of code to integrate **Spring MVC** and **Hibernate** to implement the **CRUD** operation at the database level using **Mysql** database. Given below is described the Student Management System application implements which were explained in previous sections. These include the following:
 - Bootstrap configuration into Spring MVC using CDN
 - Custom CSS
 - Spring and Hibernate integration
 - CRUD operation implementation with Mysql

CRUD EXAMPLES IN SPRING MVC AND HIBERNATE

- The required tools and software to complete the implementations of Student Management System application are as follows:
 - Eclipse IDE
 - Maven
 - Tomcat 9
 - MySql 8
 - Java 8 or Higher

Summary

- The session has briefly explained the Bootstrap CSS framework. Bootstrap is a feasible solution to tackle the challenges of making a responsive website as it evolves more and more. The Bootstrap makes things a lot easier to make the website responsive. Bootstrap CSS framework configuration into Spring MVC web application has been described in detail. Custom CSS can be used to override the Bootstrap styles in order to customize the Bootstrap and match the theme of a website. Hibernate is a very popular ORM framework to interact with databases, and Spring provides the required implementation to integrate Hibernate and Spring

*Thank
You*



MCS-220

Unit-9 -Web Technologies

INTRODUCTION TO SPRING BOOT

Topics to be covered

- Introduction
- Spring Boot: An Overview
- Spring Boot Working and Annotations
- Spring Boot Starter
- Spring Boot Project Structure
- Spring Boot Runners
- Spring Boot Web Application
- Spring Boot DevTools and Spring Boot Actuator
- Spring Boot DevTools
- Spring Boot Actuator
- Spring Boot Actuator Example
- Spring Boot- Application Properties
- Command Line Properties
- Properties File

Topics to be covered

- YAML File
- Externalized Properties
- @Value annotation
- Active Profile
- Spring Active Profile in Application
- Running Spring Boot Apps from command line
- Running the Code with Maven in Exploded Form
- Running the Code as a Stand-Alone Packaged Application
- Application Packaging as WAR and Deployment on Tomcat
- Summary

Introduction

- To make the software development easy, fast and effective several frameworks have been introduced such as Struts, Spring and ORM tools such as Hibernate, Eclipse.
- Spring based enterprise application requires many configurations and complicated dependency management which make the set up tedious and error-prone.
- Each time Spring-based enterprise application requires repetition of the same configuration steps:

Introduction

- To import the required dependencies such as Spring MVC, Spring JPA, Spring JDBC, Spring rest etc.
- Import the specified Spring version compatible third-party libraries such as Hibernate, Jackson etc.
- Configure web layer beans such as view resolver, resource manager etc.
- Configure DAO layers such as data source, transaction management, entity manager etc.
- Import web container libraries in the case of web applications.

SPRING BOOT : AN OVERVIEW

- **Spring Boot** is an extension of the Spring framework that takes one step ahead and simplifies the configuration in order to fasten the development and execution of the spring application.
- It is a module that enriches the Spring framework with **Rapid Application Development (RAD)** feature.
- Spring Boot provides comprehensive infrastructure support for the development and monitoring of the enterprise-ready applications based on micro services.
- Spring Boot is a combination of **Spring framework with auto-configuration** and **embedded Servers**.

SPRING BOOT : AN OVERVIEW

- Spring Boot provides a vast number of features and benefits. A few of them are as follows:
 - Everything is auto-configured in Spring Boot.
 - Spring Boot starter eases the dependency management and application configuration
 - It simplifies the application deployment by using an embedded server
 - Production ready features to monitor and manage applications, such as health checks, metrics gathering etc.
 - Reduces the application development time and run the application independently
 - Very easy to understand and develop Spring application
 - Increases productivity and reduces the cost

Spring Boot Working and Annotations

- A class with the main method and annotated with **@SpringBootApplication** is the entry point of the Spring Boot application. Spring Boot auto-configures all required configurations.
- It performs auto-configuration by scanning the classes in class-path annotated with @Component or @Configuration. @SpringBootApplication annotation comprising the following three annotations with their default values-
 - @EnableAutoConfiguration
 - @ComponentScan
 - @Configuration

Spring Boot Working and Annotations

- **@EnableAutoConfiguration**
- **Spring Boot** looks for auto-configuration beans on its class-path and automatically applies them.
- For example, if H2 dependency is added and you have not manually configured any database connection beans, then Spring will auto-configure H2 as an in-memory database.
- The *@EnableAutoConfiguration* annotation should be applied in the root package so that every sub-packages and class can be examined.

Spring Boot Working and Annotations

- **@ComponentScan**
- The **@ComponentScan** annotation enables Spring to scan for things like configurations, controllers, services, and other components .
- @ComponentScan the component scanning started from specified packages by defining **basePackageClasses**, **basePackages** attributes into **@ComponentScan**. The **@ComponentScan** annotation is used with **@Configuration**.

Spring Boot Working and Annotations

- **@Configuration**
- The @Configuration annotation is used for annotation-based configuration into the Spring application.
- A class annotated with **@Configuration** indicates that the class declares one or more **@Bean** methods and can be processed by Spring container to generate bean definitions and service requests for those beans at runtime.
- **@ConditionalOnClass** and **@ConditionalOnMissingClass**
 - Spring @ConditionalOnClass and @ConditionalOnMissingClass annotations let @Configuration classes be included based on the presence or absence of specific classes.

Spring Boot Working and Annotations

- **@ConditionalOnBean** and **@ConditionalOnMissingBean**
- These annotations are used to define conditions based on the **presence or absence** of a specific bean.
- **@ConditionalOnProperty**
- The annotation is used to conditionally create a Spring bean depending on the configuration of a property.

@Bean

```
@ConditionalOnProperty(name = "usepostgres", havingValue = "local") DataSourcedataSource()
```

```
{
```

```
//...
```

```
}
```


Spring Boot Working and Annotations

- **@ConditionalOnResource**
- The annotation is used to conditionally create a Spring bean based on the presence or absence of resources. The SpringConfig class beans will be created if the resource log4j.properties file is present.

```
@Configuration
```

```
@ConditionalOnResource(resources = { "log4j.properties" }) class SpringConfig
```

```
{
```

```
    @Bean
```

```
    public Log4j log4j()
```

```
    {
```

```
        return new Log4j();
```

```
    }
```

```
}
```

Spring Boot Starter

- Development of a web application using Spring MVC will require identifying all the required dependencies, compatible versions and how to connect them together.
- Followings are some of the dependencies which are used in Spring MVC web application.
- For all the dependencies we need to choose the compatible version dependencies.

```
<dependency>  
<groupId>org.springframework</groupId>  
<artifactId>spring-webmvc</artifactId>  
<version>4.2.2.RELEASE</version>  
</dependency>
```

Spring Boot Starter

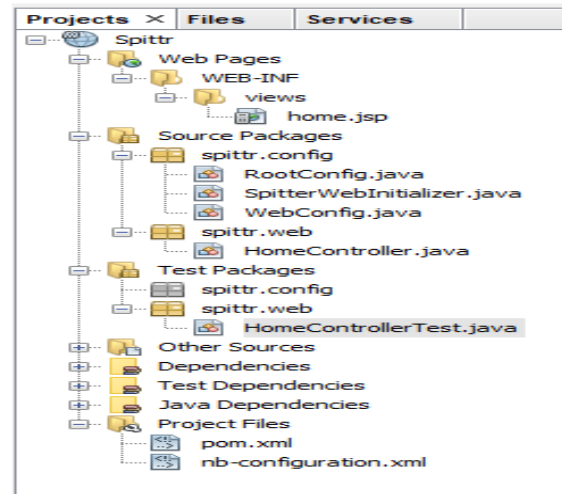
```
<dependency>
<groupId>com.fasterxml.jackson.core</groupId>
<artifactId>jackson-databind</artifactId>
<version>2.5.3</version>
</dependency>
<dependency>
<groupId>org.hibernate</groupId>
<artifactId>hibernate-validator</artifactId>
<version>5.0.2.Final</version>
</dependency>
<dependency>
<groupId>log4j</groupId>
<artifactId>log4j</artifactId>
<version>1.2.17</version>
</dependency>
```

Spring Boot Starter

spring-boot-starter-web	It is used for building web applications, including RESTful applications using Spring MVC. It uses Tomcat as the default embedded container.
spring-boot-starter-jdbc	It is used for JDBC with the Tomcat JDBC connection pool.
spring-boot-starter-validation	It is used for Java Bean Validation with Hibernate Validator.
spring-boot-starter-security	It is used for Spring Security.
spring-boot-starter-data-jpa	It is used for Spring Data JPA with Hibernate.
spring-boot-starter	It is used as a core starter, including auto-configuration support, logging etc.
spring-boot-starter-test	It is used to test Spring Boot applications with libraries, including JUnit, Hamcrest, and Mockito.
spring-boot-starter-thymeleaf	It is used to build MVC web applications using Thymeleaf views.

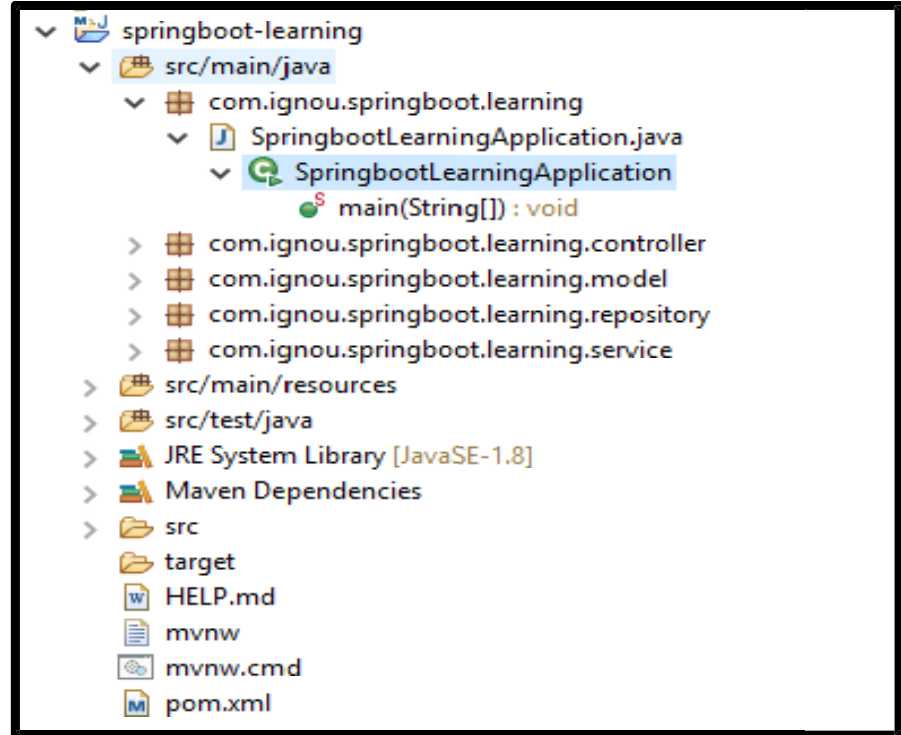
Spring Boot Project Structure

- Spring Boot provides a high degree of flexibility in code layout.
- In Spring Boot, default package declaration is not recommended since it can cause issues such as malfunctioning of auto-configuration or Component Scan. Spring Boot annotations like:
 - `@ComponentScan`,
 - `@EntityScan`,
 - `@ConfigurationPropertiesScan`, and
 - `@SpringBootApplication`



Spring Boot Project Structure

- In the structure, controller, model, repository and service are sub-package of base package **com.ignou.springboot.learning**. All the components and configurations are eligible for implicit scanning with main class as below:



Spring Boot Project Structure For Default Scanning

Spring Boot Project Structure

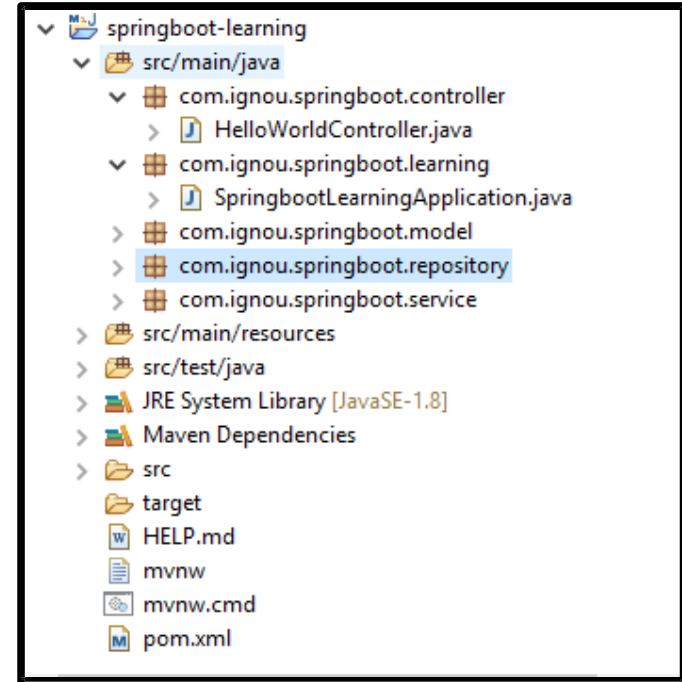
```
package com.ignou.springboot.learning;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SpringbootLearningApplication
{
    public static void main(String[] args)
    {
        SpringApplication.run(SpringbootLearningApplication.class,
args);
    }
}
```

Spring Boot Project Structure

- Spring Boot has a high degree of flexibility. Thus, it allows us to relocate the scanning elsewhere by specifying the base package manually.



Spring Boot Project Structure For Custom Scan Package

Spring Boot Project Structure

```
package com.ignou.springboot.learning;

import org.springframework.boot.SpringApplication;

import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication(scanBasePackages= {"com.ignou.springboot"})

public class SpringbootLearningApplication
{
    public static void main(String[] args)
    {
        SpringApplication.run(SpringbootLearningApplication.class, args);
    }
}
```

Spring Boot Runners

- Spring Boot provides two runner interfaces named as `ApplicationRunner` and `CommandLineRunner`.
- These interfaces enable to execute a piece of code just after the Spring Boot application is started.
- Both interfaces are Functional Interface. Spring Boot Application starts, to implement either of these functional interfaces and override the single method **run**.
- **Application Runner**
- Spring bean which implements the **ApplicationRunner** interfaces in a Spring Boot Application, the bean gets executed when application is started. `ApplicationRunner` example is as following:

Spring Boot Runners

```
package com.ignou.springboot.learning;

import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.stereotype.Component; @Component

public class ApplicationRunnerImpl implements ApplicationRunner
{
    @Override
    public void run(ApplicationArguments args) throws Exception
    {
        System.out.println("Application Runner Executed.");
    }
}
```

Spring Boot Runners

- The execution result of Spring Boot application having the above component is shown below, and you can observe that the bean implementing `ApplicationRunner` is executed just after the application starts.

```

  ____  _
 / ___|| | | |
| |___| |_| |
 \___|_||_|_|_|
:: Spring Boot ::
  (v2.4.2)
  
```

```

2021-02-09 20:55:24.660 INFO 21044 --- [ restartedMain] c.i.s.l.SpringbootLearningApplication : Starting SpringbootLearningApplication using Java 15.0.1 on LAPTOP-NGL1
2021-02-09 20:55:24.662 INFO 21044 --- [ restartedMain] c.i.s.l.SpringbootLearningApplication : No active profile set, falling back to default profiles: default
2021-02-09 20:55:24.712 INFO 21044 --- [ restartedMain] .e.DevToolsPropertyDefaultsPostProcessor : Devtools property defaults active! Set 'spring.devtools.add-properties'
2021-02-09 20:55:24.713 INFO 21044 --- [ restartedMain] .e.DevToolsPropertyDefaultsPostProcessor : For additional web related logging consider setting the 'logging.level.
2021-02-09 20:55:25.742 INFO 21044 --- [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 8080 (http)
2021-02-09 20:55:25.753 INFO 21044 --- [ restartedMain] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2021-02-09 20:55:25.753 INFO 21044 --- [ restartedMain] org.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/9.0.41]
2021-02-09 20:55:25.855 INFO 21044 --- [ restartedMain] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2021-02-09 20:55:25.856 INFO 21044 --- [ restartedMain] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 1143 ms
2021-02-09 20:55:26.033 INFO 21044 --- [ restartedMain] o.s.s.concurrent.ThreadPoolTaskExecutor : Initializing ExecutorService 'applicationTaskExecutor'
2021-02-09 20:55:26.200 INFO 21044 --- [ restartedMain] o.s.b.d.a.OptionalLiveReloadServer : LiveReload server is running on port 35729
2021-02-09 20:55:26.250 INFO 21044 --- [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8080 (http) with context path ''
2021-02-09 20:55:26.262 INFO 21044 --- [ restartedMain] c.i.s.l.SpringbootLearningApplication : Started SpringbootLearningApplication in 1.943 seconds (JVM running for
Application Runner Executed.
  
```

Execution Result of `ApplicationRunner`

Spring Boot Runners

- **Command Line Runner**
- The ***Command LineRunner*** interface is also meant for the same purpose as **ApplicationRunner** interface. Spring bean which implements the ***CommandLineRunner*** interface in a Spring Boot Application, the bean gets executed when the application is started. ***CommandLineRunner*** example is as following-

```
package com.ignou.springboot.learning;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;
@Component
public class CommandLineRunnerImpl implements CommandLineRunner
{
    @Override
    public void run(String... args) throws Exception
    {
        System.out.println("Command Line Runner executed");
    }
}
```

- ```

/\ / _'-()-\\ \\\\
(() -.-[]-V \\
VV [] | | | | | | | |))))
' [] , [] [] [] \, / / / /
=====|=====|=/././
:: Spring Boot :: (v2.4.2)

2021-02-09 21:01:17.168 INFO 21044 --- [restartedMain] c.i.s.l.SpringbootLearningApplication : Starting SpringbootLearningApplication using Java 15.0.1 on
2021-02-09 21:01:17.168 INFO 21044 --- [restartedMain] c.i.s.l.SpringbootLearningApplication : No active profile set, falling back to default profiles: def
2021-02-09 21:01:17.297 INFO 21044 --- [restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 8080 (http)
2021-02-09 21:01:17.298 INFO 21044 --- [restartedMain] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2021-02-09 21:01:17.298 INFO 21044 --- [restartedMain] org.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/9.0.41]
2021-02-09 21:01:17.315 INFO 21044 --- [restartedMain] o.a.c.c.C.[Tomcat-1].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2021-02-09 21:01:17.315 INFO 21044 --- [restartedMain] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 145
2021-02-09 21:01:17.346 INFO 21044 --- [restartedMain] o.s.s.concurrent.ThreadPoolTaskExecutor : Initializing ExecutorService 'applicationTaskExecutor'
2021-02-09 21:01:17.368 INFO 21044 --- [restartedMain] o.s.b.d.a.OptionalLiveReloadServer : LiveReload server is running on port 35729
2021-02-09 21:01:17.375 INFO 21044 --- [restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8080 (http) with context path ''
2021-02-09 21:01:17.377 INFO 21044 --- [restartedMain] c.i.s.l.SpringbootLearningApplication : Started SpringbootLearningApplication in 0.223 seconds (JVM
Command Line Runner executed

```

+91 - 9319133134 , [query@cftedu.in](mailto:query@cftedu.in)

# Spring Boot Web Application Using Thymeleaf

- The previous sections have described the basic concepts of Spring Boot. This section explains the process of creating a “Hello World” website using the Spring Boot concepts. Required tools and software are as follows:
  - Eclipse
  - Maven
  - Spring Boot
  - JDK 8 or above
- Step 1: Create a Spring Boot Project using Spring Initializr. Visit at <https://start.spring.io/> and generate the Spring Boot project with added dependency. For the project, **Spring Web** and **Thymeleaf** dependency will be used.

# Spring Boot Web Application Using Thymeleaf

The screenshot shows the Spring Initializr web application in a browser. The interface is clean and modern, with a white background and green accents. The top navigation bar includes the Spring logo and the text 'spring initializr'. The main content area is divided into several sections: 'Project' with radio buttons for 'Maven Project' (selected) and 'Gradle Project'; 'Language' with radio buttons for 'Java' (selected), 'Kotlin', and 'Groovy'; 'Spring Boot' with radio buttons for '2.6.0 (SNAPSHOT)', '2.6.0 (M1)', '2.5.4 (SNAPSHOT)', '2.5.3' (selected), '2.4.10 (SNAPSHOT)', and '2.4.9'; and 'Project Metadata' with input fields for 'Group' (com.example), 'Artifact' (demo), and 'Name' (demo). A 'Dependencies' section on the right has a button 'ADD DEPENDENCIES... CTRL + B' and the text 'No dependency selected'. At the bottom, there are three buttons: 'GENERATE CTRL + G', 'EXPLORE CTRL + SPACE', and 'SHARE...'. The browser's address bar shows 'start.spring.io'. The Windows taskbar at the bottom displays the search bar, taskbar icons, and system tray information including '30°C Partly cloudy' and '12:02 AM 8/11/2021'.

Spring Initializr

start.spring.io

Project

- ☒ Maven Project
- ☐ Gradle Project

Language

- ☒ Java
- ☐ Kotlin
- ☐ Groovy

Spring Boot

- ☐ 2.6.0 (SNAPSHOT)
- ☐ 2.6.0 (M1)
- ☐ 2.5.4 (SNAPSHOT)
- ☒ 2.5.3
- ☐ 2.4.10 (SNAPSHOT)
- ☐ 2.4.9

Project Metadata

Group: com.example

Artifact: demo

Name: demo

Dependencies

ADD DEPENDENCIES... CTRL + B

No dependency selected

GENERATE CTRL + G

EXPLORE CTRL + SPACE

SHARE...

Type here to search

30°C Partly cloudy 12:02 AM 8/11/2021

SpringBoot Project Initialization Using Spring Initializr

+91 - 9319133134 , query@cftedu.in



# Spring Boot Web Application Using Thymeleaf

- Step 2: Extract the generated project and Import this into eclipse as maven project. Project structure is shown below.



SpringBoot Project Structure

# Spring Boot Web Application Using Thymeleaf

- Step 3: Add a HomeController into the controller package. HomeController has two endpoints:

- “/”
- “/greeting?name=Jack”

```
package com.example.demo.controller;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam; @Controller

public class HomeController
{
 @RequestMapping(value="/")
 public String home()
 {
 return "home";
 }
 @GetMapping("/greeting")
 public String greeting(@RequestParam(name="name",
required=false,defaultValue="World") String name, Model model)
 {
 model.addAttribute("name", name);
 return "greeting";
 }
}
```

# Spring Boot Web Application Using Thymeleaf -I

- Step 4: Thymeleaf is used as a view technology in this example. For both the end point, add the corresponding HTML template file into src/main/resources/templates

```
<!DOCTYPEhtml>
<html>
<head>
<metacharset="ISO-8859-1"/>
<title>Spring Boot Web Application</title>
</head>
<body>
<h1>Welcome to Thymeleaf Spring Boot web application</h1>
</body>
</html>
```

Home.html

# Spring Boot Web Application Using Thymeleaf - II

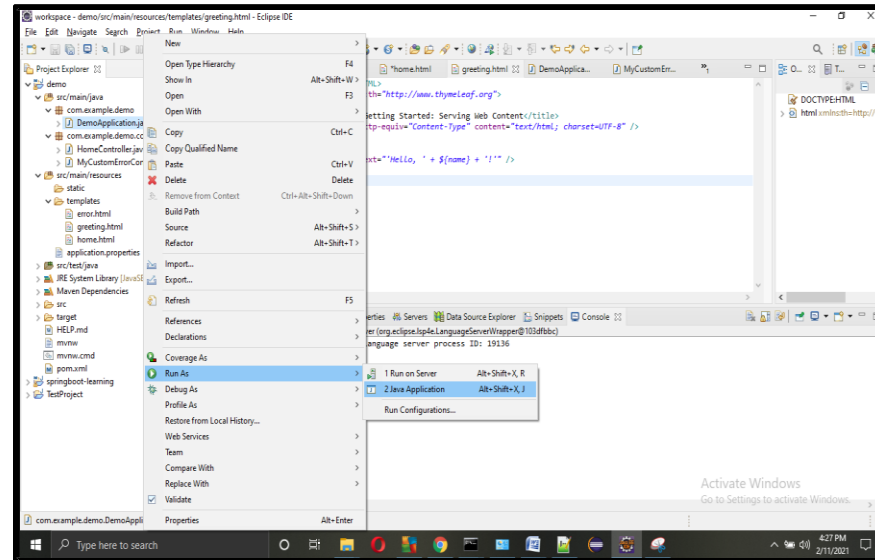
```
<!DOCTYPEHTML>
<htmlxmlns:th="http://www.thymeleaf.org">
<head>
<title>Getting Started: Serving Web Content</title>
<metahttp-equiv="Content-Type"content="text/html; charset=UTF-8"/>
</head>
<body>

<pth:text="'Hello, ' + ${name} + '!'" />
</body>
</html>
```

Greeting.html

# Spring Boot Web Application Using Thymeleaf

- Step 5: Now the project is ready to run. Run **DemoApplication.java** file as Java Application in eclipse as shown below.



Spring Boot Project Execution Into Eclipse

+91 - 9319133134 , query@cftedu.in

# SPRING BOOT DEVTOOLS AND SPRING BOOT ACTUATOR

- Spring Boot DevTools and Spring Boot Actuators are very important modules that enhance the development experience and increase productivity. These modules reduce the developer's effort and reduce the cost of the project.



# Spring Boot DevTools

- Spring Boot DevTools was released with Spring Boot 1.3. DevTools stands for developer tool. The aim of DevTools module is to enhance the application development experience by reducing the development time of the Spring Boot Application.
- During a web application development, a developer changes the code many times and then restarts the application to verify the changed code.
- DevTools reduces the developer effort. It detects the code changes and restarts the application automatically.
- DevTools can be integrated into a Spring Boot application just by adding the following dependency into pom.xml.

# Spring Boot Dev Tools

```
<dependency>
```

```
 <groupId>org.springframework.boot</groupId>
```

```
 <artifactId>spring-boot-devtools</artifactId>
```

```
 <scope>runtime</scope>
```

```
 <optional>true</optional>
```

```
</dependency>
```



# Spring Boot Actuator

- Spring Boot Actuator is a sub-project of the Spring Boot framework. The Actuator provides production-ready features such as application monitoring, Network traffic, State of database and many more.
- Actuator provides production-grade tools. In Spring Boot application, Actuator is primarily used to expose operational information about the running application such as info, health, dump, metrics, env etc.
- It provides **HTTP** and **JMX** endpoints to manage and monitor the Spring Boot application. There are three main features of Spring Boot Actuator-
  - Endpoints
  - Metrics
  - Audit

# Spring Boot Actuator

- **Endpoints:-** The actuator endpoints enable us to monitor and interact with the application. In Spring Boot 2.x, most of the endpoints of the Actuator are disabled.
- By default, only two endpoints **/health** and **/info** are available. Other required endpoints can be enabled by adding ***management.endpoints.web.exposure.include*** property into `application.properties`. **By default, all Actuator endpoints are now placed under the */actuator* path.** Some of the important endpoints are listed below –
  - **/health** provides the health status of the application
  - **/info** provides general information about the application. It might be build information or the latest commit information.
  - **/metrics** provides metrics of application. Returned metrics include generic metrics as well as a custom metric.
  - **/env** provides the current environment properties.

# Spring Boot Actuator

- **Metrics:** Spring Boot integrates the micrometer to provide dimensional metrics. The micrometer is the instrumentation library that empowers the delivery of application metrics from Spring. Metrics of the application can be accessed by **/metrics** endpoint.
- **Audit:** Spring Boot provides a flexible audit framework that publishes events to an Audit Event Repository. It automatically publishes the authentication events if spring-security is in execution.

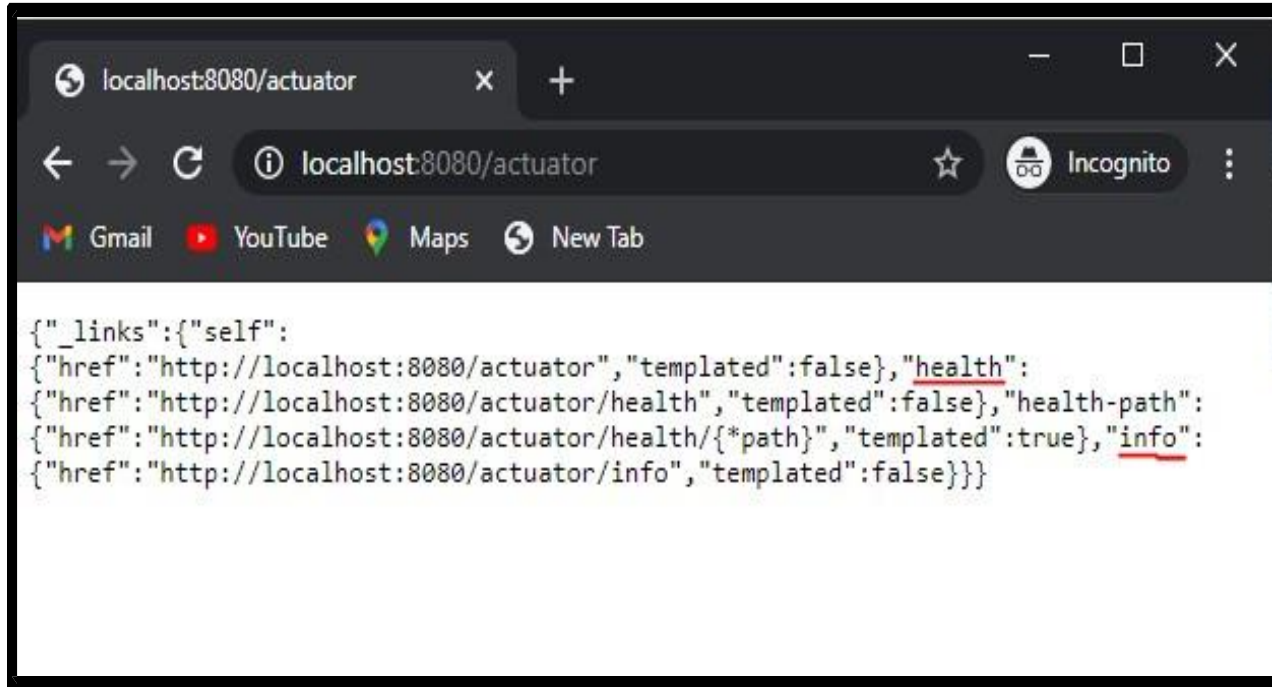
# Spring Boot Actuator Example

- The required steps to integrate the Spring Boot Actuator into the previously developed website into section 9.2.5. Add the spring-boot-starter-actuator dependency into pom.xml –

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

- Start the web application and access the **http://localhost:8080/actuator** endpoint. By default, only two end points are enabled

# Spring Boot Actuator Example



```
{ "_links": { "self": { "href": "http://localhost:8080/actuator", "templated": false }, "health": { "href": "http://localhost:8080/actuator/health", "templated": false }, "health-path": { "href": "http://localhost:8080/actuator/health/{*path}", "templated": true }, "info": { "href": "http://localhost:8080/actuator/info", "templated": false } } }
```

# SPRING BOOT- APPLICATION PROPERTIES

- Application Properties enable us to work in different environments such as Prod, Dev, Test. This section explains how to set up and use properties in Spring Boot via Java configuration and `@PropertySource`.
- **Command Line Properties**
  - Spring Boot Environment properties can be passed via command-line properties. Command-line properties take precedence over the other property sources. The following screenshot shows the command line property **`--server.port=9090`** to change the port number



```
Command Prompt
C:\demo\target>java -jar demo-0.0.1-SNAPSHOT.jar --server.port=9090
```

- **Note:** Double hyphen (--) can be used as a delimiter to pass multiple command-line properties.

# SPRING BOOT- APPLICATION PROPERTIES

- **Properties File**
- By default, Spring Boot can access configurations available in an **application.properties** file kept under the classpath. The application.properties file should be kept in **src/main/resources** directory. The sample application.properties file.

```
server.port = 9090
spring.application.name = demoservice
```

Property server.port changes the port number on which the web application runs.

# SPRING BOOT- APPLICATION PROPERTIES

- **YAML File**
- Spring Boot also supports YAML based properties configuration. Properties file application.yml can be used instead of application.properties. **YAML is a convenient format for specifying hierarchical configuration data.** The application.yml file also should be kept in **src/main/resources** directory. Sample application.yml file is shown below-

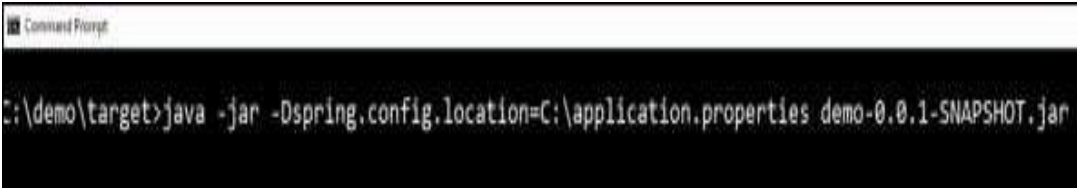
```
server:
port: 9090
spring:
application:
name: demoservice
```



# Externalized Properties

- Spring Boot supports keeping properties file at any location. We can externalize the properties file so that if any property is changed, a new build is not required. Just application restart will take effect on the changed property. While executing the application jar file, the following command-line argument is used to specify the location of the property file.

**-Dspring.config.location = C:\application.properties**



```
C:\demo\target>java -jar -Dspring.config.location=C:\application.properties demo-0.0.1-SNAPSHOT.jar
```

# @Value annotation

- Properties, defined for environment or application, can be fetched in java code using
- **@Value** annotation. Syntax to use @Value annotation is shown below –

```
@Value("${property_key}")
```

- If a property **spring.application.name** is defined into application.properties or application.yml, it can be accessed into java code using as-

```
@Value("${spring.application.name}")
String appName;
```

- While running, if the property is not found, an exception is thrown. The default value can be set while fetching the value using @Value annotation as follows-

```
@Value("${property_key:default_value}")
```

# Active Profile

- An application is executed in multiple environments such as test, development, production etc. To modify application.properties file based on the environment is not an ideal approach. There must be multiple properties files corresponding to each environment. At run time, Spring Boot must be able to select the desired environment properties file.
- Spring Boot supports different properties based on the Spring active profile. Consider that there are separate properties file for each environment as shown below:

## **application.properties**

```
server.port = 9090
spring.application.name = demoservice
```

## **application-dev.properties**

```
server.port = 9090
spring.application.name = demoservice
```

## **application-prod.properties**

```
server.port = 9090
spring.application.name = demoservice
```

# Active Profile

- An application is executed in multiple environments such as test, development, production etc. To modify application.properties file based on the environment is not an ideal approach. There must be multiple properties files corresponding to each environment. At run time, Spring Boot must be able to select the desired environment properties file.
- Spring Boot supports different properties based on the Spring active profile. Consider that there are separate properties file for each environment as shown below:

## **application.properties**

```
server.port = 9090
spring.application.name = demoservice
```

## **application-dev.properties**

```
server.port = 9090
spring.application.name = demoservice
```

## **application-prod.properties**

```
server.port = 9090
spring.application.name = demoservice
```

# Active Profile

- By default, Spring Boot uses application.properties file. But Spring Boot allows to set the active profile and corresponding properties file that is used by Spring Boot. The following command shows how to set an active profile while starting an application from the command-line.
- While running an application from eclipse IDE, the active profile can be set as shown in the screenshot.



```
Command Prompt
C:\demo\target>java -jar demo-0.0.1-SNAPSHOT.jar --spring.profiles.active=dev
```

# Spring Active Profile in application.yml

- YAML file allows active profile parameters to be kept into a single application.yml file. Unlike multiple application.properties files, there is a single application.yml. Delimiter (---) is used to separate each profile in an application.yml file. Sample application.yml file is shown with active profile dev and prod.

```
server:
port: 9090
spring:
application:
name: demoservice

server:
port: 8080
spring:
config:
activate:
on-profile: dev
application:
name: demoservice

```

```
server:
port: 8080
spring:
config:
activate:
on-profile: prod
application:
name: demoservice
```

# RUNNING SPRING BOOT APPS FROM COMMAND LINE

- This section describes a couple of ways to run a Spring Boot app from a command line in a terminal window. Later it explains how to package the app as war and which can be deployed on any application server such as Apache Tomcat, WebLogic, JBoss, etc.
- The **Spring Boot Maven** plugin is the recommended tool to build, test and package the Spring Boot Application code. The plugin is configured by adding it into pom.xml.

```
<build>
<plugins>
...
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
...
</plugins>
</build>
```

# RUNNING SPRING BOOT APPS FROM COMMAND LINE

- The plugin comes with lots of convenient features such as –
  - It can package all our dependencies (including an embedded application server if needed) in a single, runnable fat jar/war
  - It resolves the correct dependency versions



# Running the Code with Maven in Exploded Form

- The Spring Boot Maven plugin has the ability to automatically deploy the web application in an embedded application server. If **spring-boot-starter-web** dependency has been included, plugin knows that Tomcat is required to run the code. On execution of **mvnspring-boot:run** command in the project root folder, the plugin reads the pom configuration. If a web application container is required, the plugin triggers the download of Apache Tomcat and initializes the startup of Tomcat. The command to run the Spring Boot application using maven plugin is as following –

```
mvnspring-boot:run
```

- Execution of the above command starts the Spring Boot application and the produced log is shown

# Running the Code with Maven in Exploded Form

- The Spring Boot Maven plugin has the ability to automatically deploy the web application in an embedded application server. If **spring-boot-starter-web** dependency has been included, plugin knows that Tomcat is required to run the code. On execution of **mvnspring-boot:run** command in the project root folder, the plugin reads the pom configuration. If a web application container is required, the plugin triggers the download of Apache Tomcat and initializes the startup of Tomcat. The command to run the Spring Boot application using maven plugin is as following –

```
mvnspring-boot:run
```

- Execution of the above command starts the Spring Boot application and the produced log is shown

# Running the Code as a Stand-Alone Packaged Application

- Once the development phase is over, the application is moved to production, and we need to package the application. Just include the **Spring Boot Maven** plugin and execute the following command in order to package the application.

```
mvn clean package spring-boot:repackage
```

- The execution of the above command packages the application and produces the file into the target folder. The generated jar file can be executed using the following command.

```
java -jar <File Name>
```

- As you can notice, that `-cp` option and main class has been skipped into `java -jar` command because the Spring Boot Maven plugin takes care of all these configurations into the manifest file.

# Application Packaging as WAR and Deployment on Tomcat

- By default, Spring Boot builds a standalone Java application that can run as a desktop application. The standalone Java application is not suitable for the environment where installation of a new service or manual execution of an application is not allowed, such as Production environment.
- The Servlet containers require the applications to meet some contracts to be deployed. For Tomcat the contract is the Servlet API 3.0. This section considers the example used in section 9.2.5 and explains how the application can be packaged as WAR and deployed into Tomcat.
- First, change the packaging type as war into pom.xml with the following content.

**<packaging>war</packaging>**

# Application Packaging as WAR and Deployment on Tomcat

- Initialize the Servlet context required by Tomcat by extending the *SpringBootServletInitializer* class:

```

:: Spring Boot :: (v2.4.2)

2021-03-17 19:07:45.957 INFO 25676 --- [main] c.example.restdemo.RestdemoApplication : Starting RestdemoApplication using Java 11.0.10 on LAPTOP-NGL13PO with PID 25676 (D:\Rahul\workspace\springboot-hibernate\unit_9\section_9.5\restdemo\target\classes started by Aditiya Singh in D:\Rahul\workspace\springboot-hibernate\unit_9\section_9.5\restdemo)
2021-03-17 19:07:45.961 INFO 25676 --- [main] c.example.restdemo.RestdemoApplication : No active profile set, falling back to default profiles: default
2021-03-17 19:07:47.672 INFO 25676 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 8080 (http)
2021-03-17 19:07:47.687 INFO 25676 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2021-03-17 19:07:47.688 INFO 25676 --- [main] org.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/9.0.41]
2021-03-17 19:07:47.690 INFO 25676 --- [main] o.a.catalina.core.AprLifecycleListener : Loaded Apache Tomcat Native library [1.2.26] using APR version [1.7.0].
2021-03-17 19:07:47.690 INFO 25676 --- [main] o.a.catalina.core.AprLifecycleListener : APR capabilities: IPv6 [true], sendfile [true], accept filters [false], random [true].
2021-03-17 19:07:47.690 INFO 25676 --- [main] o.a.catalina.core.AprLifecycleListener : APR/OpenSSL configuration: useAprConnector [false], useOpenSSL [true]
2021-03-17 19:07:47.696 INFO 25676 --- [main] o.a.catalina.core.AprLifecycleListener : OpenSSL successfully initialized [OpenSSL 1.1.1i 8 Dec 2020]
2021-03-17 19:07:47.918 INFO 25676 --- [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2021-03-17 19:07:47.918 INFO 25676 --- [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 1889ms
2021-03-17 19:07:48.156 INFO 25676 --- [main] o.s.s.concurrent.ThreadPoolTaskExecutor : Initializing ExecutorService 'applicationTaskExecutor'
2021-03-17 19:07:48.388 INFO 25676 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8080 (http) with context path=/
2021-03-17 19:07:48.402 INFO 25676 --- [main] c.example.restdemo.RestdemoApplication : Started RestdemoApplication in 3.098 seconds (JVM running for 3.868s)

```

# Application Packaging as WAR and Deployment on Tomcat - II

```
@SpringBootApplication
public class DemoApplication extends SpringBootServletInitializer
{
 public static void main(String[] args)
 {
 SpringApplication.run(DemoApplication.class, args);
 }
}
```

- By default, generated war file name includes version number also. The Name of the generated war can be modified with following –

```
<build>
 <finalName>${artifactId}</finalName>
 ...
</build>
```

# Application Packaging as WAR and Deployment on Tomcat - III

- Execute the following command in order to build the Tomcat deployable war if artifact id is **demo**, war file generated at target/demo.war. Follow the below steps in order to deploy the generated war file on Tomcat.
  - Download Apache Tomcat and unpack it into the tomcat folder.
  - Copy the generated war file from target/demo.war to tomcat/webapps/demo.war
  - Go to bin dir of Tomcat and start the tomcat using **catalina.bat start** (on windows) or **catalina.sh start** (on Unix)
  - Access the application using <http://localhost:8080/demo>

# SUMMARY

- This session has described Spring Boot facilitates the developers to create a Spring based web application with minimum lines of code since it provides auto configuration out- of-the-box. This unit has explained the following:
  - A class with the main method and annotated with **@SpringBootApplication** is the entry point of the Spring Boot application.
  - Spring Boot starters are the dependency descriptor that addresses the problem of compatible versions of dependency management. It is also a one-stop-shop for all the Spring and related technology that you need.
  - Spring Boot provides two runner interfaces named as ***ApplicationRunner*** and ***CommandLineRunner***. These functional interfaces enable you to execute a piece of code just after the Spring Boot application is started.



# SUMMARY

- Spring Boot **DevTools** important features such as Automatic restart, Live Reload, Property Defaults such as cache disable and allows us to update pages without restarting the application
- **Actuator** provides production-ready features such as application monitoring, Network traffic, State of database and many more. Without any implementation, Actuator provides production-grade tools.
- By default, only two endpoints **/health** and **/info** are available. Other required endpoints can be enabled by adding `management.endpoints.web.exposure.include` property into application. Properties.

*Thank  
You*



**MCS-220**

**Unit-10 -Web Technologies**

**CONFIGURATION OF HIBERNATE(ORM)**

# Topics to be covered

- Introduction
- Hibernate Overview
- Hibernate Configuration with Annotation
- Hibernate CRUD (Create, Read, Update, and Delete) Features
- Hibernate Get Entity – get vs load
- Hibernate Insert Entity – persist, save and saveOrUpdate
- Hibernate Query Language (HQL)
- Spring Data JPA Overview
- Summary

# Introduction

- Object-relational mapping (ORM, O/RM, and O/R mapping tool) is a programming technique for converting data between incompatible type systems using object-oriented programming languages. ORM makes it possible to perform **CRUD (Create, Read, Update and Delete)** operations without considering how those objects relate to their data source. It manages the mapping details between a set of objects and the underlying database. It hides and encapsulates the changes in the data source itself. Thus, when data sources or their APIs change, only ORM change is required rather than the application that uses ORM.
- Hibernate is a pure Java object-relational mapping (ORM) and persistence framework which maps the POJOs to relational database tables. The usage of Hibernate as a persistence framework enables the developers to concentrate more on the business logic instead of making efforts on SQLs and writing boilerplate code.

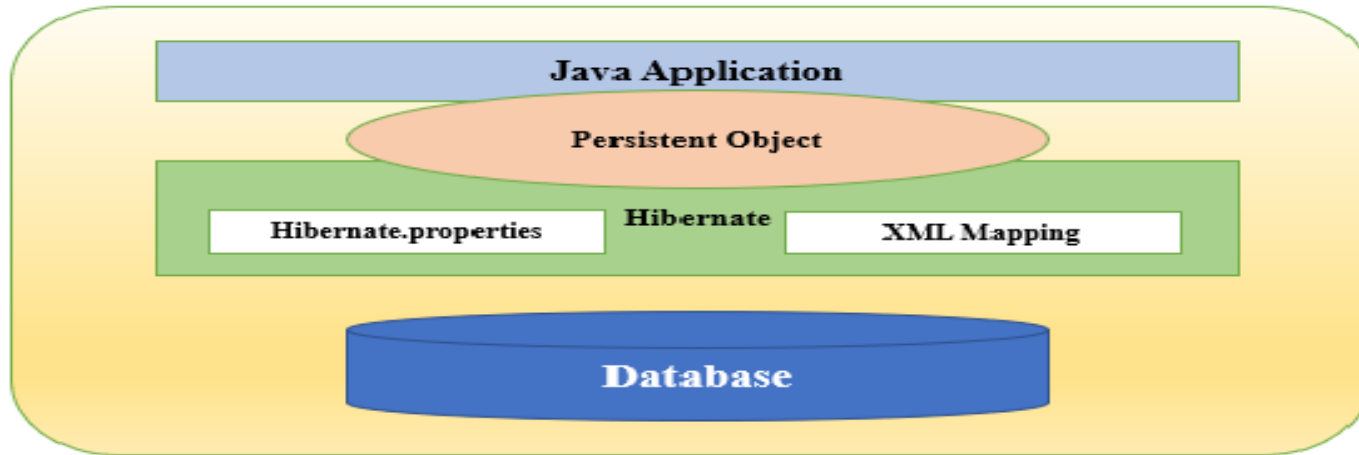
# Hibernate Overview

- Hibernate is an open-source Java persistence framework created by Gavin King in 2011. It simplifies the development of Java applications to interact with databases. Hibernate is a lightweight, powerful and high-performance ORM (Object Relational Mapping) tool that simplifies data creation, data manipulation and data access.
- **The Java Persistence API (JPA) is a specification that defines how to persist data in Java applications.** Hibernate is a popular ORM which is a standard implementation of the JPA specification with a few additional features specific to Hibernate. Hibernate lies between Java Objects and database servers to handle all persistence and retrieval of those objects using appropriate mechanisms and patterns.



# Hibernate Architecture

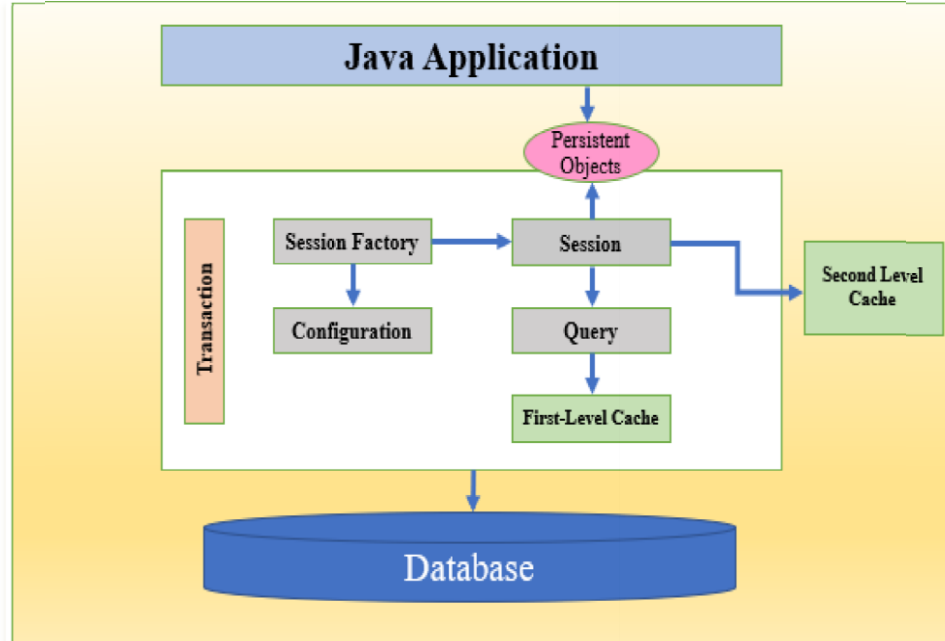
- Hibernate has four-layered architecture. The Hibernate includes many objects such as persistent objects, session factory, session, connection factory, transaction factory, transaction etc. High-level architecture diagram.



**Hibernate Architecture**

# Hibernate Architecture

- A detailed view of the Hibernate Application Architecture with some of the core classes.



Hibernate Detailed Architectural View



# HIBERNATE CONFIGURATION WITH ANNOTATION

- This section explains various annotations available in Hibernate. Later a “Hello World” hibernate example will be created using explained annotation.
- **@Entity**
- In JPA, POJOs are entities that represent the data that can be persisted into a database. An entity represents the **table** in a database. Every instance of an entity represents a **row** in the table. Let’s say there is a POJO named Student, which represents the data of **Student**.

```
@Entity(name = “student”)
public class Student
{

}
```

# HIBERNATE CONFIGURATION WITH ANNOTATION

- **@Id**
- The primary key uniquely identifies the instances of the entity. The primary key is a must for each JPA entity. The **@Id** annotation is used to define the primary key in a JPA entity. Identifiers are generated using **@GeneratedValue** annotation. There are four strategies to generate id. The *strategy* element in **@GeneratedValue** can have value from any of **AUTO**, **TABLE**, **SEQUENCE**, or **IDENTITY**.

```
@Entity
public class Student
{
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 @Column(name = "id")
 private int id;
}
```

# HIBERNATE CONFIGURATION WITH ANNOTATION

- **@Table**
- By default, the name of the table in the database will be the same as the entity name.
- **@Table** annotation is used to define the name of the table in the database.

```
@Entity
@Table(name="STUDENT")
public class Student
{
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 @Column(name = "id")
 private int id;
}
```

# HIBERNATE CONFIGURATION WITH ANNOTATION

- **@Column**
- Similar to @Table annotation, @Column annotation is used to provide the details of columns into the database. Check the other elements defined into @Column, such as **length**, **nullable**, **unique**.

```
@Entity
@Table(name="STUDENT")
public class Student
{
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 @Column(name = "id")
 private int id;
 @Column(name = "firstname", length=50, nullable=false, unique=false)
 private String firstName;
}
```

# HIBERNATE CONFIGURATION WITH ANNOTATION

- **@Transient**
- @Transient annotation is used to make a field into an entity as non-persistent. For example, the age of a student can be calculated from the date of birth. Thus, age field can be made as non-persistent into Student class as shown below-

```
@Entity
@Table(name="STUDENT")
public class Student
{
 @Id
 @GeneratedValue(strategy =
 GenerationType.IDENTITY)
 @Column(name = "id")
 private int id;
 @Column(name = "firstname", length=50, nullable=false, unique=false)
 private String firstName;
 @Column(name = "lastname", length=50, nullable=true, unique=false)
 private String lastName;
 @Transient private int age;
}
```

# HIBERNATE CONFIGURATION WITH ANNOTATION

- **@Temporal**
- The types in the `java.sql` package such as `java.sql.Date`, `java.sql.Time`, `java.sql.Timestamp` are in line with SQL data types. Thus, its mapping is straightforward, and either the **@basic** or **@column** annotation can be used. The type **java.util.Date** contains both date and time information. This is not directly related to any SQL data type. For this reason, another annotation is required to specify the desired SQL type. **@Temporal** annotation is used to specify the desired SQL data type. It has a single element *TemporalType*. *TemporalType* can be **DATE**, **TIME** or **TIMESTAMP**. The type of `java.util.Calendar` also requires **@Temporal** annotation to map with the corresponding SQL data type.

# HIBERNATE CONFIGURATION WITH ANNOTATION

```
@Entity
@Table(name="STUDENT")
public class Student
{
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY) @Column(name = "id")
 private int id;

 @Column(name = "firstname", length=50, nullable=false, unique=false)
 private String firstName;

 @Column(name = "lastname", length=50, nullable=true, unique=false)
 private String lastName; @Transient
 private int age;

 @Temporal(TemporalType.DATE)
 @Column(name = "dateofbirth", nullable = false)
 private Date dob;
}
```

# HIBERNATE CONFIGURATION WITH ANNOTATION

- The types from the *java.time* package in **Java 8** is directly mapped to corresponding SQL types. So there's no need to explicitly specify *@Temporal* annotation:
  - *LocalDate* is mapped to *DATE*
  - *Instant*, *LocalDateTime*, *OffsetDateTime* and *ZonedDateTime* are to *TIMESTAMP*
  - *LocalTime* and *OffsetTime* are mapped to *TIME*



# HIBERNATE CRUD (CREATE, READ, UPDATE AND DELETE) FEATURES

- This section describes the persistence context. It also explains how hibernate uses the concept of persistence context to solve the problem of managing the entities at runtime.
- The persistence context sits between the application and database store. Persistence context keeps monitoring the managed entity and marks the entity as dirty if some modification has been done during a transaction. Persistence context can be considered as a staging area or first-level cache where all the loaded and saved objects to the database reside during a session.

# HIBERNATE CRUD (CREATE, READ, UPDATE AND DELETE) FEATURES

- The instance of ***org.hibernate.Session*** represents the persistence context in Hibernate. Similarly, the instance of ***javax.persistence.EntityManager*** represents the persistence context in JPA. When Hibernate is used as a JPA provider, EntityManager interface is used to perform database related operations. In this case, basically ***java.persistence.EntityManager*** wraps the session object. Hibernate Session has more capability than JPA EntityManager. Thus, sometimes it is useful to work directly with Session.

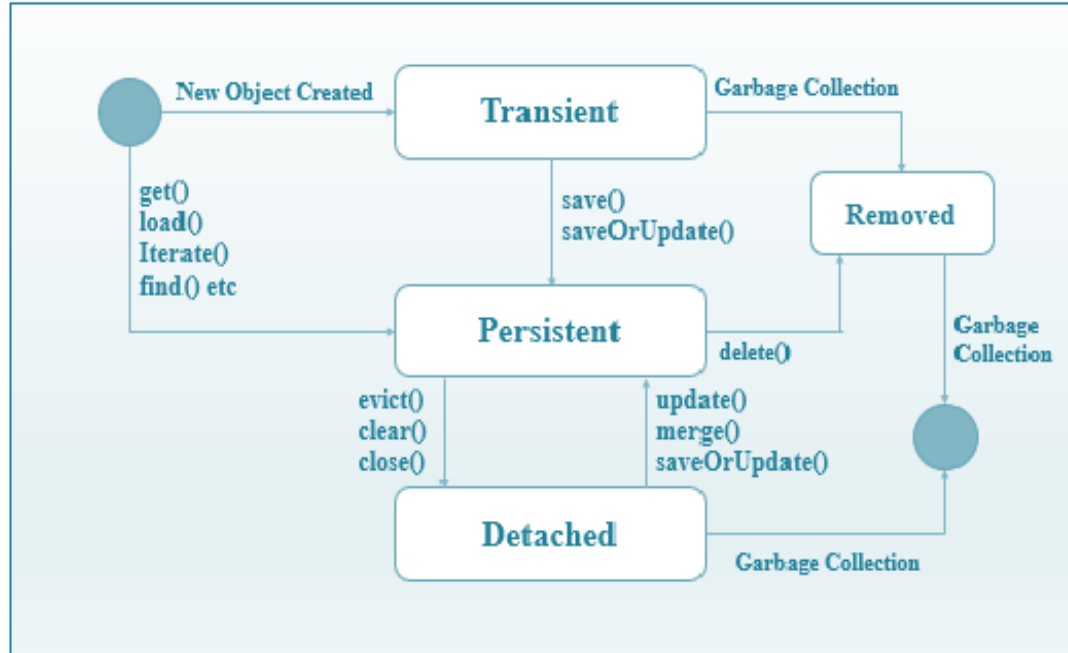
# HIBERNATE CRUD (CREATE, READ, UPDATE AND DELETE) FEATURES

- **Hibernate Entity Lifecycle States**
- Hibernate works with POJO. Without any Hibernate specific annotation and mapping, Hibernate does not recognize these POJO's. Once properly annotated with required annotation, hibernate identifies them and keeps track of them to perform database operations such as create, read, update and delete. These POJOs are considered as mapped with Hibernate. An instance of a class mapped with Hibernate can be any of the four persistence states which are known as hibernate entity lifecycle states and depicted.
  - Transient
  - Persistent
  - Detached
  - Removed

# HIBERNATE CRUD (CREATE, READ, UPDATE AND DELETE) FEATURES

- **Transient:** An object of a POJO class is in a transient state when created using a new operator in Java. In the transient state, the object is not related to any database table. An object in a transient state is not managed by session or Entity Manager. The transient state object is not tracked for any modification from the persistence context. Thus, any modification of a transient state object does not impact the database.

# HIBERNATE CRUD (CREATE, READ, UPDATE AND DELETE) FEATURES



Hibernate Entity Lifecycle States

# HIBERNATE CRUD (CREATE, READ, UPDATE AND DELETE) FEATURES

- **Persistent:** When a Transient entity is saved, it is associated with a unique session object and it enters into a Persistent state. A Persistent state entity is a representation of a row into a database, although the row might not exist in the database yet; upon flushing the session, the entity is guaranteed to have a corresponding row into the database. Session manages the Persistent state objects and keeps track of every modification done. Session propagates all the modifications into the database automatically. The Hibernate session's following methods make an entity into Persistent state.
  - session.save()
  - session.update()
  - session.saveOrUpdate()
  - session.lock()
  - session.merge()

# HIBERNATE CRUD (CREATE, READ, UPDATE AND DELETE) FEATURES

```
public class PersistentStateExample
{
 public static void main(String[] args)
 {
 Transaction txn = null;

 try (Session sess = HibernateUtil.getSessionFactory().openSession())
 {
 txn = sess.beginTransaction(); // start a transaction
 SimpleDateFormat dateFormat = new SimpleDateFormat("dd-MM-yyyy");
 Date dob = dateFormat.parse("10-09-1995");

 Student student = new Student("Rahul", "", "rahulsingh@gmail.com", dob); // 1
 sess.saveOrUpdate(student); // 2
 List<Student> students = sess.createQuery("from Student", Student.class).list();
 students.forEach(s -> System.out.println(s)); // 3
 student.setLastName("Singh"); // 4
 students = sess.createQuery("from Student", Student.class).list();
 students.forEach(s -> System.out.println(s)); // 5
 txn.commit(); // 6
 }

 catch (Exception e)
 {
 if (txn != null)
 {
 txn.rollback();
 }

 e.printStackTrace();
 }
 }
}
```

# HIBERNATE CRUD (CREATE, READ, UPDATE AND DELETE) FEATURES

- Output:-

```
Hibernate: drop table if exists student
Feb 24, 2021 1:01:59 PM org.hibernate.resource.transaction.backend.jdbc.internal.DdlTransactionIsolatorNonJtaImpl getIsolatedConnection
INFO: HHH10001501: Connection obtained from JdbcConnectionAccess [org.hibernate.engine.jdbc.env.internal.JdbcEnvironmentInitiator$ConnectionProviderJ
Hibernate: create table student (id integer not null auto_increment, dateofbirth date not null, email varchar(255), firstname varchar(50) not null, l
Feb 24, 2021 1:01:59 PM org.hibernate.resource.transaction.backend.jdbc.internal.DdlTransactionIsolatorNonJtaImpl getIsolatedConnection
INFO: HHH10001501: Connection obtained from JdbcConnectionAccess [org.hibernate.engine.jdbc.env.internal.JdbcEnvironmentInitiator$ConnectionProviderJ
Feb 24, 2021 1:02:00 PM org.hibernate.tool.schema.internal.SchemaCreatorImpl applyImportSources
INFO: HHH000476: Executing import script 'org.hibernate.tool.schema.internal.exec.ScriptSourceInputNonExistentImpl@77d680e6'
Hibernate: insert into student (dateofbirth, email, firstname, lastname) values (?, ?, ?, ?)
Feb 24, 2021 1:02:00 PM org.hibernate.hql.internal.QueryTranslatorFactoryInitiator initiateService
INFO: HHH000397: Using ASTQueryTranslatorFactory
Hibernate: select student0_.id as id1_0, student0_.dateofbirth as dateofbi2_0, student0_.email as email3_0, student0_.firstname as firstnam4_0, s
Student [id=1, firstName=Rahul, lastName=, age=0, dob=Sun Sep 10 00:00:00 IST 1995, email=rahulsingh@gmail.com]
Hibernate: update student set dateofbirth=?, email=?, firstname=?, lastname=? where id=?
Hibernate: select student0_.id as id1_0, student0_.dateofbirth as dateofbi2_0, student0_.email as email3_0, student0_.firstname as firstnam4_0, s
Student [id=1, firstName=Rahul, lastName=Singh, age=0, dob=Sun Sep 10 00:00:00 IST 1995, email=rahulsingh@gmail.com]
```



# HIBERNATE CRUD (CREATE, READ, UPDATE AND DELETE) FEATURES

- Check the following code into the above example code.
  - The ***student*** object created using the ***new*** operator is in a transient state.
  - session.saveOrUpdate() method makes the ***student*** object into persistent state. Once the object is in persistent state, Session keeps track of any modification and automatically flushes the changes into the database.
  - All the records of the student table are being rendered. Notice that at this point lastName as empty for id 1.
  - The student's last name is updated. The student entity is in the persistent state, and Session keeps track of this change and updates the database automatically. The update statement is issued and can be found in the log.

# HIBERNATE CRUD (CREATE, READ, UPDATE AND DELETE) FEATURES

- All the records of the student table are being rendered. Check at this point lastName is updated for id 1.
- The transaction is committed, and all required changes are synced with the database.

# HIBERNATE CRUD (CREATE, READ, UPDATE AND DELETE) FEATURES

- **Detached:** On call of any session method such as `clear()`, `evict()` or `close()` make the persistent state objects into the detached state. Session does not keep track of any modification to detached state objects. Detached entity has a corresponding row in the database but changes to the entity will not be reflected in the database.
- In order to persist the changes of entity, detached entities must be re-attached to the valid Hibernate Session. The detached state entity can be re-attached to the Hibernate Session with the call of following methods-
  - `session.update()`
  - `session.merge()`
  - `session.saveOrUpdate()`

# HIBERNATE CRUD (CREATE, READ, UPDATE AND DELETE) FEATURES

```
public class DetachedStateExample
{
 public static void main(String[] args)
 {
 try
 {
 Session sess = HibernateUtil.getSessionFactory().openSession(); SimpleDateFormat dateFormat = new SimpleDateFormat("dd-MM-yyyy"); Date dob =
 dateFormat.parse("10-09-1995");
 Student student = new Student("Rahul", "", "rahulsingh@gmail.com", dob);

 sess.saveOrUpdate(student); // student entity is in persistent state List<Student> students = sess.createQuery("from Student",
 Student.class).list();

 sess.close(); // session close() makes student entity in detached state. students.forEach(s -> System.out.println(s)); student.setLastName("Singh"); // This
 change is not tracked by Hibernate

 sess = HibernateUtil.getSessionFactory().openSession(); sess.update(student); // session update() makes the detached object into
 persistent state.

 students = sess.createQuery("from Student", Student.class).list(); students.forEach(s -> System.out.println(s));
 sess.close();
 }
 catch (Exception e)
 {
 e.printStackTrace();
 }
 }
}
```

# HIBERNATE CRUD (CREATE, READ, UPDATE AND DELETE) FEATURES

- **Removed:** Once a persistent object passes through session's delete() method, it enters into Removed state. When an application commits the session the entries in the database that correspond to remove entities are deleted. At this state, java instance exists but any change made to the instance will not be saved into the database. If the removed entities are not referenced anywhere, then it is eligible for garbage collection.
- The following sections explain the various Hibernate operations and differences between the same types of operations.

# Hibernate Get Entity – get vs load

- The Hibernate provides `session.load()` and `session.get()` methods to fetch the entity by id. This section explains both the methods with examples and differences between them.
- **Session load() method:** There are several overloaded methods of `load()` into the Hibernate Session interface. Each `load()` method requires the object's primary key as an identifier to load the object from the database. Along with the identifier, hibernate also requires to know which class or entity name to use to find the object. Some important overloaded methods are as follows:
  - `public Object load(Class<T>theClass, Serializable id)` throws `HibernateException`
  - `public Object load(String entityName, Serializable id)` throws `HibernateException`
  - `public void load(Object object, Serializable id)` throws `HibernateException`

# Hibernate Get Entity – get vs load

- **Session get() method:** The get() method is similar to the load() method to fetch the entity from the database. Like load() method, get() methods also take an identifier and either an entity name or a class. Some important overloaded methods are as follows:
  - `public Object get(Class<T>clazz, Serializable id)` throws `HibernateException`
  - `public Object get(String entityName, Serializable id)`  
throws `HibernateException`

# Hibernate Get Entity – get vs load

- **Difference between load() and get() methods**
- Although load() and get() methods perform the same task, still the differences exist in their return values in case the identifier does not exist in the database.
  - The get() method returns NULL if the identifier does not exist.
  - The load() method throws a runtime exception if the identifier does not exist



# Hibernate Insert Entity – persist, save and saveOrUpdate

- The Session interface into Hibernate provides a couple of methods to transit an object from transient state or detached state to persistent state e.g. `persist()`, `save()`, and `saveOrUpdate()`. These methods are used to store an object in a database. There are significant differences among these methods.
- **Session `persist()` method:** The *`persist()`* method is used to add a record into the database. This method adds a new entity instance to persistent context. On call of *`persist()`* method, the transit state object moves into a persistent state object but not yet saved to the database. The insert statement generates only upon committing the transaction, flushing or closing the session. This method has a return type as **void**. The semantics of *`persist()`* method is as follows:

# Hibernate Insert Entity – persist, save and saveOrUpdate

- A transient state object becomes a persistent object, and the operation cascades all of its relation with cascade=PERSIST or cascade=ALL. The spec does not say that persist() method will generate the **Id** right away but on commit or flush, **non-null Id** is guaranteed. The persist() method does not promise the Id as non-null just after calling this method. Thus one should not rely upon it.
- The persist() method has no impact on persistent objects, but the operation still cascaded all of its relation with cascade=PERSIST or cascade=ALL.
- The persist() method on detached objects throws an exception, either upon calling this method or upon committing or flushing the session.
- A sample example explains the above points with output. For better understanding, have hands-on with the following code.

# Hibernate Insert Entity – persist, save and saveOrUpdate

```
public class EntityPersistExample
{
 public static void main(String[] args)
 {
 try
 {
 Session sess = HibernateUtil.getSessionFactory().openSession();
 Transaction txn = sess.beginTransaction();
 SimpleDateFormat dateFormat = new SimpleDateFormat("dd-MM-yyyy");
 Date dob = dateFormat.parse("10-09-1995");
 Student student = new Student("Abhishek", "Singh", "abhisheksingh@gmail.com", dob);
 sess.persist(student); // student entity is transitioned from transit -> persistent
 sess.persist(student); // No impact
 sess.evict(student); // student entity is transitioned from persistent -> detached

 sess.persist(student); // Exception. Since persistent state object is being persisted.
 txn.commit();
 sess.close();
 }
 catch (Exception e)
 {
 e.printStackTrace();
 }
 }
}
```

# Hibernate Insert Entity – persist, save and saveOrUpdate

- Output:-

```
INFO: HHH000400: Using dialect: org.hibernate.dialect.MySQL5Dialect
Hibernate: drop table if exists student
Feb 27, 2021 5:45:56 PM org.hibernate.resource.transaction.backend.jdbc.internal.DdlTransactionIsolatorNonJtaImpl getIsolatedConnection
INFO: HHH10001501: Connection obtained from JdbcConnectionAccess [org.hibernate.engine.jdbc.env.internal.JdbcEnvironmentInitiator$ConnectionProvider
Hibernate: create table student (id integer not null auto_increment, dateofbirth date not null, email varchar(255), firstname varchar(50) not null, l
Feb 27, 2021 5:45:56 PM org.hibernate.resource.transaction.backend.jdbc.internal.DdlTransactionIsolatorNonJtaImpl getIsolatedConnection
INFO: HHH10001501: Connection obtained from JdbcConnectionAccess [org.hibernate.engine.jdbc.env.internal.JdbcEnvironmentInitiator$ConnectionProvider
Feb 27, 2021 5:45:57 PM org.hibernate.tool.schema.internal.SchemaCreatorImpl applyImportSources
INFO: HHH000476: Executing import script 'org.hibernate.tool.schema.internal.exec.ScriptSourceInputNonExistentImpl@77d680e6'
Hibernate: insert into student (dateofbirth, email, firstname, lastname) values (?, ?, ?, ?)
javax.persistence.PersistenceException: org.hibernate.PersistentObjectException: detached entity passed to persist: com.org.ignou.entity.Student
 at org.hibernate.internal.ExceptionConverterImpl.convert(ExceptionConverterImpl.java:154)
 at org.hibernate.internal.ExceptionConverterImpl.convert(ExceptionConverterImpl.java:181)
 at org.hibernate.internal.ExceptionConverterImpl.convert(ExceptionConverterImpl.java:188)
 at org.hibernate.internal.SessionImpl.firePersist(SessionImpl.java:807)
 at org.hibernate.internal.SessionImpl.persist(SessionImpl.java:785)
 at com.org.ignou.EntityPersistExample.main(EntityPersistExample.java:26)
Caused by: org.hibernate.PersistentObjectException: detached entity passed to persist: com.org.ignou.entity.Student
 at org.hibernate.event.internal.DefaultPersistEventListener.onPersist(DefaultPersistEventListener.java:127)
 at org.hibernate.event.internal.DefaultPersistEventListener.onPersist(DefaultPersistEventListener.java:62)
 at org.hibernate.internal.SessionImpl.firePersist(SessionImpl.java:800)
```

# Hibernate Insert Entity – persist, save and saveOrUpdate

- **Session save() method:** The **save()** method is similar to the **persist()** method which stores the record into the database. This is the original Hibernate Session method which does not conform to JPA specification. The **save()** method differs from the **persist()** method in terms of its implementation. The documentation of this method states that at first, it assigns the generated id to the identifier and then persists the entity. The **save()** method returns the **Serializable** value of this identifier. The semantics of **save()** method is as follows:
  - A transient state object becomes a persistent object and the operation cascades all of its relation with **cascade=PERSIST** or **cascade=ALL**. At first it assigns the generated id to the identifier and then persists the entity.
  - The **save()** method has no impact on persistent objects, but the operation still cascaded all of its relations with **cascade=PERSIST** or **cascade=ALL**.
  - The **save()** method on detached instances creates a new persistent instance. It assigns the new generated Id to the identifier, which results in a duplicate record in the database upon committing or flushing.

# Hibernate Insert Entity – persist, save and saveOrUpdate

- Example:-

```
public class EntitySaveExample
{
 public static void main(String[] args)
 {
 try
 {
 Session sess = HibernateUtil.getSessionFactory().openSession(); Transaction txn = sess.beginTransaction();

 SimpleDateFormat dateFormat = new SimpleDateFormat("dd-MM-yyyy"); Date dob = dateFormat.parse("10-09-1995");

 Student student = new Student("Abhishek", "Singh", "abhisheksingh@gmail.com", dob); Integer id1 = (Integer) sess.save(student); //student
 entity is transitioned from transit ->
 persistent
 System.out.println("Id1: "+id1);

 Integer id2 = (Integer) sess.save(student); //No impact System.out.println("Id2: "+id2);
 sess.evict(student); // student entity is transitioned from persistent -> detached

 Integer id3 = (Integer) sess.save(student); // New Id generated and assigns it to identifier System.out.println("Id3: "+id3);
 txn.commit();
 sess.close();
 }
 catch (Exception e)
 {
 e.printStackTrace();
 }
 }
}
```

# Hibernate Insert Entity – persist, save and saveOrUpdate

- Output:-

```
INFO: HHH10001003: Autocommit mode: false
Feb 27, 2021 7:30:21 PM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl$PooledConnections <init>
INFO: HHH000115: Hibernate connection pool size: 20 (min=1)
Feb 27, 2021 7:30:21 PM org.hibernate.dialect.Dialect <init>
INFO: HHH000400: Using dialect: org.hibernate.dialect.MySQL5Dialect
Hibernate: drop table if exists student
Feb 27, 2021 7:30:22 PM org.hibernate.resource.transaction.backend.jdbc.internal.DdlTransactionIsolatorNonJtaImpl getIsolatedConnection
INFO: HHH10001501: Connection obtained from JdbcConnectionAccess [org.hibernate.engine.jdbc.env.internal.JdbcEnvironmentInitiator$ConnectionProviderJ
Hibernate: create table student (id integer not null auto_increment, dateofbirth date not null, email varchar(255), firstname varchar(50) not null, l
Feb 27, 2021 7:30:22 PM org.hibernate.resource.transaction.backend.jdbc.internal.DdlTransactionIsolatorNonJtaImpl getIsolatedConnection
INFO: HHH10001501: Connection obtained from JdbcConnectionAccess [org.hibernate.engine.jdbc.env.internal.JdbcEnvironmentInitiator$ConnectionProviderJ
Feb 27, 2021 7:30:22 PM org.hibernate.tool.schema.internal.SchemaCreatorImpl applyImportSources
INFO: HHH000476: Executing import script 'org.hibernate.tool.schema.internal.exec.ScriptSourceInputNonExistentImpl@77d680e6'
Hibernate: insert into student (dateofbirth, email, firstname, lastname) values (?, ?, ?, ?)
Id1: 1
Id2: 1
Hibernate: insert into student (dateofbirth, email, firstname, lastname) values (?, ?, ?, ?)
Id3: 2
```

Activate Windows



# Hibernate Insert Entity – persist, save and saveOrUpdate

- **Session saveOrUpdate() method:** This method either performs **save()** or **update()** on the basis of identifier existence. E.g. If an identifier exists for the instance, **update()** method will be called otherwise **save()** will be performed. The **saveOrUpdate()** method handles the cases where we need to save a detached instance. Unlike the **save()** operation on detached instances, the **saveOrUpdate()** method does not result in a duplicate record. Similar to **update()**, this method is used to reattach an instance to the session. This method can be considered as a universal tool to make an object persistent regardless of its state, whether it is transient or detached.



# Hibernate Insert Entity – persist, save and saveOrUpdate

```
public class EntitySaveOrUpdateExample
{
 public static void main(String[] args)
 {
 try
 {
 Session sess = HibernateUtil.getSessionFactory().openSession();
 Transaction txn = sess.beginTransaction();
 SimpleDateFormat dateFormat = new SimpleDateFormat("dd-MM-yyyy");
 Date dob = dateFormat.parse("10-09-1995");
 Student student = new Student("Abhishek", "Singh", "abhisheksingh@gmail.com", dob);
 sess.saveOrUpdate(student); // student entity is transitioned from transit -> persistent
 System.out.println("Id1: " + student.getId());
 sess.saveOrUpdate(student); // No impact
 System.out.println("Id2: " + student.getId());

 sess.evict(student); // student entity is transitioned from persistent -> detached

 sess.saveOrUpdate(student); // Since Id exist, thus only update operation is performed on
 detached entity
 System.out.println("Id3: " + student.getId());
 txn.commit();
 sess.close();
 }
 catch (Exception e)
 {
 e.printStackTrace();
 }
 }
}
```

# Hibernate Insert Entity – persist, save and saveOrUpdate

- Output:- As in output, you can see that saveOrUpdate() operation does not result in duplicate records even for the detached entity . Thus, this operation can be used as a universal tool to make an object persistent regardless of its state.

```
INFO: HHH000400: Using dialect: org.hibernate.dialect.MySQL5Dialect
Hibernate: drop table if exists student
Feb 27, 2021 11:57:31 PM org.hibernate.resource.transaction.backend.jdbc.internal.DdlTransactionIsolatorNonJtaImpl getIsolatedConnection
INFO: HHH10001501: Connection obtained from JdbcConnectionAccess [org.hibernate.engine.jdbc.env.internal.JdbcEnvironmentInitiator$ConnectionProvide
Hibernate: create table student (id integer not null auto_increment, dateofbirth date not null, email varchar(255), firstname varchar(50) not null,
Feb 27, 2021 11:57:32 PM org.hibernate.resource.transaction.backend.jdbc.internal.DdlTransactionIsolatorNonJtaImpl getIsolatedConnection
INFO: HHH10001501: Connection obtained from JdbcConnectionAccess [org.hibernate.engine.jdbc.env.internal.JdbcEnvironmentInitiator$ConnectionProvide
Feb 27, 2021 11:57:32 PM org.hibernate.tool.schema.internal.SchemaCreatorImpl applyImportSources
INFO: HHH000476: Executing import script 'org.hibernate.tool.schema.internal.exec.ScriptSourceInputNonExistentImpl@77d680e6'
Hibernate: insert into student (dateofbirth, email, firstname, lastname) values (?, ?, ?, ?)
Id1: 1
Id2: 1
Id3: 1
Hibernate: update student set dateofbirth=?, email=?, firstname=?, lastname=? where id=?
```

Activate Windows

# Hibernate Query Language (HQL)

- Hibernate Query Language (HQL) is an object-oriented query language. HQL is similar to the database SQL language. The main difference between HQL and SQL is HQL uses class name instead of table name, and property names instead of column name. HQL queries for Select, Update, Delete and Insert is explained below.
- **HQL Select** query to retrieve the student data of id 1.

```
Query query = session.createQuery("from Student where id = :id ");
query.setParameter("id", "1");
List list = query.list();
```

# Hibernate Query Language (HQL)

- **HQL Update** query to update the student's last name as 'Singh' whose id is 1.

```
Query query = session.createQuery("Update Student set lastName = :lastName where id = :id ");
query.setParameter("lastName", "Singh"); query.setParameter("id", "1");
int result = query.executeUpdate();
```

- **HQL Delete** query to delete the student where id is 1.

```
Query query = session.createQuery("Delete Student where id = :id ");
query.setParameter("id", "1");
int result = query.executeUpdate();
```

# Hibernate Query Language (HQL)

- **HQL Insert**

- In HQL, only the INSERT INTO ... SELECT ... is supported; there is no INSERT INTO ... VALUES. HQL only support insert from another table. For example, insert student records from another student\_backup table. This is also called bulk-insert statement.

```
Query query = session.createQuery("Insert into Student(firstName, lastName, emailId)" +
 "Select firstName, lastName, email from student_backup");
int result = query.executeUpdate();
```

# SPRING DATA JPA OVERVIEW

- A well-structured codebase of a web project consists of various layers such as Controller, Service and DAO (data access object) layer. Each layer has its own responsibilities. While developing a web application, business logic should be focused instead of technical complexity and boilerplate code. The DAO layer usually consists of lots of boilerplate code, and that can be simplified. The simplification reduces the number of artifacts that need to be defined and maintained. It also makes the data access pattern and configuration consistent.
- Different storage technologies require different API's to access data, different configurations and different methods. JPA handles the object-relational mappings and technical complexities in JDBC based database interaction. The JPA provides standardization across SQL data stores. The Java Persistence API (JPA) specification and Spring Data JPA are very popular to simplify the DAO layer. Before the explanation of Spring Data JPA, the following section explains the relationship between Spring Data JPA, JPA and Hibernate.

# SPRING DATA JPA OVERVIEW

- **Spring Data JPA** is one of the many sub-projects of **Spring Data** that simplifies the data access for relational data stores. Spring Data JPA is not a JPA provider; instead it wraps the JPA provider and adds its own features like a no-code implementation of the repository pattern. Spring Data JPA uses Hibernate as the default JPA provider. JPA provider is configurable, and other providers can also be used with Spring Data JPA. Spring Data JPA provides a complete abstraction over the DAO layer into the project.
- Spring Data JPA introduces the concept of **JPA Repositories**. JPA Repositories are a set of Interfaces that defines query methods. It does not require writing native queries anymore. Sometimes custom queries may be required in order to fetch the data from the database. These queries are JPQL (Java Persistence Query Language), not native queries. The advantages of using Spring Data JPA are as follows.

# SPRING DATA JPA OVERVIEW

- **DAO Abstraction with No-Code Repositories**
- Spring Data JPA defines many Repository Interfaces such as ***CrudRepository***, ***PagingAndSortingRepository***, ***JpaRepository*** having methods to store, retrieve, sorted retrieval, paginated result and many more. The interfaces only define query methods and spring provides proxy implementation at runtime. With Spring Data JPA, we don't have to write SQL statement, instead we just need to extend the interface defined by Spring Data JPA for one of the entities. Based on JPA specification, the underlying JPA implementation enables the *Entity* objects and their metadata mapping. It also enables the entity manager who is responsible for persisting and retrieving entities from the database.



# SPRING DATA JPA OVERVIEW

- **Query Methods**

- Another robust and comfortable feature of Spring Data JPA is the Query Methods. Based on the name of methods declared in the repository interfaces are converted to low-level SQL queries at runtime. If the custom query isn't complex, only a method is required to be defined in the repository interface with a name starting with ***find...By***. Here is an example of a Student Repository to find a student by id and list of students based on firstName and lastName.

```
public interface StudentRepository extends CrudRepository<Student, Long>
{
 Optional<Student>findById(Long studentId);
 List<Student>findByFirstNameAndLastName(String firstName, String
lastName);
}
```

# SPRING DATA JPA OVERVIEW

- **Query Methods**

- Another robust and comfortable feature of Spring Data JPA is the Query Methods. Based on the name of methods declared in the repository interfaces are converted to low-level SQL queries at runtime. If the custom query isn't complex, only a method is required to be defined in the repository interface with a name starting with ***find...By***. Here is an example of a Student Repository to find a student by id and list of students based on firstName and lastName.

- The first method corresponds to **select \* from student where id = ?**
- The second method corresponds to **select \* from student where firstName=?andlastName = ?**

# SPRING DATA JPA OVERVIEW

```
public interface StudentRepository extends CrudRepository<Student, Long>
{
 Optional<Student>findById(Long studentId);
 List<Student>findByFirstNameAndLastName(String firstName, String
lastName);
}
```

# SPRING DATA JPA OVERVIEW

- **Seamless Integration**

- Spring Data JPA seamlessly integrates **JPA** into the Spring stack, and its repositories reduce the boilerplate code required to the JPA specification. **Spring Data JPA** also helps your DAO layer integrating and interacting with other Spring based components in your application, like accessing property configurations or auto-wiring the repositories into the application service layer. It also works perfectly with Spring Boot auto-configuration. We only need to provide the data source details and the rest of the things like configuring and using *EntityManager*.

# SUMMARY

- Hibernate as ORM tool and Spring Data JPA, which uses Hibernate as the default JPA provider.
- Many different data storage technologies are available in this world, and each one comes with its own data access API and driver.
  - ORM stands for Object Relational Mapping, where a Java object is mapped to a database table and with APIs. We need to work with objects and not with native queries.
  - The Java Persistence API (JPA) is a specification that defines how to persist data in Java applications
  - The Hibernate architecture elements include such as SessionFactory, Session, Query, First Level Cache and Second Level Cache. The first level cache can't be disabled.

*Thank  
You*



**MCS-220**

**Unit-11 -Web technologies**

**CRUD APPLICATION USING SPRING BOOT AND HIBERNATE**

# Topics to be covered

- Introduction
- Spring Data Repository
- CrudRepository
- PagingAndSortingRepository
- JpaRepository
- Hibernate Association Mappings
- One-to-Many / Many-to-One
- Owning Side and Bi-directionality
- Eager vs Lazy loading
- Cascading Operations
- One-to-One
- Implementation with Foreign key
- Implementation with shared Primary key
- Many-to-Many
- Create records using Spring Boot and Hibernate
- Read Records Using Spring Boot and Hibernate
- Update records using Spring Boot and Hibernate
- Delete records using Spring Boot and Hibernate
- Summary



# Introduction

- Nowadays, there are varieties of data stores such as relational databases, NoSQL databases like MongoDB, Casandraetc, Big Data solutions such as Hadoop etc. Different storage technologies have different configurations, different methods and different API's to fetch the data.
- Spring Data simplifies and makes homogeneous data access technologies for relational and non-relational databases, cloud-based data services and map-reduce frameworks. Spring Data provides an abstraction that allows us to connect in the same way to relational databases and NoSQL databases. Thus, switching can be done effortlessly between data stores.

# Introduction

- **Spring Data JPA** is one of the many sub-projects of **Spring Data** that focuses on simplification of the data access for relational data stores. It is not a JPA provider; instead, it wraps the JPA provider and adds its own features like a **no-code implementation** of the repository pattern. Hibernate is the default JPA provider into Spring Data JPA. Spring Data JPA is very flexible, and other providers can also be configured as JPA providers. Spring Data JPA provides a complete abstraction over the DAO layer into the project.
- The actual strength of Spring Data JPA lies in the **repository abstraction**. The **repository abstraction** enables us to write the business logic code on a higher abstraction level. Developers do not need to worry about DAO layer implementations; instead they just need to learn Spring Data repository interfaces. Spring Data provides an implementation for these repository interfaces out-of-the-box.

# SPRING DATA REPOSITORY

- The previous unit gave us the overview of Spring Data JPA. This section explains the Spring Data Repository to create the persistence layer into the Spring Boot application.
- The ***Spring Data repository*** aims to reduce the boilerplate code required to implement the DAO layer into a project for various persistence stores. Spring Data repository has different interfaces, each having different functionality. The following interfaces in the Spring Data repository have been explained.
  - CrudRepository
  - PagingAndSortingRepository
  - JpaRepository

# SPRING DATA REPOSITORY

- JpaRepository extends PagingAndSortingRepository interface, which extends CrudRepository. Thus, **JpaRepository contains all API of CrudRepository and PagingAndSortingRepository**. The persistent layer can extend any of the above interfaces based on required APIs. The implementation of persistent layers is very easy.

```
@Entity
public class Book
{
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private int id;
 private String title;
 // getter setter
}
```

# SPRING DATA REPOSITORY

- Following Repository provides a simple operation – find a Book based on its title.

```
@Repository
public interface BookRepository extends CrudRepository<Book, Long>
{
 public Book findByTitle(String bookTitle);
}
```

- The Spring Data Repository will auto-generate the implementation based on the method name. Supported keywords inside method name can be found at -  
<https://docs.spring.io/spring-data/data-jpa/docs/current/reference/html/#jpa.query-methods.query-creation>

# CrudRepository

- This interface provides all basic CRUD operation related APIs. The methods provided in CrudRepository are shown in the code.

```
public interface CrudRepository<T, ID> extends Repository<T, ID>
{
 <S extends T> S save(S entity);
 <S extends T> Iterable<S> saveAll(Iterable<S> entities);
 Optional<T> findById(ID id);
 boolean existsById(ID id);
 Iterable<T> findAll();
 Iterable<T> findAllById(Iterable<ID> ids);
 long count();

 void deleteById(ID id);
 void delete(T entity); void deleteAll(Iterable<?
 extends T> entities); void deleteAll();
}
```

# CrudRepository

- `save(...)` - Saves a given entity.
- `saveAll(...)` - Saves all given entities.
- `findById(...)` - Retrieves an entity by its id.
- `existsById(...)` - Returns whether an entity with the given id exists.
- `findAll(...)` - Returns all instances of the type.
- `findAllById(...)` - Returns all instances of the type with the given IDs
- `count(...)` - Returns the number of entities available.
- `deleteById(...)` - Deletes the entity with the given id.
- `delete(...)` - Deletes a given entity.
- `deleteAll(...)` - Deletes the given entities.
- `deleteAll()` - Deletes all entities managed by the repository.

# PagingAndSortingRepository

- PagingAndSortingRepository interface extends CrudRepository interface. It provides the paging and sorting feature apart from CRUD feature. The methods in the interface are shown in the code.

```
public interface PagingAndSortingRepository<T, ID> extends CrudRepository<T, ID>
{
 Iterable<T> findAll(Sort sort);

 Page<T> findAll(Pageable pageable);
}
```

- findAll(Sort sort) - Returns all entities sorted by the given options.
- findAll(Pageable pageable) - Returns a page of entities meeting the paging restriction provided in the pageable object.



# JpaRepository

- JpaRepository extends PagingAndSortingRepository interface, which extends CrudRepository. Thus, **JpaRepository contains all API of CrudRepository and PagingAndSortingRepository.** The methods available in JpaRepository are shown in the code.

```
public interface JpaRepository<T, ID> extends PagingAndSortingRepository<T, ID>,
 QueryByExampleExecutor<T>
{
 @Override
 List<T> findAll();
 @Override List<T> findAll(Sort
 sort);
 @Override
 List<T> findAllById(Iterable<ID> ids);
 @Override
 <S extends T> List<S> saveAll(Iterable<S> entities);
 void flush();
 <S extends T> S saveAndFlush(S entity);
 void deleteInBatch(Iterable<T> entities);
 void deleteAllInBatch();
 T getOne(ID id);
 @Override
 <S extends T> List<S> findAll(Example<S> example);

 @Override
 <S extends T> List<S> findAll(Example<S> example, Sort sort);
}
```

# HIBERNATE ASSOCIATION MAPPINGS

- The previous unit has explained that hibernate can identify POJO classes as persistent only when they are annotated with certain annotations. While making the POJO classes as persistent entities using JPA annotations, we may face situations where two entities can be related and must be referenced from each other, either uni-directional or bi-directional. Association mappings are one of the key features of JPA and Hibernate. That establishes the relationship between two database tables as attributes in your model and allows you to easily navigate the association in your model and JPQL or criteria queries.

# HIBERNATE ASSOCIATION MAPPINGS

- When only one pair of entities contains a reference to the other, the association is **unidirectional**. If the pair of entities contains a reference to each other, then it is referred to as **bi-directional**. Entities can contain references to other entities, either directly as an embedded property or field or indirectly via a collection of some sort (arrays, sets, lists, etc.). There is no impact of unidirectional or bidirectional association on database mapping. **Foreign Key Relationships** are used to represent the associations in the underlying tables.
- JPA and Hibernate have the same association as you are aware of relational databases. The followings are the association that will be described in subsequent sections.
  - one-to-many/many-to-one
  - one-to-one
  - many-to-many

# One-to-Many / Many-to-One

- One-to-Many and Many-to-One relationships are the opposite sides of the same coin and closely related. **One-to-Many** mapping is described as one row in a table and is mapped to multiple rows in another table.
- For the illustration of the One-to-Many relationship, the **Teacher** and their **Courses** will be taken. A teacher can give multiple courses but a course which is given by only one teacher is an example of one-to-many relationship. While another perspective, many courses are given by a teacher, is an example of the many-to-one relationship. Before diving into details of how to map this relationship, let's create the entities.

# One-to-Many / Many-to-One

@Entity

**publicclass** Teacher

{

@Id

@GeneratedValue(strategy = GenerationType.*IDENTITY*)

**private** Long id;

**private** String name;

}

@Entity

**publicclass** Course

{

@Id

@GeneratedValue(strategy = GenerationType.*IDENTITY*)

**privateint**id; **private** String title;

}

# One-to-Many / Many-to-One

- Now, the **Teacher** class should include a list of courses, and mapping will be required to map this relationship into the database. This will be annotated with a `@OneToMany` annotation.

```
@Entity
public class Teacher
{
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private Long id;
 private String name;
 @OneToMany(cascade = CascadeType.ALL)
 private List<Course> courses;
}
```

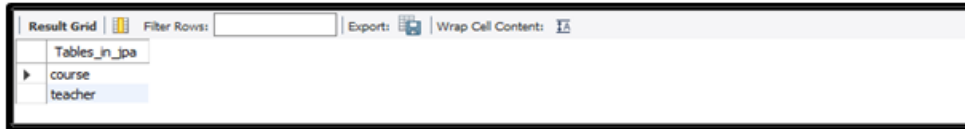
# One-to-Many / Many-to-One

- With the above configuration, Hibernate uses an association table to map the relationship. With the above configuration, three tables named *teacher*, *course* and *teacher\_courses* will be created. But, definitely, we are not looking for such mapping. In order to avoid the third table *teacher\_courses*, **@JoinColumn** annotation is used to specify the foreign key column.

```
@Entity
public class Teacher
{
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private Long id;
 private String name;
 @OneToMany(cascade = CascadeType.ALL)
 @JoinColumn(name = "teacher_id", referencedColumnName = "id")
 private List<Course> courses;
}
```

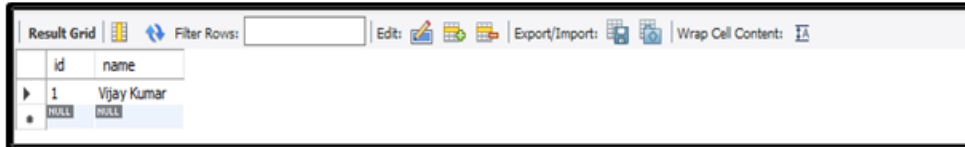
# One-to-Many / Many-to-One

- The tables created into the database and save() operation on teacher object will result in the following into the database



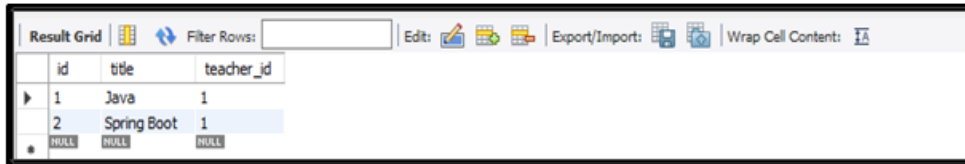
Tables_in_jpa
course
teacher

**Hibernate Created Tables**



id	name
1	Vijay Kumar
NULL	NULL

**Hibernate Created Records Into teacher Table**



id	title	teacher_id
1	Java	1
2	Spring Boot	1
NULL	NULL	NULL

**Hibernate Created Records Into course Table**



# Owning Side and Bi-directionality

- The previous slide has defined the Teacher class as the owning side of the One-to-Many relationship. This is because the Teacher class defines the join column between the two tables. The **Course** is called the referencing side in the relationship.
- Course class can be made as to the owning side of the relationship by mapping the Teacher field with @ManytoOne in the course class instead.

# Owning Side and Bi-directionality

@Entity

**publicclass** Teacher

{

    @Id

    @GeneratedValue(strategy = GenerationType.*IDENTITY*)

**private** Long *id*;

**private** String *name*;

}

@Entity

**publicclass** Course

{

    @Id

    @GeneratedValue(strategy = GenerationType.*IDENTITY*)

**privateintid**; **private** String *title*;

    @ManyToOne(cascade = CascadeType.*ALL*)

    @JoinColumn(name = "*teacher\_id*", referencedColumnName = "*id*")

**private** Teacher *teacher*;

}

# Owning Side and Bi-directionality

- This time **@ManytoOne** annotation has been used instead of **@OnetoMany** annotation. In the above example, it can be observed that the Teacher class does not have a list of courses. Uni-directional relationship has been used here; thus, only one of the entities has reference to another entity. **@ManytoOne** will result similar to **@OnetoMany**.
- **Note:** *It's a good practice to put the owning side of a relationship in the class/table where the foreign key will be held.*
- According to best practice, the second version of code using **@ManytoOne** is better. If you look at the second version of the code, the Teacher class does not offer to access the list of courses. This can be achieved by the bidirectional relationship.

# Eager vs Lazy loading

- It is worth noting that a mapped relationship should not impact the software's memory by putting too many unnecessary entities. For example, consider that the Course is a heavy object, i.e. having too many attributes. For some business logic, all **Teacher** objects are loaded from the database. Business logic does not need to retrieve or use courses in it but **Course** objects are still being loaded alongside the **Teacher** objects.
- Such loading will degrade the performance of applications. Technically, this can be solved by using the Data Transfer Object Design Pattern and retrieving Teacher information *without* the courses.
- JPA has considered all such problems ahead and made **One-to-Many relationships load lazily by default**.

# Eager vs Lazy loading

- Lazy loading means relationships will be loaded when it is actually needed. The execution of **@OnetoMany** relationship with the default fetch type will issue two queries. First is for the Teacher object and the second for the Course objects when it's needed. The log of the query issued by Hibernate is as follows.

```
2021-03-06 17:52:47.095 DEBUG 8124 --- [restartedMain] org.hibernate.SQL : select
teacher0_.id as id1_1_0_, teacher0_.name as name2_1_0_ from teacher teacher0_ where
teacher0_.id=?
2021-03-06 17:52:47.096 TRACE 8124 --- [restartedMain] o.h.type.descriptor.sql.BasicBinder :
binding parameter [1] as [BIGINT] - [1]
2021-03-06 17:52:47.104 TRACE 8124 --- [restartedMain] o.h.type.descriptor.sql.BasicExtractor :
extracted value ([name2_1_0_] : [VARCHAR]) - [Vijay Kumar]
2021-03-06 17:52:47.111 TRACE 8124 --- [restartedMain] org.hibernate.type.CollectionType
:
Created collection wrapper: [com.ignou.hibernate.association.learning.entity.Teacher.courses#1]
2021-03-06 17:52:47.117 DEBUG 8124 --- [restartedMain] org.hibernate.SQL : select
courses0_.teacher_id as teacher_3_0_0_, courses0_.id as id1_0_0_, courses0_.id as id1_0_1_,
courses0_.teacher_id as teacher_3_0_1_, courses0_.title as title2_0_1_ from course courses0_ where
courses0_.teacher_id=?
```

# Eager vs Lazy loading

- While the execution of **@OnetoMany** relationship with fetch type as **Eager** will issue only one query, Log of the query issued by Hibernate is as follows.

```
2021-03-06 18:28:11.107 DEBUG 15728 --- [restartedMain] org.hibernate.SQL : select
teacher0_.id as id1_1_0_, teacher0_.name as name2_1_0_, courses1_.teacher_id as
teacher_3_0_1_, courses1_.id as id1_0_1_, courses1_.id as id1_0_2_, courses1_.teacher_id as
teacher_3_0_2_, courses1_.title as title2_0_2_ from teacher teacher0_ left outer join course
courses1_ on teacher0_.id=courses1_.teacher_id where teacher0_.id=?
2021-03-06 18:28:11.107 TRACE 15728 --- [restartedMain] o.h.type.descriptor.sql.BasicBinder
: binding parameter [1] as [BIGINT] - [1]
```

# Eager vs Lazy loading

- ***Many-to-One* relationships are *eager* by default.** A eager relationship means all related entities are loaded at the same time. The default behavior of fetch type can be changed with fetch attribute as shown below-

```
@OneToMany(mappedBy = "teacher", cascade= CascadeType.ALL, fetch =
FetchType.EAGER)
private List<Course> courses;
```

```
@ManyToOne(fetch = FetchType.LAZY)
private Teacher teacher;
```

# Cascading Operations

- **JPA** operations affect the only entity on which it has been performed. These operations will not affect the other entities that are related to it. For example, In case of Person-Address relationship, the Address entity doesn't have any meaning of its own without a Person entity. Thus, on delete of Person entity, Address entity should also get deleted.
- Cascading is about JPA actions involving one entity propagating to other entities via an association. JPA provides **javax.persistence.CascadeType** enumerated types that
- define the cascade operations. These cascading operations can be defined with any type of mapping i.e. One-to-One, One-to-Many, Many-to-One, Many-to-Many. JPA provides the following cascade type to perform cascading operations.



# Cascading Operations

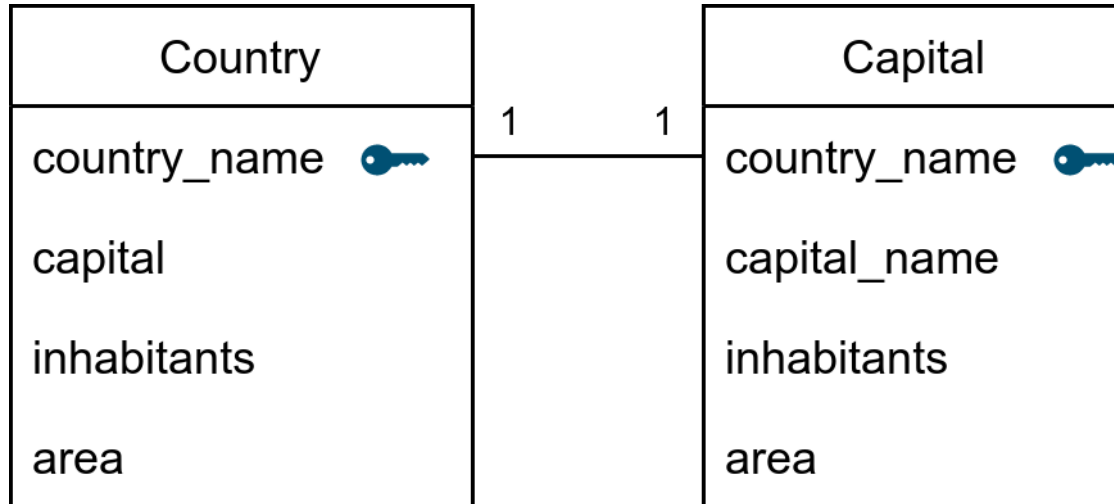
Cascade Type	Description
ALL	CascadeType.ALL propagates <b>all operations</b> including hibernate specific from a parent to a child entity.
PERSIST	CascadeType.PERSIST propagates the <b>save()</b> or <b>persist()</b> operation to the related entities.
MERGE	CascadeType.MERGE propagates only the <b>merge()</b> operation to the related entities.
REFRESH	CascadeType.REFRESH propagates only the <b>refresh()</b> operation to the related entities.
REMOVE	CascadeType.REMOVE removes all related entities association when the owning entity is deleted.
DETACH	CascadeType.DETACH detaches all related entities if owning entity is detached.

# Cascading Operations

- **Hibernate Cascade Type**
- Hibernate provides three additional Cascade Types along with provided by JPA. Hibernate provides *org.hibernate.annotations.CascadeType* enumerated types that define the cascade operations.
  - CascadeType.REPLICATE
  - CascadeType.SAVE\_UPDATE
  - CascadeType.Lock
- Clear understanding of **One-to-Many/Many-to-One** association, as explained above, will help you to understand other associations. The following section will explain the other association as bidirectional with example.

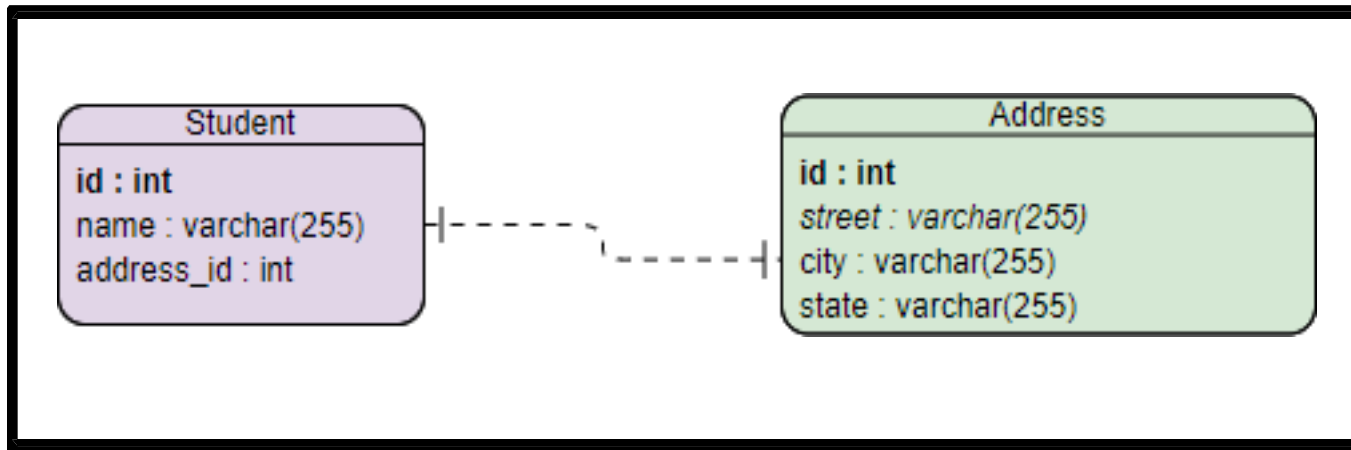
# One-to-One

- **One-to-One** mapping is described as one row in a table mapped to only one row in another table.
- For the illustration of One-to-One relationship, The **Student** and the **Address** have been taken.



# Implementation with Foreign key

- ER diagram of foreign key based one-to-one mapping is shown below image. The address\_id column in Student is the foreign key to the address.



ER Diagram For One-to-One Mapping With Foreign Key

# Implementation with Foreign key

- ER diagram of foreign key based one-to-one mapping is shown below image. The address\_id column in Student is the foreign key to the address.

@Entity

**public** class Student

{

    @Id

    @GeneratedValue(strategy = GenerationType.**IDENTITY**)

**private** Long id;

**private** String name;

    @OneToOne(cascade = CascadeType.**ALL**) @JoinColumn(name="address\_id",  
    referencedColumnName = "id") **private** Address address;

    // getter setter

}

# Implementation with Foreign key

- ER diagram of foreign key based one-to-one mapping is shown below image. The address\_id column in Student is the foreign key to the address.

@Entity

**public** class Student

{

    @Id

    @GeneratedValue(strategy = GenerationType.**IDENTITY**)

**private** Long id;

**private** String name;

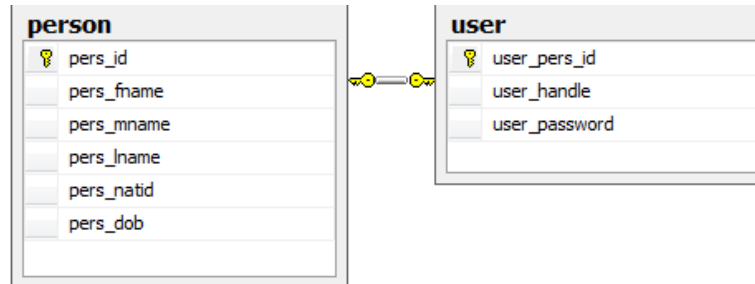
    @OneToOne(cascade = CascadeType.**ALL**) @JoinColumn(name="address\_id",  
    referencedColumnName = "id") **private** Address address;

    // getter setter

}

# Implementation with Foreign key

- The Student and the Address has a one-to-one relationship. Entity Student comprises another entity Address, and it is annotated with **@OneToOne** annotation. Check the cascade attribute into the annotation. Student entity is the owner, and thus, it has **@JoinColumn** annotation on Address entity. **@JoinColumn** is to configure the name of the column in the Student table that maps to the primary key in the address table. If the name attribute is not provided in **@JoinColumn**, Hibernate will follow some rules to select the default name of the column.



# Implementation with Foreign key

- The Address entity is having @OneToOne annotation on another entity Student with attribute mappedBy. The mappedBy attribute is used since the Address is non-owning. Note that on both entities @OneToOne annotations are used since its bidirectional.

```
@Entity
public class Address
{
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private int id; private String street; private String state;

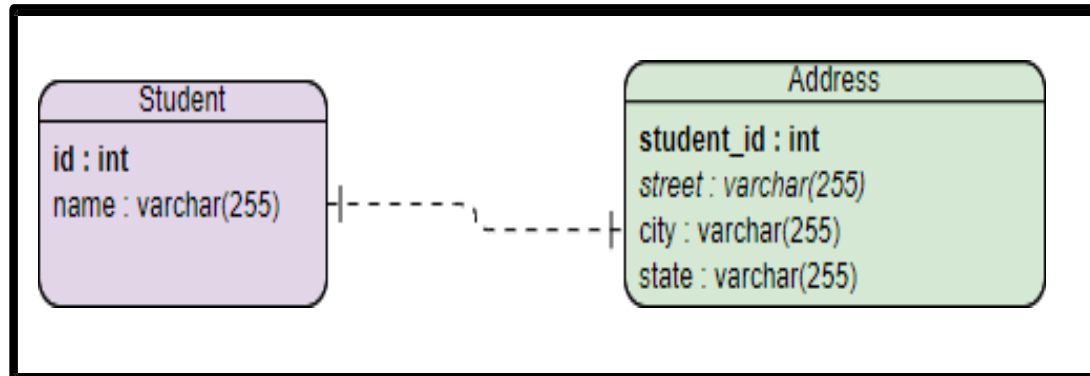
 private String pincode;
 private String country; @OneToOne(mappedBy = "address") private
 Student student;

 // getter setter
}
```



# Implementation with shared Primary key

- In this strategy, instead of creating a new column *address\_id*, we'll mark the **primary key column (*student\_id*) of the *address* table as the foreign key to the *student* table**. ER diagram is shown in Figure 11.6. This strategy optimizes the storage space.



ER Diagram For One-to-One Mapping With Shared Primary Key

# Implementation with shared Primary key

- The **@PrimaryKeyJoinColumn** annotation indicates that the primary key of Student entity is used as the foreign key value for the associated Address entity. The **mappedBy** attribute is used here because the foreign key is now present in the address table.

```
@Entity
public class Student
{

 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private Long id;

 private String name;

 @OneToOne(mappedBy = "student", cascade = CascadeType.ALL)
 @PrimaryKeyJoinColumn
 private Address address;

 // getter setter
}
```

# Implementation with shared Primary key

- The identifier of the Address entity is still having @Id annotation but it no longer uses @GeneratedValue annotation. Instead it references the student\_id column. The @MapsId indicates that the primary key values will be copied from the Student entity.

@Entity

**publicclass** Address

{

@Id

@Column(name ="student\_id")

**privateint**id;

**private** String street;

**private** String state;

**private** String **pin**code;

**private** String **country**;

@OneToOne

@MapsId

@JoinColumn(name="student\_id")

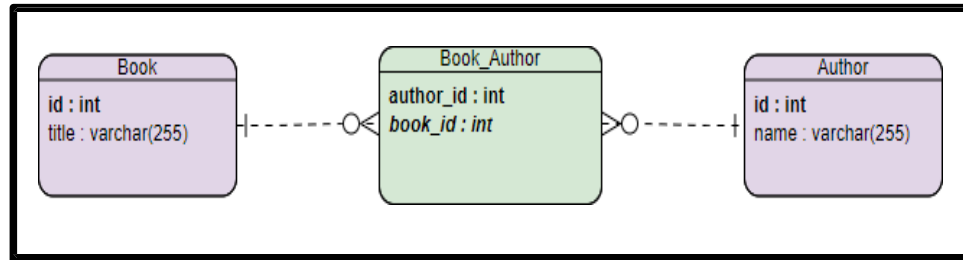
**private** Student **student**;

// getter setter

}

# Implementation with shared Primary key

- **Many-to-Many** mapping is described as many rows in a table mapped to many rows in another table.
- For the illustration of Many-to-Many relationship, **Book** and the **Author** have been taken. A book may be written by multiple writers and a writer will write multiple books. Thus, the Book and Author relationship is many-to-many and is shown as an ER diagram.



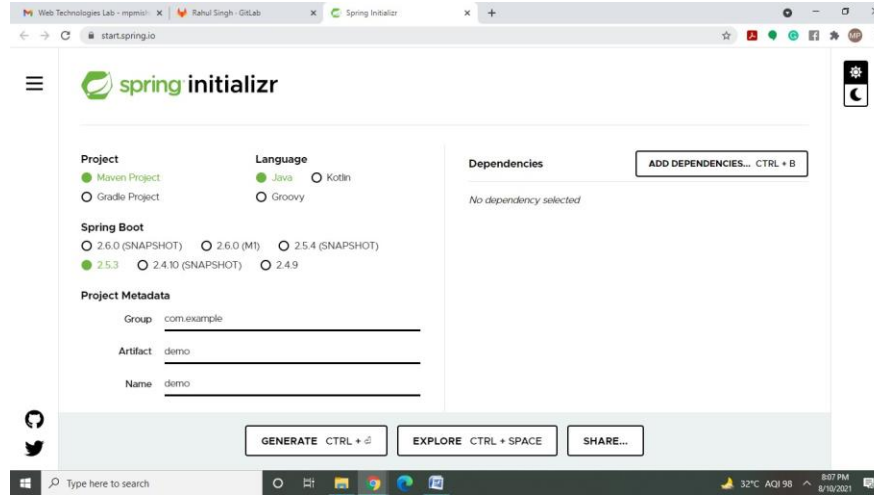
ER Diagram For Many-to-Many Relationship

# CREATE RECORDS USING SPRING BOOT AND HIBERNATE

- The previous unit explained Spring Data JPA and hibernate as the default provider for Spring Data JPA. This unit has elaborated on Spring Data Repository and hibernate association mapping. This section provides the complete Rest Application to create records using Spring Boot and Hibernate by using all the explained concepts. The example considers the many-to-many relationship between **Book** and **Author**. Many books can be written by an Author and many authors can write a book. The same example will be considered for CRUD operation. The required tools and software are as follows:
  - Eclipse IDE
  - Maven
  - Spring Boot
  - Hibernate
  - Mysql
  - Postman

# CREATE RECORDS USING SPRING BOOT AND HIBERNATE

- Visit Spring Initializr (<https://start.spring.io>) in order to generate the spring project. Add the required dependencies and export the project into eclipse as maven project. Modify the data source properties into application.properties file and do some hands-on.



## SpringBoot Project Setup Using Spring Initializr

+91 - 9319133134 , [query@cftedu.in](mailto:query@cftedu.in)

# READ RECORDS USING SPRING BOOT AND HIBERNATE

- This section explains how to read the records and search the records using Spring Boot and Spring Data JPA with Hibernate. The previous section example will be used to explain the read operation. We will just add the new feature into the existing rest application. Add the new features into BookRepository and BookController as shown below in order to test the read records operation.

```
@Repository
public interface BookRepository extends CrudRepository<Book, Long>
{
 public List<Book> findByTitleContaining(String pattern); // 1
 public List<Book> findByTitleContaininsIgnoreCase(String pattern); //2
 public List<Book> findByTitleLike(String pattern); // 3 @Query(“Select b
 from Book b where b.title like %:pattern%”)
 public List<Book> findByTitle(String pattern); // 4
}
```

**BookRepository.java**

# READ RECORDS USING SPRING BOOT AND HIBERNATE

- BookRepository extends the CrudRepository interface, which provides the methods to read all records, specific record by id etc. Here we have added three new methods in order to provide search features. All three methods generate the same **like queries**, but there are some differences.
  - Query methods using Containing, Contains, and IsContaining generate the like query. For example, the query method findByTitleContaining(String pattern) generates the following sql query.  
**SELECT \* FROM books WHERE title LIKE '%<pattern>%'**
  - The **findByTitleContainsIgnoreCase(String pattern)** is similar to **findByTitleContaining(String pattern)** but with case insensitive.



# READ RECORDS USING SPRING BOOT AND HIBERNATE

- Spring Data JPA also provides a Like keyword, but it behaves differently. An example to call the **findByTitle(...)** is shown in BookRepository code in the next paragraph.
- Sometimes we need to create queries that are too complicated for Query Methods or would result in absurdly long method names. In those cases, we can use the @Query annotation to query our database.
  - Named Parameters (**:name**) – Query parameters can be passed as named parameters into the query. Check the example in the above BookRepository.
  - Ordinal Parameters (**?index**) - JPQL also supports ordinal parameters, whose form is **?index**. An example of ordinal parameters is as “**Select b from Book b where b.title like %?1%**”

# UPDATE RECORDS USING SPRING BOOT AND HIBERNATE

- To update the record/records, CrudRepository provides save(...) and saveAll(...) methods. If the entity has an identifier, JPA will perform update operation instead of save operation. Add the following into the BookController.java in order to provide the **update** feature to the rest application.

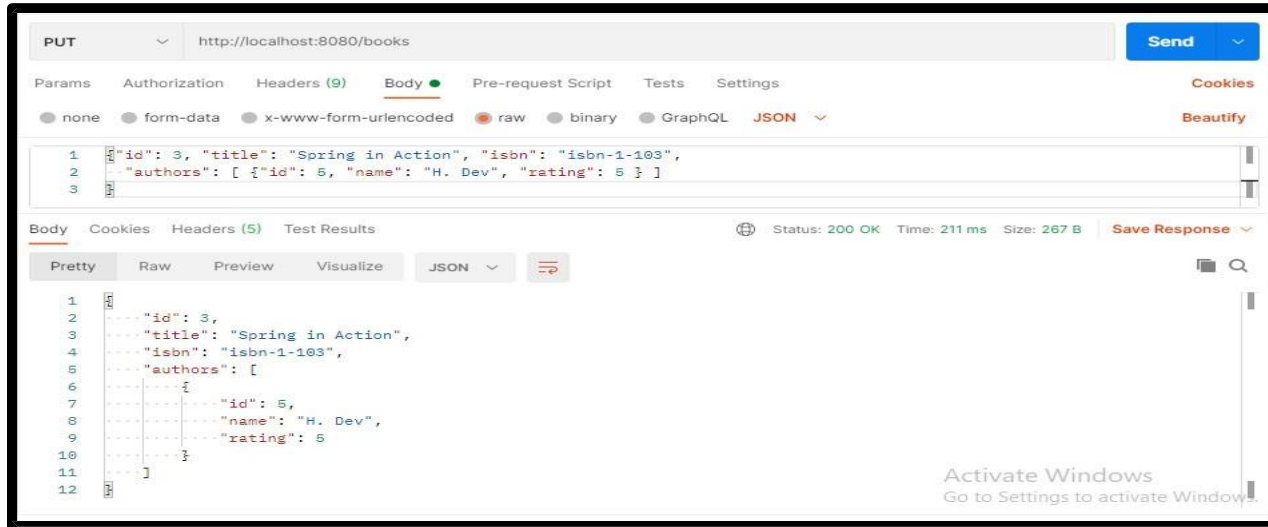
```
BookController.java @RestController @RequestMapping("/books")
 public class BookController
 {

 @Autowired
 private BookRepository bookRepository;

 @PutMapping
 public Book updateBook(@Valid @RequestBody Book b)
 {
 return bookRepository.save(b);
 }
 }
```

# UPDATE RECORDS USING SPRING BOOT AND HIBERNATE

- To test the update feature, use the postman and use the PUT method with a valid Book record into the request body. Following request, update the authors of the given book with id 3.



Update Record Using SpringBoot and Hibernate

# DELETE RECORDS USING SPRING BOOT AND HIBERNATE

- To delete a record or all records, CrudRepository provides deleteById(...), delete(...) and deleteAll(...) methods. Add the following code into the BookController.java in order to provide the **delete** feature into the rest application.

```
@RestController @RequestMapping("/books")
public class BookController
{

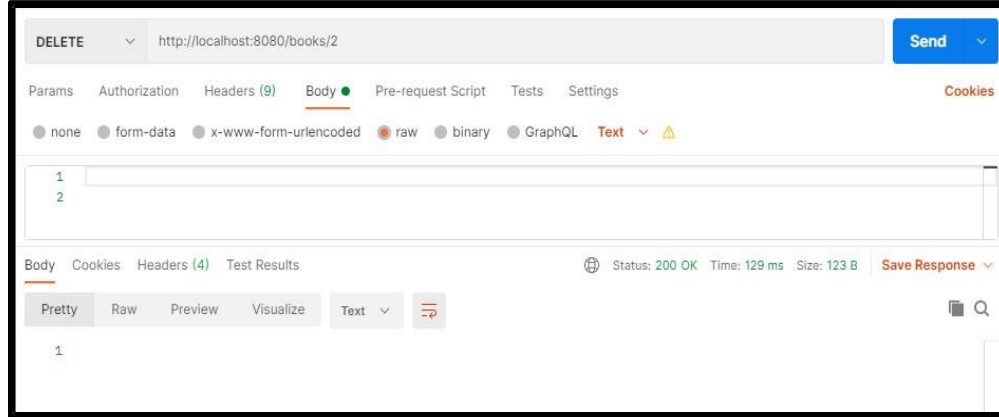
 @Autowired
 private BookRepository bookRepository;

 @DeleteMapping("/{bookId}") @ResponseStatus(value =
 HttpStatus.OK)
 public void deleteBook(@PathVariable(value="bookId", required=true) Long bookId)
 {
 bookRepository.deleteById(bookId);
 }
}
```

**BookController.java**

# DELETE RECORDS USING SPRING BOOT AND HIBERNATE

- The delete operation does not return any body response. It just returns the response status as 200 OK on successful deletion. Use the postman to issue the **DELETE** request, as shown into the screenshot. The following is the delete request to delete the book with id as 2. The response status is **200 OK**, and the response body is empty as expected.



# SUMMARY

- This session has explained Spring Data Repository with a complete example using other hibernate association concepts. For better understanding, start coding by using the examples explained in this unit. The important points are as following:
  - The goal of Spring Data repository abstraction is to significantly reduce the amount of boilerplate code required to implement data access layers for various persistence stores.
  - CrudRepository, PagingAndSortingRepository and JpaRepository have been explained with all available methods.
  - Association mappings are one of the key features of JPA and Hibernate. That establishes the relationship between two database tables as attributes in your model and allows you to navigate the association easily in your model and JPQL or criteria queries.

# SUMMARY

- JPA has considered the related entity loading overhead problems ahead and made One-to-Many relationships load lazily by default.
- The fetch type for Many-to-One relationship is eager by default.
- Cascading is about JPA actions involving one entity propagating to other entities via an association. JPA provides `javax.persistence.CascadeType` enumerated types that define the cascade operations. JPA provides the cascade types as ALL, PERSIST, MERGE, REFRESH, REMOVE, DETACH.
- Query methods using Containing, Contains, and IsContaining generate the like query. For example, query method `findByTitleContaining(String pattern)` generates the sql query as `SELECT * FROM books WHERE title LIKE '%<pattern>%'`
- Query parameters as Named Parameters (`:name`) and Ordinal Parameters (`?index`)

*Thank  
You*





# **MCS 220 Web Technologies**

## **Web Security**

### **Unit – 13 Custom Login Using Security**

# Topics to be Covered

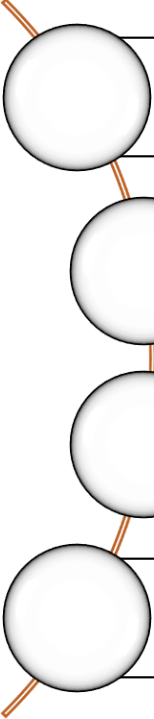
- Introduction
- Custom Login form creation
- Spring Config for Custom Login Form
- Create Request mapping and building Custom Login Form
- Testing Custom Login Form
- Integration Test in Spring Boot Adding
- Logout Support
- Summary

# Introduction

Testing is a very important phase of any application to ensure whether it works as per expected functionality or not.

Integration testing and unit testing play a vital role in making the application error prone.

# Objective



Create custom login page for spring security

Configure the custom login page

Execute the spring boot application with custom login page spring security

Develop logout support in Spring Applications

# Customer Login Form Creation

Customized login form creation and configuration is very easy in spring security. Custom login form in Spring Application is nothing but a simple html or jsp, like other web pages in java. A custom login form enables an enterprise to customize the login form as per their choice of design. It provides the developer to have full control over CSS and processing URL.

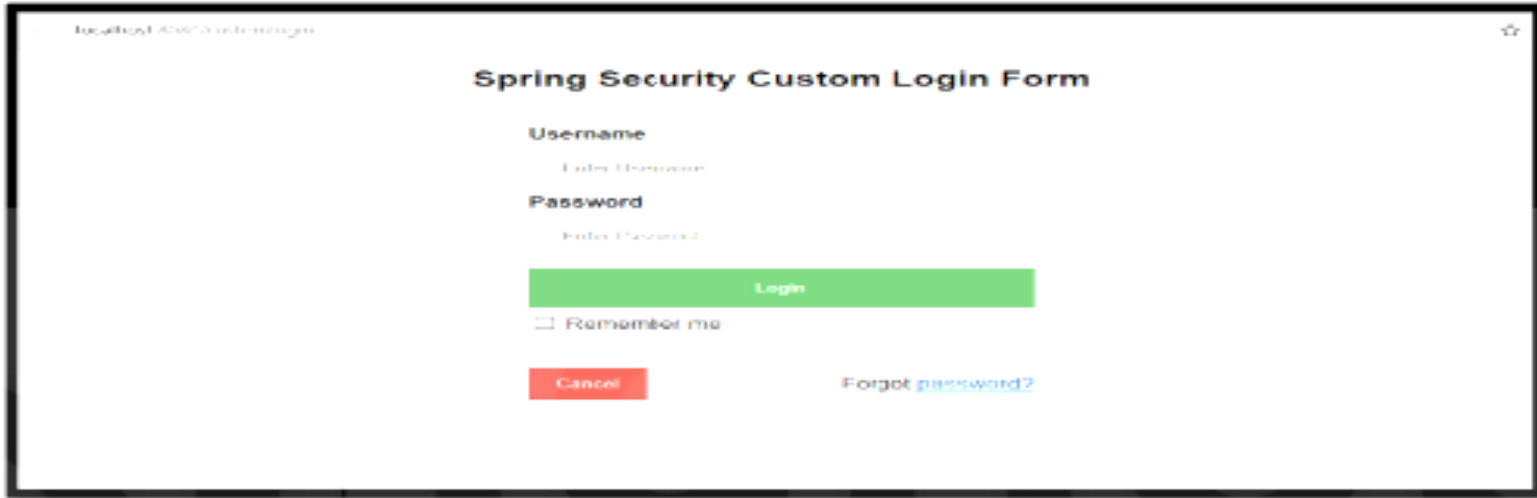
A custom login form must be having below artifacts:

- Form action URL where the form is POST'ed to trigger the authentication process
- Username input field for the username
- Password input field for the password

# Form Action URL Custom login Controller

```
package com.ignou.javabasedspringsecurity.controller;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
@Controller @RequestMapping(value="/customlogin")
public class LoginController { @GetMapping public String
login() { return "customlogin"; } }
```

# Form Action URL Custom login Controller



The screenshot displays a web browser window with the address bar showing 'localhost:8080/actuator/login'. The page title is 'Spring Security Custom Login Form'. The form contains two input fields: 'Username' with a placeholder 'Enter Username' and 'Password' with a placeholder 'Enter Password'. Below these fields is a green 'Login' button. Under the 'Login' button is a checkbox labeled 'Remember me'. At the bottom left is a red 'Cancel' button, and at the bottom right is a link 'Forgot password?'. The browser's address bar also shows a star icon on the right.

# CREATE REQUEST MAPPING AND BUILDING CUSTOM LOGIN FORM

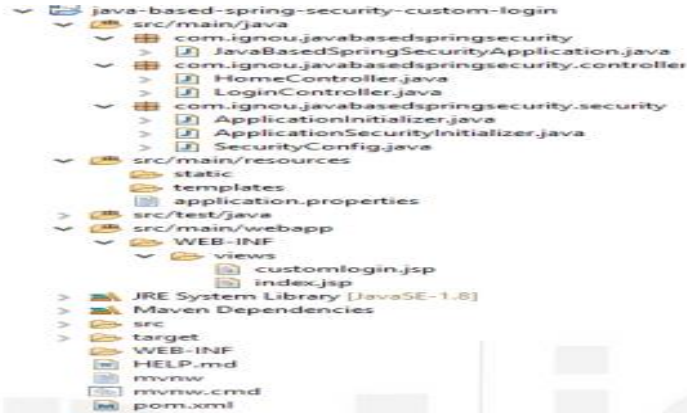
```
<html>
<body>
<div>
<center>Custom Spring Login Form</center>
<form action="/signin" method="post">
<input name="username" type="text" placeholder="Enter Username" />
<input name="password" type="text" placeholder="Enter password" />
<input type="submit" />
</form>
</div>
</body>
</html>
```



# TESTING CUSTOM LOGIN FORM

The execution of spring boot application with custom login form using spring security.

Configuration code and request mapping code has been written 7 Custom Login Using Security in code sample.



# TESTING CUSTOM LOGIN FORM

localhost:8080/customlogin

### Spring Security Custom Login Form

**Username**

**Password**

☒ Remember me

[Forgot password?](#)

localhost:8080/customlogin/error-true

### Spring Security Custom Login Form

**Invalid credentials**

**Username**

**Password**

☒ Remember me

[Forgot password?](#)

# Running the Test Case as a User with Request Post Processor

@Test

**Public void** testAlreadyLoggedInUser() **throws** Exception

```
{
 mvc.perform(get("/").with(user("user"))).andExpect(authenticated().withUsername("user"))
 .andExpect(authenticated().withRoles("USER")) .andExpect(model().attribute("message",
 "User, Spring Security with custom login page implemented Successfully !!"));
}
```

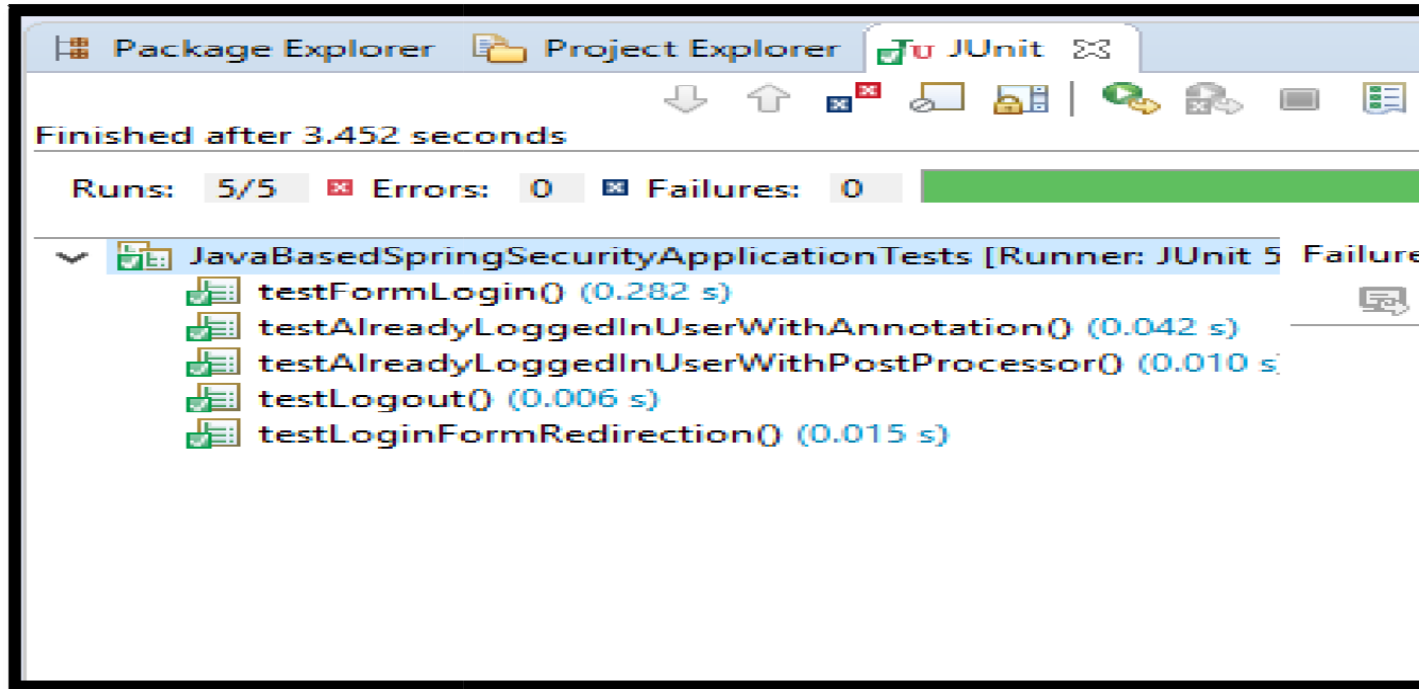
# Security Mock MVC Request Builders

Request Builder interface is provided by Spring MVC Test.

The request builders can be used to create the Mock HttpServletRequest into the test.

Few implementations of Request Builder has been provided by Spring Security to make testing easy and efficient.

# Form Based Authentication Testing



# Summary

Testing is a very important phase of any application to ensure whether it works as per expected functionality.

Customized login from creation and configuration is very easy in spring security.

Custom login form in Spring Application is nothing but a simple html or JSP, web pages in java.

Custom login form must be mapped with controller in order to render the page to the user.

Thank  
You